

Cours Réseaux Avancés

Cours Réseaux Avancés Master 1 ING3IA

جامعة أبو بكر بلقايد
UNIVERSITY OF TLEMCE



Bambrik Ilyas

Table des matières



Objectifs	6
I - Chapitre I Introduction aux réseaux	7
1. Définition d'un réseau	7
2. Équipements réseau	8
3. Topologies réseaux	8
II - Chapitre II Modèle OSI et Modèle TCP / IP	13
1. Modèle OSI (Open Systems Interconnection)	13
2. Couche application	14
3. Couche présentation	14
4. Couche session	15
5. Couche transport	16
6. Couche réseau	17
7. Couche liaison de données	17
8. Couche physique	18
9. Model TCP / IP (Transmission Control Protocol / Internet Protocol)	18
10. Niveaux de fonctionnement des équipements dans le modèle TCP / IP	19
III - Chapitre III Applications et protocoles de la couche transport	20
1. Protocoles de la couche application	20
2. Exercice : Ports et applications réseaux	20
3. Protocoles couche transport	20
4. UDP (User Datagram Protocol)	21
5. TCP (Transmission Control Protocol)	21
6. Entête TCP	23
7. Signification des Bits flag	24

8. Établissement de connexion TCP par "Three way handshake"	24
9. Clôture de connexion	25
10. Acquittement des segments	26
11. Mécanisme Sliding Window	27
12. Problème de l'acquittement cumulative	29
IV - Chapitre IV Adressage dans la Couche Internet	30
1. Couche Internet	30
2. Entête IPv4	30
3. Flags	31
4. Entête IPv6 principale (entête principale)	32
V - Chapitre V Adressage dans la Couche Couche Liaison	35
1. Couche Liaison	35
2. Trame Ethernet	36
3. ARP (Address Resolution Protocol)	37
4. ARP Request (recherche d'une adresse MAC dans le réseau local)	37
5. ARP Reply	38
6. Gratuitous ARP	38
VI - Chapitre VI Internet Control Message Protocol	40
1. Diagnostic du réseau IP avec Internet Control Message Protocol (ICMP)	40
2. Structure du message ICMP	40
3. Echo Request (Type ==8) et Echo Reply (Type ==0)	41
4. ICMP Time Exceeded (Type ==11)	42
5. ICMP Destination Unreachable (Type ==3)	44
6. ICMP Redirect (Type ==5)	45
VII - Chapitre VII Routage et Protocoles de routage	47
1. Routage	47
2. Protocoles de routage	50
VIII - Chapitre IIX Protocoles Applicatif TELNET	54

1. TELNET (TELEtype NETwork)	54
2. Fonctionnement	54
3. NVT (Network Virtual Terminal)	55
4. Négociation d'options	56
IX - Chapitre IX Protocoles Applicatif FTP	58
1. FTP (File Transfer Protocol)	58
2. Fonctionnement FTP	58
3. Liste des commandes FTP et réponses serveurs	61
4. TFTP (Trivial FTP)	61
X - Chapitre X Protocoles Applicatif DNS	63
1. Domain Name System	63
2. Nom de domaines	63
3. Enregistrements DSN (resource records)	64
4. Organisation hiérarchique	65
XI - Chapitre XI Protocoles Applicatif HTTP	66
1. Hypertext Transfer Protocol (HTTP)	66
2. Syntaxe URL HTTP	66
3. Versions HTTP	67
4. Fonctionnement HTTP	67
5. Requêtes et Réponses HTTP	67
6. Méthodes HTTP	69
7. Codes de réponses	70
8. Caching et proxies	70
9. Types de headers (entêtes)	71
10. HTTPS	72
XII - Chapitre XII Protocoles Applicatif d'echange Mail (SMTP, POP et IMAP)	73
1. Protocoles d'échange de mail	73
2. Format de mail électronique	73
3. Multipurpose Internet Mail Extensions (MIME)	74

4. Contenu MIME composé	75
5. SMTP (Simple Mail Transfer Protocol)	77
6. Transmission du mail avec SMTP	78
7. Fonctionnalités de sécurité de SMTP	80
8. Post Office Protocol (POP)	80
9. Commandes et réponses POP	80
9.1. Authentification	80
9.2. Commande d'échange avec le serveur	81
10. Internet Message Access Protocol (IMAP)	82
11. Les commandes IMAP	82
11.1. Commandes d'authentification	84
11.2. Commande après authentification :	85
11.3. Commandes après sélection de la boîte	86
XIII - Chapitre XIII Software Defined Network	88
1. Problématiques dans le réseau TCP/IP	88
2. Principes du SDN	88
3. Table de Flux	88
4. Contrôleur SDN	90
5. Protocole OpenFlow	91
6. Messages OpenFlow	91
XIV - Chapitre XIV QUIC (Quick UDP Internet Connections)	94
1. Effets négatifs du chiffrement et TCP sur la QoS	94
2. Fonctionnalités du protocole Quic	95
2.1. Multiplexage des flux	95
2.2. Always secure	96
2.3. Reduced latency	97
2.4. Reliability	98
3. Avantages de QUIC et utilisation du connection ID	98
Références	99

Objectifs

L'objectif général de ce cours, destiné aux Master 1 SIC et IA, est de familiariser l'étudiant avec le fonctionnement du réseau TCP/IP car c'est l'architecture réseau la plus déployée actuellement.

A l'issue de ce cours l'apprenant doit être capable de:

- Comprendre les fonctionnalités de chaque couche TCP/IP ;
- Comprendre les protocoles implémentés à chaque couche TCP/IP ;
- Appliquer les connaissances réseau pour interagir avec les services applicatifs (HTTP, FTP, IMAP, POP et SMTP) ;
- Apprendre à analyser le fonctionnement des applications réseau ;

Chapitre I Introduction aux réseaux

I

1. Définition d'un réseau

Définition

Un réseau est un ensemble d'entités capables d'échanger des informations. Plusieurs réseaux existent actuellement :

- Les réseaux sociaux ;
- Le réseau de télécommunication ;
- Réseau de neurones ;
- Réseaux informatiques (*notre sujet d'intérêt*) ;

Dans un réseau, le partage de l'information peut se faire :

- Envoie direct (*Unicast*) ;
- Diffusion / sharing (*Broadcast*) ;
- Partage avec un groupe définis (*Multi-cast*) ;

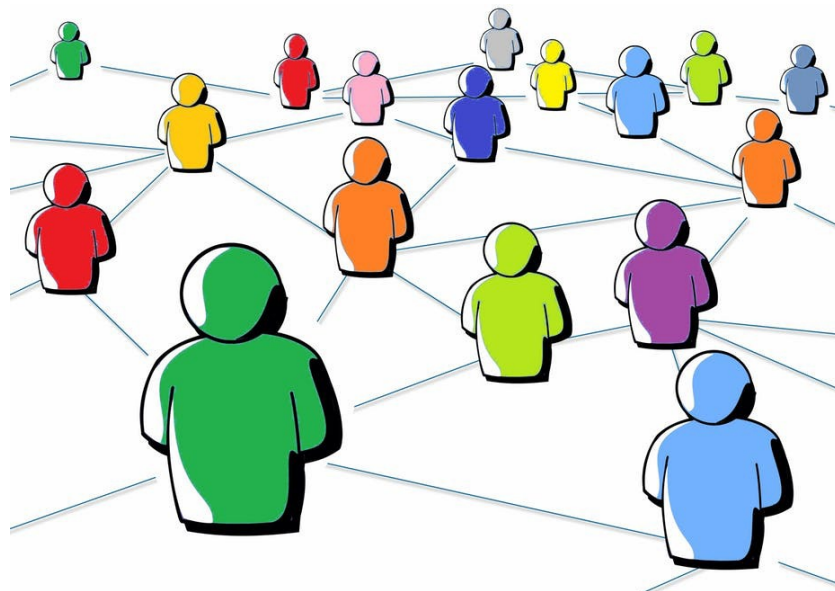


Figure I.1. Exemple de réseau

2. Équipements réseau

Dans un réseau informatique, plusieurs types d'équipement peuvent être présents:

- Des stations utilisateurs (PC, SmartPhone, PC de bureau) ;
- Hub (concentrateur);
- Répéteur ;
- Switch (commutateur) ;
- Routeur ;
- Câbles ;
- Cartes réseaux Wifi ;

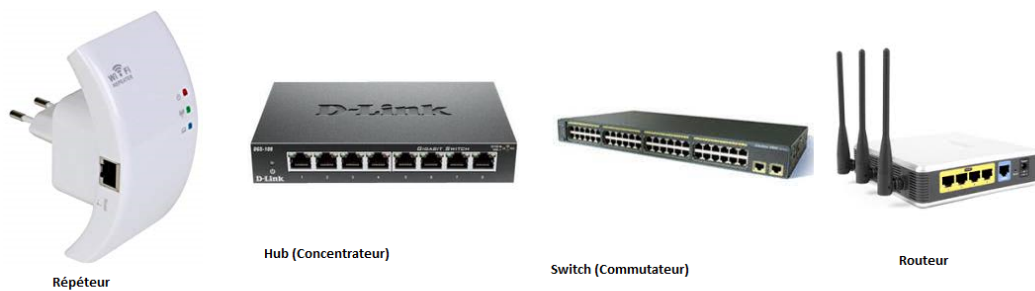


Figure 1.2. Équipements réseau

Exemple

Chaque équipement a un rôle dans le réseau et opère à un niveau défini (*ce qu'on va voir dans le modèle OSI et TCP / IP*).

Par exemple :

- Une application peut se situer que dans des stations utilisateurs (*et non dans un switch ou hub*) ;
- Un hub ne fait que rediffuser le signal sur le reste de ses ports et ne peut pas router les données ;
- Pour relier deux réseaux, un routeur doit être déployé (*et non pas un switch*) ;

3. Topologies réseaux

Définition : Topologie physique et topologie logique

- La topologie *physique* du réseau désigne l'organisation physique des machines (architecture / placement des machines dans le réseau) ;
- La topologie *logique* désigne *comment les machines communiquent* ;

Topologie en bus

- Dans cette topologie, les machines sont connectées par un bus (des câbles liés avec des connecteurs T). *Comme si toutes les machines sont reliées par le même câble* ;
- Toutes les transmissions sont reçues par toutes les machines (*au lieu que seulement le destinataire approprié reçoit ces données*) ;

- Toutes les machines sont dans le *même domaine de collision* car toutes les machines partagent le même câble.

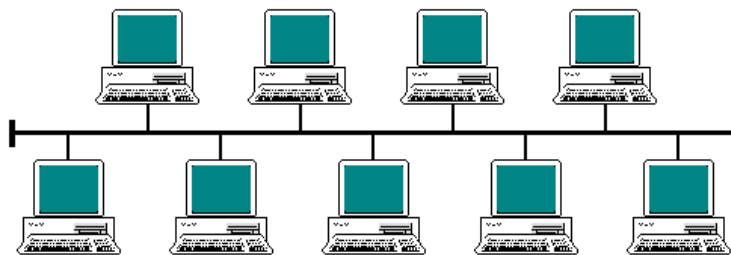


Figure I.3. Topologie en bus

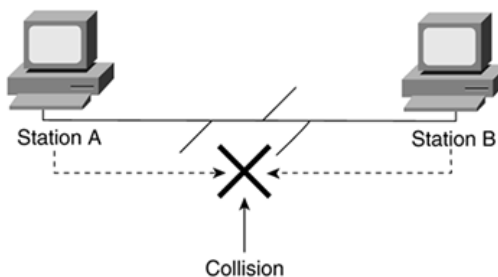


Figure I.4. Collision

Une collision se produit lorsque deux transmissions se chevauchent. Ainsi, les données transmises ne seront pas livrées correctement.

Comment éviter les collisions :

- CSMA / CA et exponential random back-off (*protocole de la couche liaison, Stay tuned !*)

👉 Exemple : Exemple de collision

Dans une discussion (*civilisée !*) entre amis, à un moment donné, deux personnes peuvent tenter de *parler au même moment* (en réseau ça correspond à deux machines qui transmettent sur le même médium / câble *simultanément*). Pour que la discussion soit correctement effectuée, une seule personne doit parler jusqu'à la fin de ces propos avant que le deuxième commence de s'exprimer.

Si deux personnes parlent en même temps, il y aura des *collisions de paroles* et les deux personnes n'arriveront pas à s'entendre (comme dans les discussions télévisées arabes...).

Topologies en étoile

- Dans une topologie en étoile, les machines sont reliées par un élément central (*un switch typiquement*).
- Si au lieu d'un switch, l'élément central est un hub, la topologie physique reste la même (en étoile). Mais, chaque transmission est entendue par toutes les machines dans le réseau (*la topologie logique devient en bus !*).

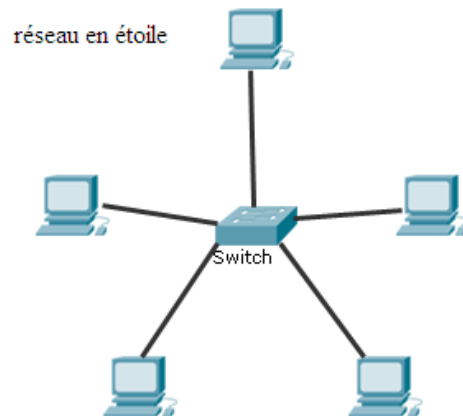


Figure I.5. Topologie en étoile

Topologie tokenring

- Les machines sont déployées en anneau.
- Cette topologie est similaire à la topologie en bus mais, la communication se fait tour par tour afin d'éviter les collisions. Un token est échangé d'une manière circulaire et la machine qui reçoit le token a le droit de transmettre (si elle possède des données à transmettre). En suite, le token est transmis à la machine suivante.

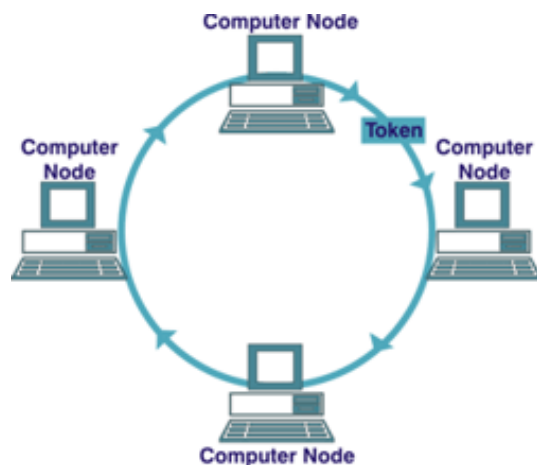


Figure I.6. Topologie tokenring

D'autres topologies

- Dans une topologie maillée, chaque machine est connectée à toutes les autres machines par un lien direct. Ainsi, si le réseau comporte N machines, chaque machine aura $N-1$ câbles. Donc il y aura un total de $(N-1)*N/2$ câbles pour lier les machines ($N/2$ car un câble est bidirectionnel).
- Le coût de mise en place de cette topologie est élevé à cause du câblage (et la nécessité d'acquérir des cartes réseaux supplémentaires) ;

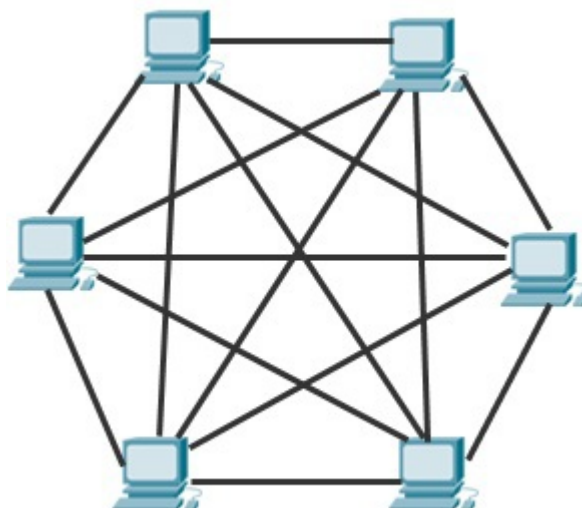


Figure 1.7. Topologie maillée

En cloud computing, le réseau interne (Data Center) est généralement organisé dans une *topologie trois tiers* :

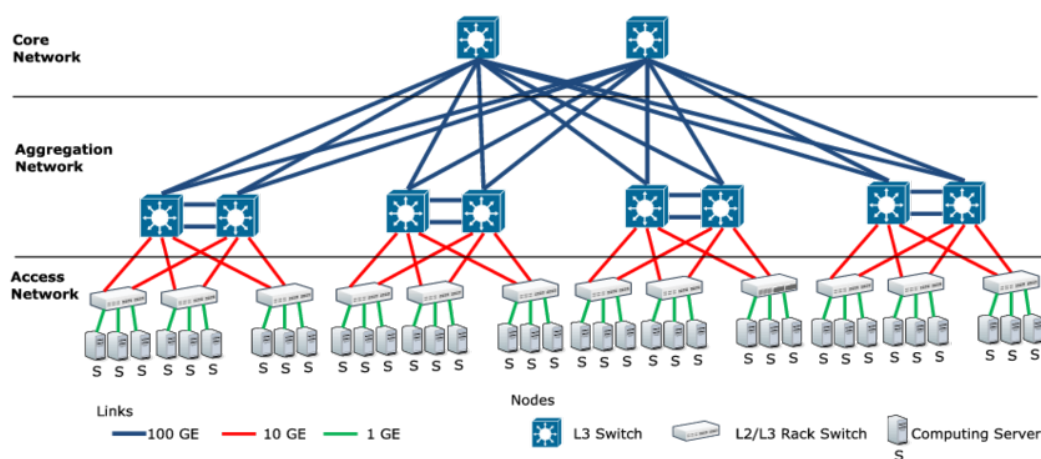


Figure 1.8. Topologie trois tiers

En outre, le réseau peut être déployé avec une topologie hybride (mélange de plusieurs topologies) :

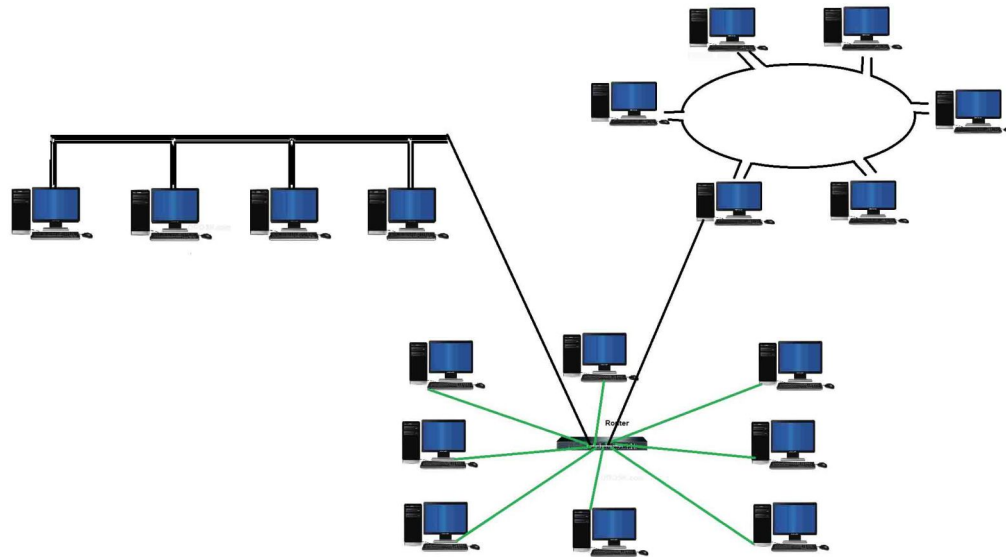


Figure I.9. Topologie hybride

Chapitre II Modèle OSI et Modèle TCP / IP

II

1. Modèle OSI (Open Systems Interconnection)

🔑 Définition

Le Modèle OSI (Open Systems Interconnection) été développé par l'ISO (International Organization for Standardization), et a vu le jour en 1978. Ce modèle est devenu un standard en 1984 (ISO 7498:1984)

- Le modèle OSI est un modèle *conceptuel* (abstrait) des réseaux informatiques.
- Les *fonctionnalités réseau sont découplés* pour assurer un fonctionnement flexible.
- Dans ce modèle, les fonctionnalités / protocoles qui sont similaires sont regroupés dans la même couche.
- Certaines fonctionnalités sont redondantes (par exemple contrôle d'erreurs, cryptographie).
- Chaque couche utilise les couches inférieurs pour fonctionner.
- *L'objectif du modèle été de créer un modèle qui assure l'interopérabilité entre les technologies des différents fabricants matériel.*
- Même ci, le modèle OSI ne spécifie pas les protocoles de chaque couche, celui -ci offre une vision claire sur le fonctionnement du réseau ;
- Ce-ci rend ce modèle très utile pour l'étude du fonctionnement du réseau ainsi que pour le développement et comparaison de technologies réseaux ;



Figure II.1. Le modèle en couche de OSI^{1*}

- Lors de la transmission, le paquet de la couche supérieure est encapsulé avec un Entête (Header) (Figure 2 - à gauche);

- Lors de la réception, l'entête de la couche inférieure est supprimé avant que le paquet est envoyé à la couche supérieure (Figure 2 - à droite);

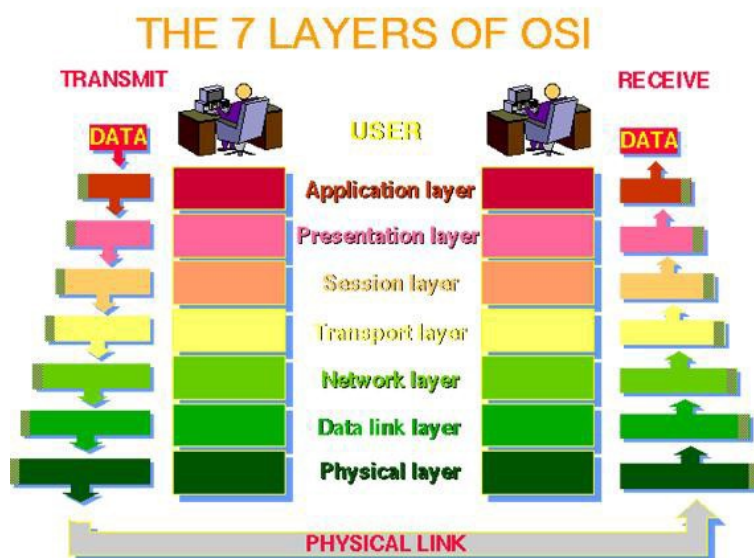


Figure II.2. Modèle OSI^{2*}

Exemple : Exemple de découplage

- Si vous mettez à jour votre pilote de carte réseau (*couche liaison et physique*), les applications réseau (*couche application*) qui fonctionnent sur votre machine ne seront pas affectées car la *couche application* et les *fonctionnalités de votre carte réseau* sont *découplées* ;
- Votre navigateur fonctionnera indifféremment dans un PC relié au réseau par un câble comme dans un Laptop connecté au réseau par une interface Wifi. Parce que les fonctionnalités de la couche application sont découplées des couches Liaison / Physique ;
- Le développeur d'une application réseau ne prend pas en considération comment les données sont écrites sur le medium (signalisation de la suite des bites) ;
- Si les fonctionnalités n'ont pas été pas découplées, le modèle sera inflexible (le changement dans une fonctionnalité entraînera des changements dans plusieurs d'autres) ;

2. Couche application

- C'est la couche avec laquelle l'utilisateur interagit (*navigateur web par exemple*) ;
- Offre une interface pour accéder au réseau et utiliser des protocoles prédéfinis ;
- Fournis des services de sécurité / authentification ;

3. Couche présentation

- Généralement, les fonctionnalités de cette couche sont assurées par le système d'exploitation (Windows, Linux).
- Assure la mise en forme de données transmises (ou reçues) par l'application.

Exemple : Fonctionnement couche présentation

- La couche application d'un client génère des données (couche 7) ;
- La couche présentation compresse / crypte les données générées par la couche application (couche 6);
- Les données sont transférées sur le réseau ;
- Les données sont reçues par le destinataire ;
- La couche présentation de la machine destination décompresse / décrypte les données reçues (couche 6) et transmet les données à la couche application.
- Les données sont affichées par la couche application du destinataire (couche 7);

4. Couche session

- Se charge de la création, maintient et clôture d'une session entre deux machines impliquées dans la communication;
- Spécifie la manière de communication (*Half duplex* : communication bidirectionnel mais dans un seul sens à la fois, *Full duplex* : communication bidirectionnel simultanée, *Simplex* : communication dans un seul sens) ;

Exemple : Initiation de session

Exemple 1 : Une conversation simple

- Pour qu'une conversation commence entre deux personnes, la personne initiant la conversation doit commencer par dire "Bonjour".
- La personne destinataire répondra, par un "Bonjour".
- Si la personne s'arrête de parler pour une durée, la session de discussion est terminée.
- Si, la source de la discussion dit "Au revoir", la session est terminée.

Exemple 2 : Transmission Fax

- Pour transmettre un Fax, le transmetteur demande un signal Fax au destinataire ;
- Le destinataire envoie le signal Fax pour que la communication commence ;

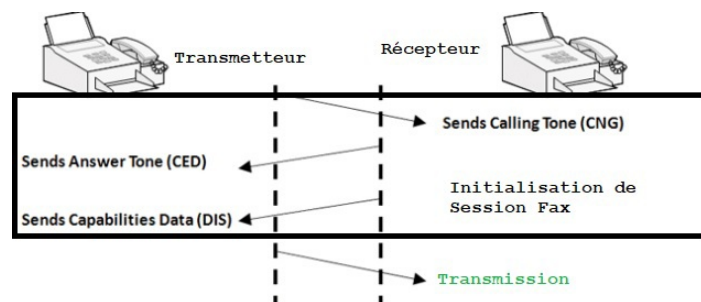


Figure II.3. Session Fax^{3*}

Exemple : Modes de communication

- Communication sur Walkietalkie est dite Half duplex : parce que une seule personne peut communiquer à la fois ;
- La communication sur téléphone est Full duplex ;
- Le satellite communique avec nos démodulateurs en mode Simplex;

Simplex Transmission

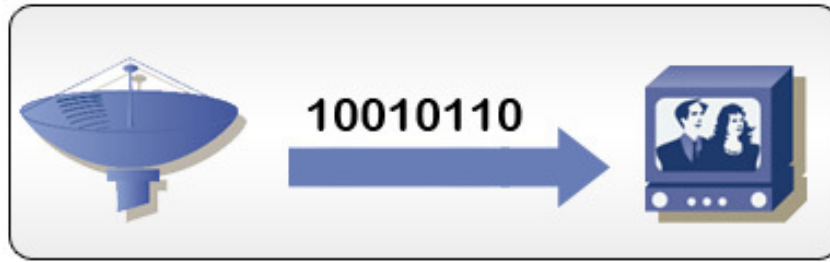


Figure II.4. Mode de communication Simplex^{3*}

5. Couche transport

- Offre des modes de transports des données générées par l'application de la machine source ;
- Les données transmises sont découpées en segments ;
- Cette couche se charge du mode de communication :
 - Devons-nous nous assurer que les données ont bien été reçues par le destinataire ? (*connection-oriented mode en anglais, mode connecté en français*) ;
 - Les données doivent-elles être transmises sans garantir leur réception ? (*connectionless-mode, mode non connecté en français*) ;
- Cette couche ajoute l'identifiant de l'application source (*concrètement dans le modèle TCP numéro de port de la source*) ainsi que celui du destinataire ;
- Contrôle de séquences (*pour réordonner et reconstruire le message originale chez le destinataire*) ;
- Contrôle d'intégrité (*checksum*) ;

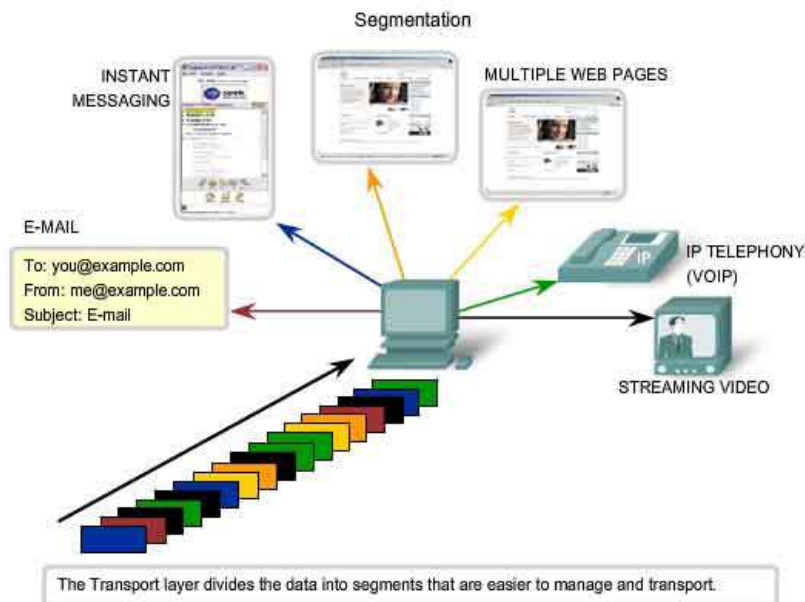


Figure II.5. Couche transport^{3*}

6. Couche réseau

- Se charge du routage des données ;
- Ne fournit pas de garantie que le message sera livré à la destination ;

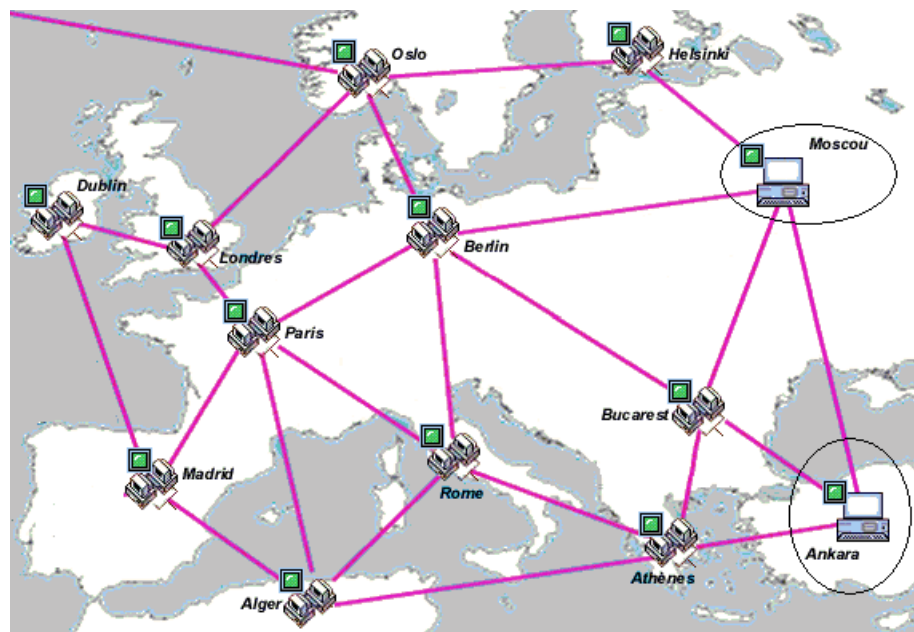
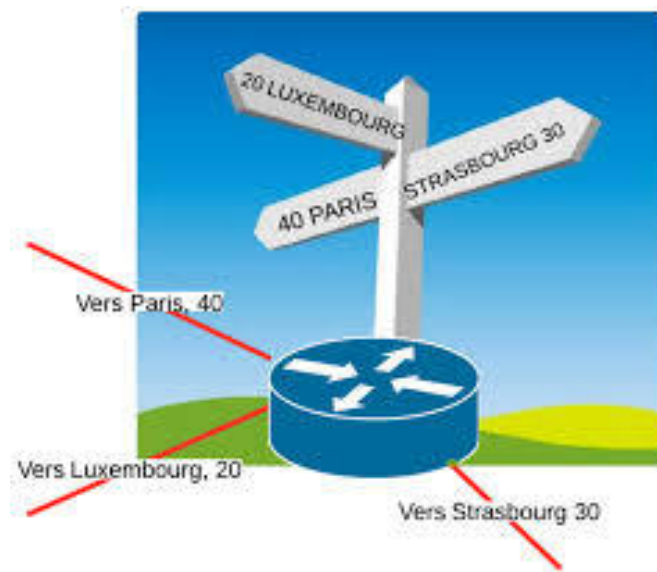


Figure II.6. Routage^{4*}

7. Couche liaison de données

- Établissement de liaison physique entre machines ;
- Découper les données (segments à ce stade) en trames ;
- Une trame ne doit pas être transmise avant la fin de la transmission de la trame déjà émise ne soit reçue ;
- Peut assurer la correction d'erreur ;

8. Couche physique

- Mettre les données sur le médium sous forme de signaux (code de Manchester, NRZI, etc) ;
- A ce niveau un paquet transmis est appelé une *trame* (*frame* en anglais) ;
- Recevoir les données du médium ;
- Se charge du multiplexage et détection de collisions ;

Encodage de Manchester :

- S'il y a transition de voltage d'une valeur supérieur à une valeur inférieur ceci *signifie la valeur binaire encodée est 0* ;
- S'il y a transition de voltage d'une valeur inférieur à une valeur supérieur ceci *signifie la valeur binaire encodée est 1* ;

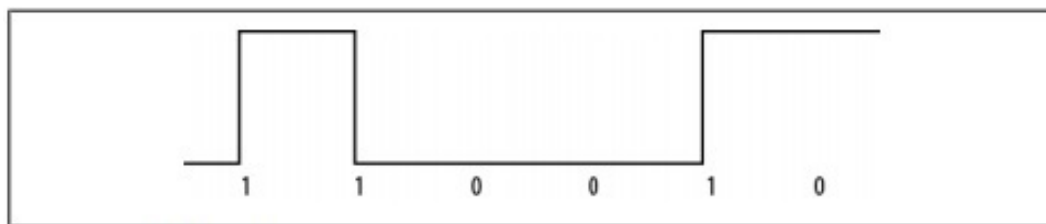


Manchester encoding (source: IEEE 802.3-2002)

Figure II.7. Encodage Manchester

Encodage de NAZI :

- S'il y a transition dans la valeur de voltage ceci *signifie la valeur binaire encodée est 1* ;
- S'il n'y a pas de transition dans la valeur de voltage ceci *signifie la valeur binaire encodée est 0* ;



NRZI encoding

Figure II.8. Encodage NAZI

9. Model TCP / IP (Transmission Control Protocol / Internet Protocol)

- Contrairement au modèle OSI, le modèle TCP / IP spécifie les protocoles utilisés ;
- Le modèle TCP / IP a émergé avant la finalisation du modèle OSI en proposant une solution plus simple (la version initiale de ce modèle est représentée par 4 couches au lieu de 7 du modèle OSI, la version 2 de TCP/IP est représentée par 5 couches) ;
- Ainsi, ce modèle est devenu le standard pour la communication sur Internet ;
- Le modèle TCP / IP est très similaire au modèle OSI (la couche application du modèle TCP / IP = la couche application OSI + couche présentation + couche session) ;
- Dans le modèle TCP/IP, la Couche Internet est parfois retrouvée sous le nom *Couche Réseau* ;
- La *Couche Liaison* est parfois retrouvée sous le nom *Couche Accès réseau*;

Application	7	Application
	6	Presentation
	5	Session
Transport	4	Transport
Internetwork	3	Network
Link/Network	2	Data Link
Physical	1	Physical

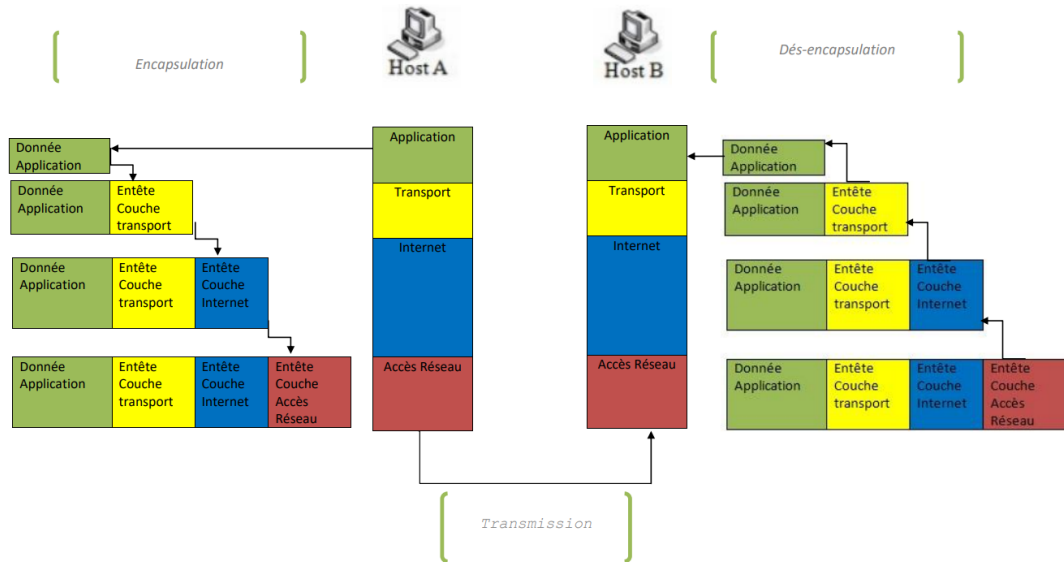
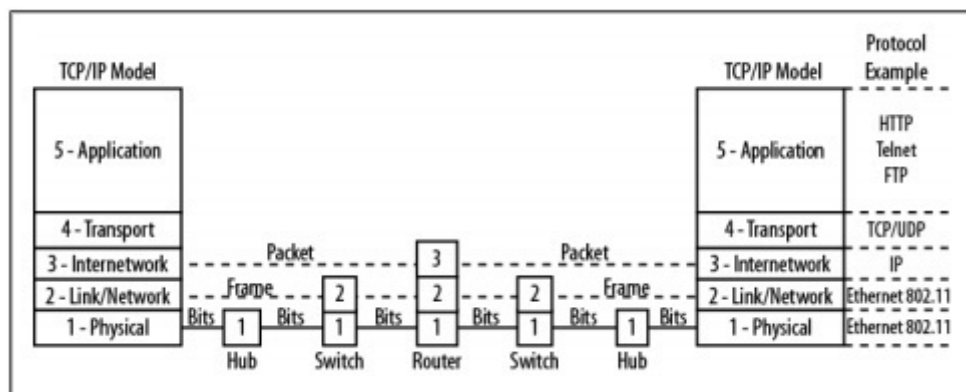
Figure II.9. Modèle TCP /IP (a) - Translation TCP/IP OSI^{5*}

Figure II.9. Modèle TCP /IP (b) - Encapsulation

10. Niveaux de fonctionnement des équipements dans le modèle TCP / IP

Rappel

Chaque équipement réseau opère à un niveau donné du modèle TCP / IP.

Figure II.10. Niveau d'opération des équipements réseau^{5*}

Chapitre III Applications et protocoles de la couche transport

III

1. Protocoles de la couche application

- Les applications interfacent avec la couche transport à travers les sockets ;
- Deux applications ne peuvent pas utiliser le même numéro de port dans la même machine ;
- Il existe des ports réservés pour des protocoles applicatifs prédéfinis [de 0 à 1023] (*FTP=port :20, HTTP=port :80, HTTPS=port :443, DNS=port 53, ...etc*) ;
- Une plage de numéros de ports est disponible pour être allouée dynamiquement aux applications [de 49152 à 65535];
- Si un port est ouvert dans la machine locale par une *application A*, d'autres applications dans la même machine peuvent utiliser un socket pour communiquer avec cette application ;
- Une application serveur est généralement multi-processus (pour chaque connexion un processus est créé afin de traiter la connexion d'un client) ;

2. Exercice : Ports et applications réseaux

Exécutez la commande "*netstat -n*" sur votre CMD (Invité de commande) pour voir les connexions / ports ouverts dans votre système d'exploitation. Si vous avez une adresse d'une machine distante, vous pouvez voir les ports ouverts de celles-ci par la commande *nmap*.

3. Protocoles couche transport

◆ Rappel : Rôle couche transport

- Assure la livraison des données à l'application cible;
- Supposant que *l'application A*, hébergée dans la machine 192.168.1.2, transmet des données vers *l'application B* situées à la machine 192.168.1.3;
- Pour effectuer cette transaction, les données générées par A doivent être enveloppées avec un entête de la couche transport (cet entête spécifie le numéro de port de l'application source et le numéro de port de l'application destination) ;
- Pour transmettre / recevoir des données, les protocoles applicatifs (comme FTP ,HTTP) doivent utiliser un protocole de transport (TCP et/ou UDP) ;
- Ce ci est appelé *sous-couchage de protocoles* ;

☞ Exemple : Exemple sou-couchage de protocoles

- Le protocole FTP (File Transfer Protocol) utilise TCP ;
- Le protocole DNS (Domain Name Server) utilise UDP et TCP ;

4. UDP (User Datagram Protocol)

- UDP est un protocole très simple. Ce protocole n'assure que de faire passer un message à l'application destination (*d'où il est dit non fiable*).
- *Ce protocole n'assure ni la segmentation, ni la réorganisation des messages au niveau du destinataire* (si l'application souhaite assurer la segmentation / classement des segments de l'autre bout, elle doit l'implémenter elle même) ;
- Le seul contrôle fourni par ce protocole est le checksum. Celui-ci permet de vérifier s'il y eu une erreur de transmission. En cas d'erreur, le message envelopper par UDP sera simplement ignoré (la couche transport du destinataire ne livrera pas le contenu à l'application). De même, la source d'un message erroné ne sera pas notifiée de cette échec ;
- *Ce protocole est généralement utilisé dans les applications temps réel* (jeux vidéo online, application de communication audio visuel comme Skype ou TeamSpeak) ;
- En utilisant ce protocole, une application ne peut qu'espérer que le destinataire recevra les données transmises;

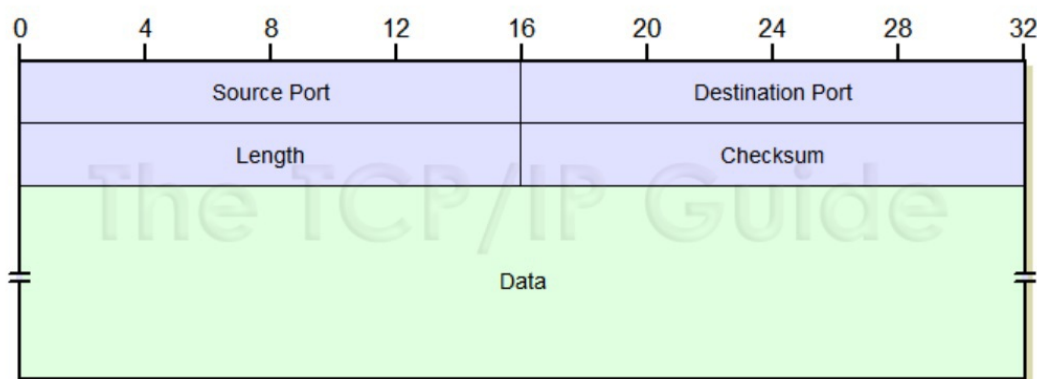


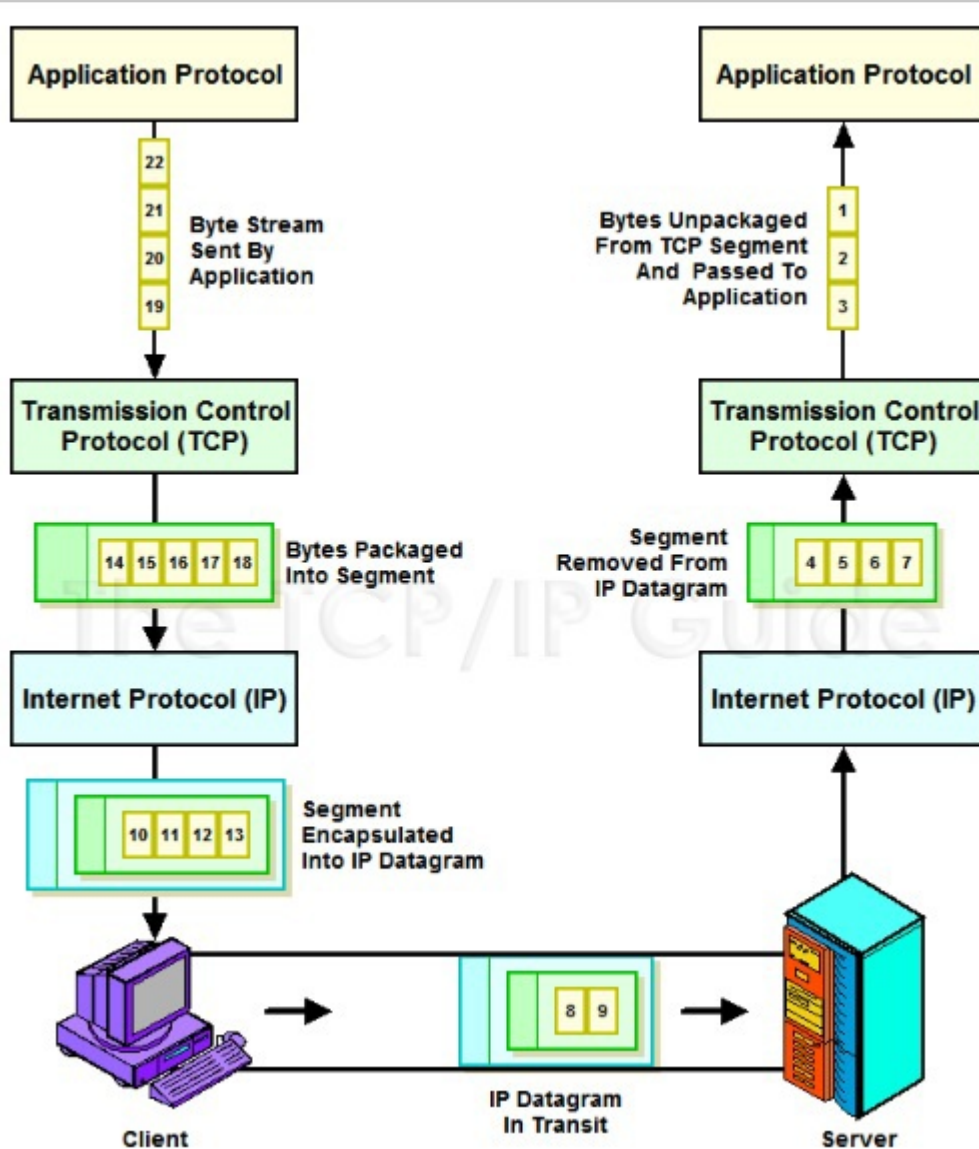
Figure III.1. Entête UDP^{3*}

5. TCP (Transmission Control Protocol)

- TCP est le protocole de transport le plus utilisé par les applications réseau (d'où l'inclusion de son nom dans le modèle TCP/IP) ;
- Ce protocole est *fiable* (dit orienté connexion) car il assure la livraison des données;
- Comme protocole de la couche Transport, le rôle de TCP est d'assurer que la destination appropriée reçoit les données qui lui sont transmises. Pour ce-ci, un segment TCP contient un numéro de port de l'application source (celle qui génère le segment) et le numéro de port du destinataire (celle qui est sensée recevoir le segment) ;
- TCP est un protocole Full-Duplex (c.à.d les deux applications qui établissent une connexion s'échangent des messages simultanément et dans les deux sens avec la même connexion) ;
- Les données sont transmis au protocole TCP sous forme de flux. *Ce protocole ensuite découpe le flux en segments de données avant de les encapsuler et les transmettre à la couche inférieure (Couche Réseau/IP) ;*

- TCP assure aussi le contrôle du flux de transmission (voir la section sur le *Sliding Window*);

Chaque fois qu'un segment est transmis, un chronomètre (compteur de temps) est lancé. Si le chronomètre expire sans que l'acquittement ne soit reçu, le segment doit être retransmis. Si la retransmission d'un segment se répète plusieurs fois sans succès (pas d'acquittement), le protocole TCP arrête les tentatives ;



"Figure III.2. Segmentation TCP^{3*}

6. Entête TCP

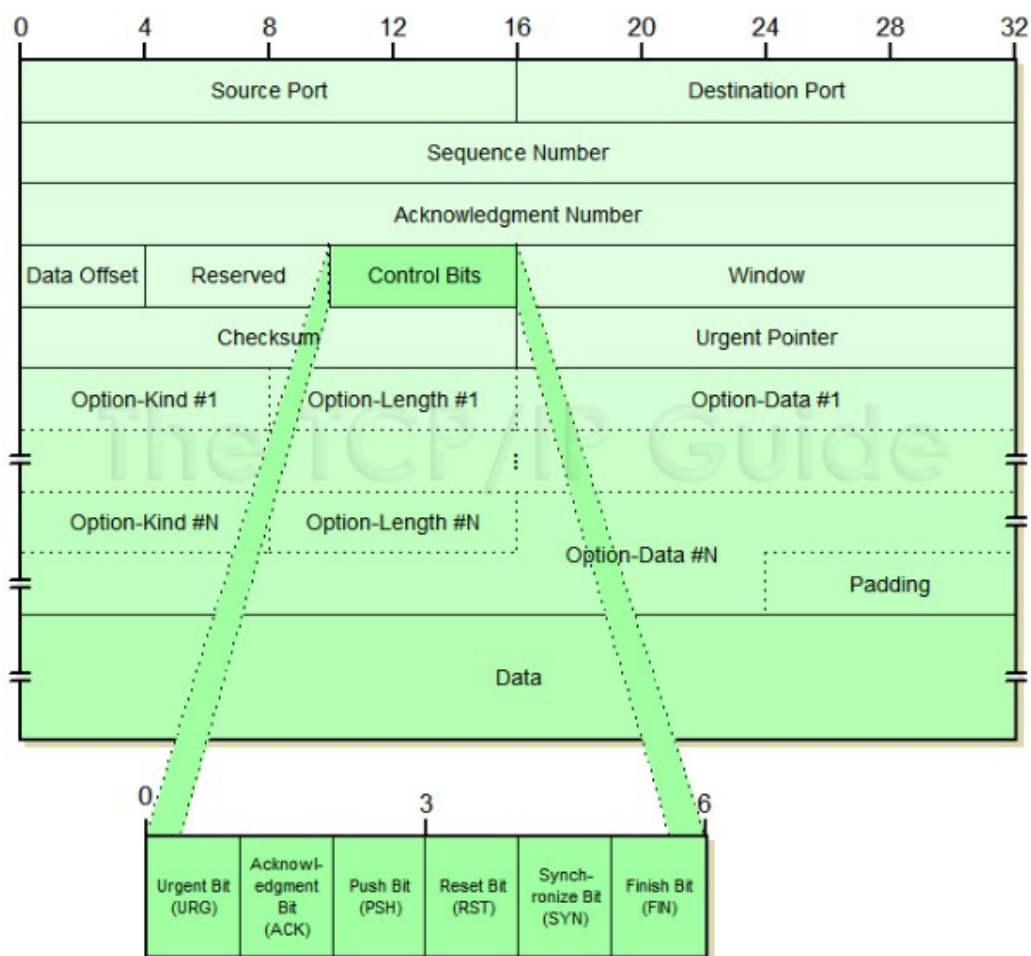


Figure III.3. Entête TCP^{3*}

1. Un segment TCP commence par le numéro de port de l'application source (2 octets)
2. Ensuite, le numéro de port de l'application destination (2 octets) ;
3. Numéro de séquence du segment (4 octets) : numéro de séquence du premier octet dans ce segment. Cette valeur est utilisée par le récepteur pour réordonner les segments et détecter les segments non reçus ;
4. Numéro d'acquiescement (Acknowledgment) (4 octets) : Si le bit du flag ACK (voir champ 7: Flags de contrôle) est allumé, la valeur de ce champ indique l'intervalle des numéros de séquences acquittés (tous les numéros de séquences [champ 3] inférieures à ce numéro ACK sont acquittés car ils ont été correctement reçus par la destination). De plus, cette valeur indique au transmetteur le numéro de séquence du prochain segment à transmettre ;
5. Offset (4 bit) : La taille total de l'entête TCP (généralement 20 octets). Ainsi, ce champ indique où l'entête TCP s'arrête et les données de l'application commencent. S'il y a des options, l'offset peut prendre jusqu'à 60 octets. Cette valeur est exprimée en word (4octets == 1 word).
6. Champs réservé (3 bit) : Ce champs est réservé (non utilisé par le protocole) et prend toujours la valeur 0 ;
7. Flags de contrôle (6 bit) : Chaque bit dans ce champs possède une signification (voir la section suivante titrée "Signification des Bits flag") ;

8. *Window (2 octets)* : Le récepteur de données doit indiquer la taille du buffer (mémoire tampon) dédiée à cette connexion. La valeur de ce champ est ajustée au cours de la transmission pour contrôler le flux de transmission ;
9. *Cheksum (2 octets)* : Valeur de contrôle calculée pour assurer l'intégrité du segment ;
10. *Pointeur donnée urgente (2 octets)* : Généralement est égale à la valeur 0. En cas où le flag urgent dans le champ Flags de contrôle (*voir champ 7*) est allumé, la valeur dans ce champ indique où les données qui doivent être traitées en priorité se terminent;
11. *Option* (de 0 à 40 bits) : Généralement inexistant sauf au début de l'établissement de connexion TCP où des paramètres sont échangés. Par exemple, le Maximum Segment Size (MSS) est communiqué dans le champ option au début de la communication par le champ Option;

7. Signification des Bits flag

URG : Le segment contient des données qui doivent être traitées d'urgence ;

ACK : Segment d'acquiescement ;

PSH : Du côté transmetteur, le flag PSH allumé indique à la couche transport du transmetteur que le segment doit être transmis immédiatement. Du côté récepteur, le flag PSH allumé indique à la couche transport du récepteur que le segment doit faire passer le paquet à la couche application immédiatement ;

RST : Le transmetteur d'un segment avec un flag RST allumé indique que cette connexion a été inattendue. Par exemple l'application A essaye de se connecter au port 5000 d'une machine distante alors que ce port n'est utilisé par aucune application. Ainsi, la machine distante répond avec un segment TCP avec le flag RST allumé;

SYN : Indicateur de synchronisation ;

FIN : Terminaison de connexion ;

8. Établissement de connexion TCP par "Three way handshake"

- Soit deux applications A et B ;
- Pour commencer, B doit créer un socket et écouter les connexions entrantes sur le numéro de port *NUMPORT* (en pratique ce-ci est fait par l'instanciation d'un socket avec *NUMPORT* comme argument, l'attente de connexion se fait par la méthode *accept()*) ;
- *Etape 1* : La couche transport de l'application A qui souhaite connecter à B, commence par transmettre un segment avec le numéro de port du destinataire == *NUMPORT* et un *flag avec le bit SYN allumé*. Ce paquet *ne contient aucune donnée* et signifie que A souhaite *SYNCHRONISER (SYN)* avec B (ce-ci est fait par la méthode *connect()* du client);
- *Etape 2* : Après la réception de cette demande de synchronisation, la couche transport de B transmet un segment vide (*pas de donnée dedans*) contenant un *flag avec le bit SYN allumé et ACK allumé*. Ce-ci signifie que la tentative de connexion a été reçue par B ;
- *Etape 3* : Ensuite, la couche transport de A transmet un segment contenant un *flag avec le bit ACK allumé* (pour signifier que la connexion a été établie) ;

Trois étapes d'établissement de connexion = Three way Handshake

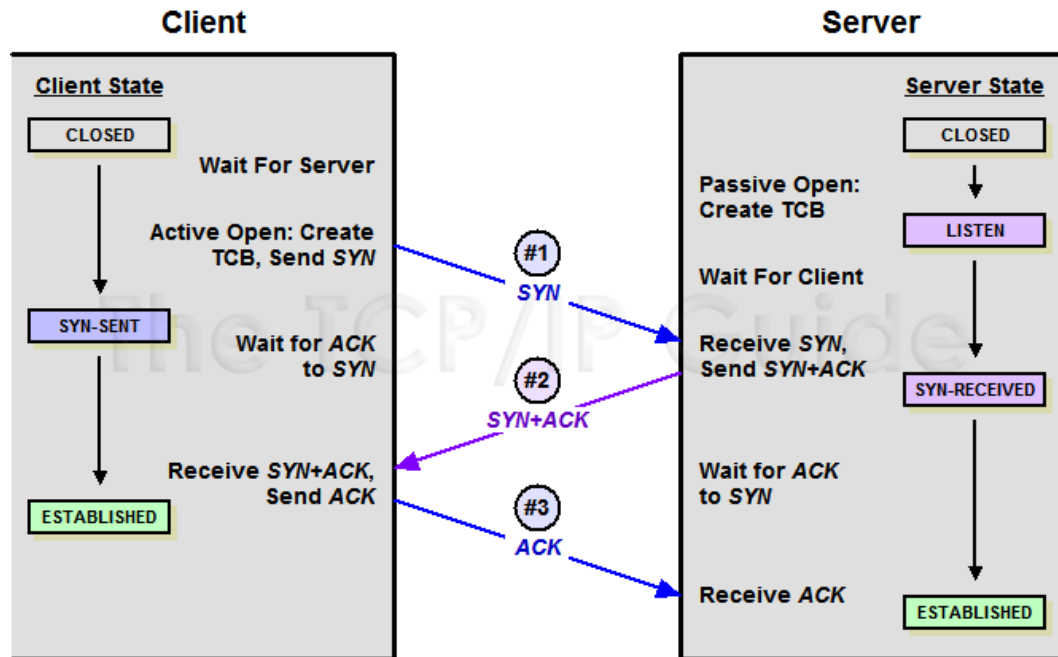


Figure III.4. Établissement de connexion avec "Three way Handshake"^{3*}

9. Clôture de connexion

- Similairement, lors de la clôture de transmission, les couches transports des deux applications s'échangent des segments contenant un flag avec le bit *FIN* (*FINISHED*) et *ACK* allumés ;
- La couche transport de l'application A transmet un segment avec le bit *FIN* allumé pour dire qu'il n'y a pas d'autres données à transmettre et la connexion entre les deux applications peut être clôturée ; Pour clôturer la connexion dans les deux directions, la couche transport de B effectue la même procédure ;

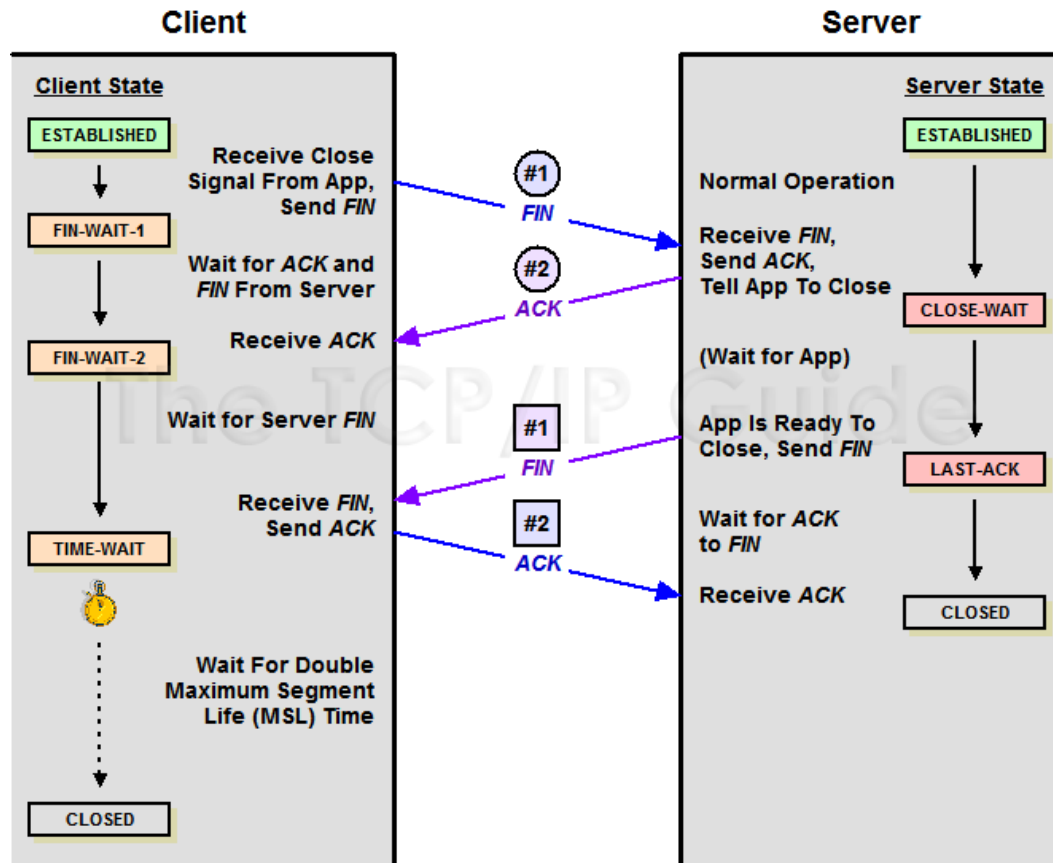
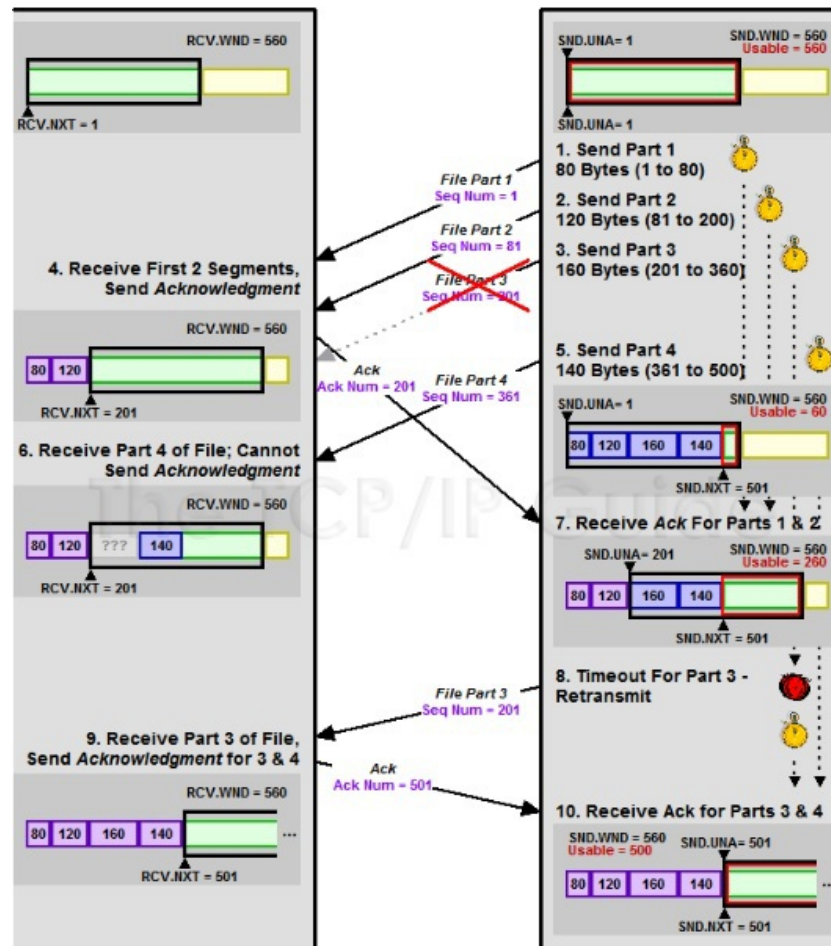


Figure III.5. Clôture de connexion avec flag *FIN*^{3*}

10. Acquittement des segments

- Dans TCP, l'acquittement des segments est cumulative. Cela veut dire que lorsque le récepteur envoie un acquittement avec une valeur (*champ 4*) == 201 (voir la figure en bas), *ce-ci veut dire que tout les BYTES avec un numéro de séquence inférieur à 201 ont été acquittés*. Si des segments avec des numéros de séquences (*champ 3*) supérieurs ou égales à 201 ont été transmis auparavant, ceux ne sont pas encore acquittés (*même s'ils ont été reçus*) ;
- Ainsi, le récepteur acquitte la réception d'une intervalle d'octets cumulativement ;

Figure III.6. Retransmission et acquittement^{3*}

Remarque : Numéro de séquence initial

- Le numéro de séquence initial du premier segment des données n'est pas forcément 0 ;
- Le transmetteur et le récepteur doivent s'entendre sur le numéro de séquence du premier segment lors de l'établissement de connexion (*three way handshake*) ;

11. Mécanisme Sliding Window

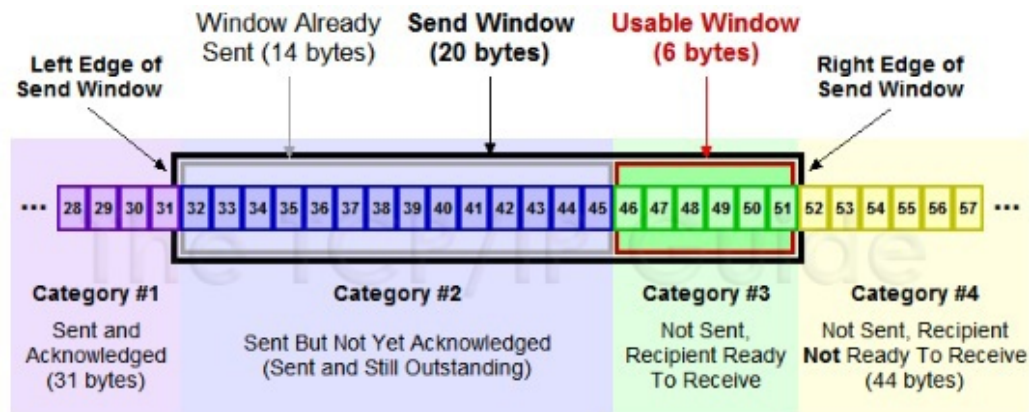
- Chaque segment ACK transmis par le récepteur contient une valeur *Window size* (*champ 8*) pour indiquer au transmetteur la quantité de données qui peut être temporisées (placée dans le buffer) par le récepteur. Ainsi, la somme des tailles des segments envoyés par le transmetteur sans être acquittés ne peut pas dépasser la taille du buffer du récepteur sans que le récepteur acquitte des messages reçus ;

Le transmetteur et le récepteur maintiennent des variables indiquant :

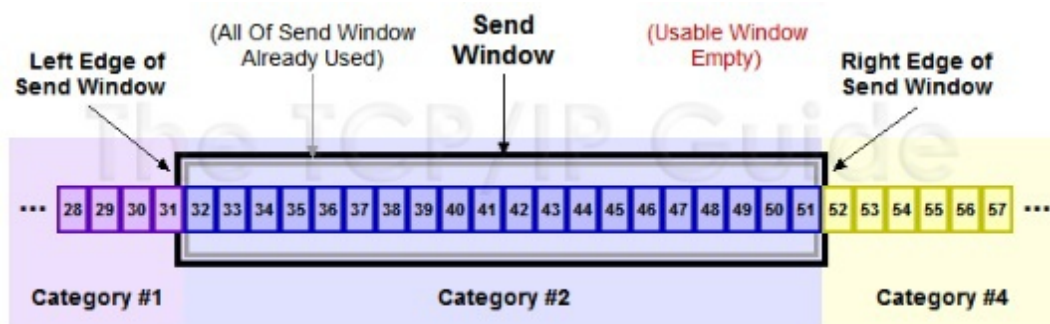
1. L'intervalle de bytes qui ont été transmis et acquittés (*Left Edge* dans la figure en bas, cette valeur= 31). Les bytes inférieurs à cette valeur ont été correctement reçus ;
2. L'intervalle de bytes qui ont été transmis *sans être acquittés* (*Window Already Sent* [encadrée en gris] dans la figure en bas, cette valeur= 32 jusqu'à 45 dans cet exemple). Les bytes dans cette intervalle ont été transmis mais pas encore acquittés ;

- Intervalle utilisable (*Usable Window* dans la figure en bas, cette valeur= 46 jusqu'à 51). Cette valeur définit combien de données peuvent être transmises sans déborder de la taille du buffer définie par le transmetteur ;

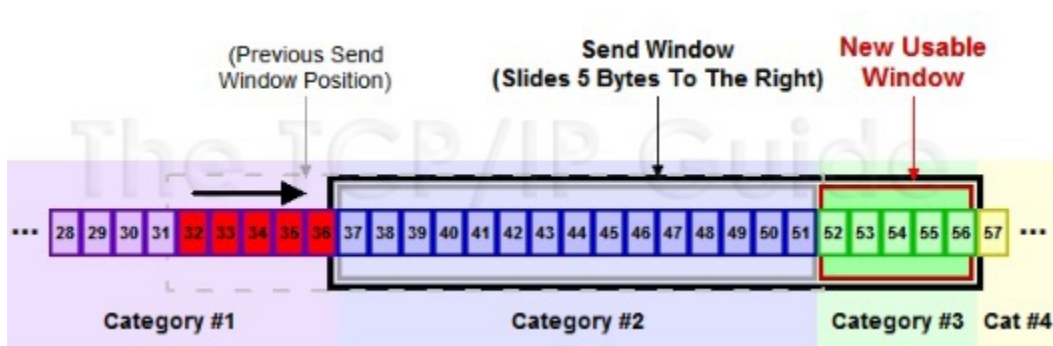
La somme des tailles des trois intervalles == taille du buffer du récepteur ;



Si le transmetteur envoie tous les segments dans l'intervalle utilisable, la transmission s'arrêtera tant que aucun ACK des bytes transmis mais non acquittés ne soient reçus (voir la figure suivante) ;



- Après, si le chrono pour un segment transmis mais non acquitté *expire sans qu'un ACK qui l'inclus ne soit reçu*, ce segment est retransmis ;
- Si un ACK est reçu, l'intervalle de transmission est décalée selon le numéro ACK reçu. Par exemple, dans la figure précédente, lorsque un ACK avec un numéro 37 est reçu, ceci signifie que les bytes de 32 - 36 ont été acquittés. Par la suite, l'intervalle de transmission glisse vers l'avant ;



- Au cours de la transmission, une option qui permet d'augmenter la taille du buffer est l'option *Window Scale*. La valeur de ce champ peut être ajoutée et incluse dans le champ Options. Celui-ci peut prendre une valeur i entre 0 et 14 qui indique que la valeur du champ *Window Size* doit être multipliée par 2^i .
- Avec le champ *Window Size* et l'option *Window Scale*, le récepteur peut ajuster la vitesse de transmission des données. Ce processus est appelé *Contrôle de Congestion*.
- Le contrôle de la congestion est un mécanisme qui limite le flux de paquets dans le réseau. Ainsi, cela empêche de surcharger le récepteur avec des paquets excessifs. La congestion se produit lorsque le débit de paquets vers un nœud est supérieur à sa capacité de traitement.
Lorsqu'une congestion se produit, elle ralentit le temps de réponse du réseau. En conséquence, les performances du réseau diminuent. La congestion se produit parce que les routeurs et les commutateurs disposent d'un tampon de file d'attente, contenant les paquets à traiter. Une fois le traitement du paquet est terminé, celui-ci est transmis au nœud suivant. En cas où un routeur/commutateurs reçoit un paquet mais son buffer est plein, le paquet reçu est supprimé.
- Pour que le transmetteur et le récepteur décide d'une vitesse de transmission que le réseau peut supporté, le transmetteur commence à augmenter le nombre de segments à transmettre graduellement avec l'augmentation du *Window Size* par le récepteur. Par la suite, la quantité de données transmises dans le réseau est revue à la baisse lorsque le récepteur reçoit plusieurs paquets avec le même numéro de séquence (ce qui indique les paquets traversent un chemin congestionné) ou bien une large quantité de segments est perdue en transition (les paquets ont été supprimés à cause d'un buffer overflow).

12. Problème de l'acquittement cumulative

- Supposant que de 100 segments transmis, seulement le troisième n'a pas été correctement reçu. Ainsi, même si la destination a reçu 99 segments, elle ne peut acquitter que les deux premiers segments.

Le transmetteur peut implémenter l'une des deux stratégies :

- Retransmettre tous les segments non acquittés ;
- Attendre l'expiration du chrono de chaque segment individuellement avant de le retransmettre ;

Une nouvelle stratégie est apparue appelée acquittement sélective où le récepteur ajoute des numéros d'acquittements individuels (au lieu d'intervalle) dans le champs options (*champs 11*) d'un segment avec un flag ACK allumé. *Cependant, ce-ci nécessite une négociation préalable lors de la synchronisation.*

Chapitre IV Adressage dans la Couche Internet

IV

1. Couche Internet

- Pour communiquer avec *une machine placée hors du réseau local*, l'adresse IP du destinataire est nécessaire (Protocole IP de la Couche Internet);
- Sur un réseau local, l'adresse physique (MAC) est utilisée pour s'adresser à une machine (Protocole ARP de la Couche Liaison);
- Le protocole IP (Internet Protocol) est le langage d' Internet ;
- Les deux versions de base de ce protocole sont IPv4 et IPv6 ;
- Le protocole IP ne garantit pas la livraison de paquets au destinataire. Ainsi, ce protocole est dit "*best effort*";

2. Entête IPv4

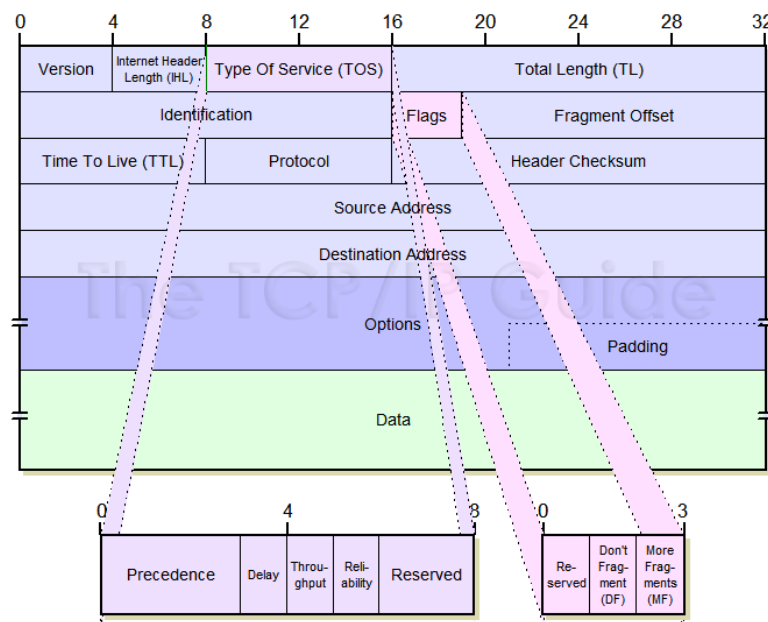


Figure IV.1. Entête IP^{3*}

- *Version (4bits)* : indique la version du protocole (IPv4 ou IPv6);
- *Taille de l'entête (IHL == Internet Header Leangth) (4bits)* : Indique la taille de l'entête. La taille est mesurée en word (1word = 4octets= 32bits). S'il y a aucune option incluse dans le message, la taille de l'entête est égale à 5 words ;

- *Type of Service (1 octets)* : La valeur de ce champ indique si le paquet est traité avec une certaine priorité pour assurer la Qualité de Service (QoS : Quality of Service) ;
- *Taille totale du paquet (2 octets)* : Taille du paquet en octets ;
- *Identification (2 octets)* : Un identificateur est affecté à chaque paquet. Toutefois, le rôle de ce champ est différent de celui du numéro de séquence TCP. Lors de l'acheminement d'un paquet sur plusieurs réseaux, une (ou plusieurs) fragmentation du paquet peut être nécessaire. Ainsi, si un routeur doit découper un paquet, l'assemblage du paquet au niveau du destinataire sera effectué grâce à la valeur de ce champ car tous les fragments du même paquet auront la même valeur d'identification ;
- *Flags (3 bits)* : Trois bits qui indiquent si le paquet a été fragmenté ou pas. La valeur du flag indique aussi si le paquet peut être fragmenté ou pas ;
- *Offset (13 bits)* : Si le flag indique que le paquet est un fragment, la valeur du champ Offset indique où le fragment doit être placé pour ré-assembler le paquet originale au niveau de la destination;
- *Time to Live (1 octet)* : Indique la durée de vie du paquet. Chaque fois un paquet est retransmis par un routeur, la valeur TTL est décrémentée. Lorsque la valeur TTL d'un paquet est égale à 0, le paquet est supprimé. Cette valeur est utilisée pour éviter qu'un paquet soit transmis dans des boucles sans fin ;
- *Protocole (1 octet)* : Indique le protocole encapsulé (le protocole de la couche supérieure) par cet entête IP. Par exemple, si le protocole encapsulé est TCP, la valeur de ce champ sera égale à 6 (si UDP, la valeur sera 17decimal, 11hexa) ;
- *Checksum de l'entête (2 octets)* : Valeur de contrôle d'erreur de l'entête seulement (et non pas du paquet complet) ;
- *Adresse IP de la source (4 octets)* ;
- *Adresse IP de la destination (4 octets)* ;
- *Option (taille variable)* : Généralement inexistant. Ce champ peut être utilisé pour enregistrer les sauts dans une route vers la destination. De même, ce champs peut être utilisé pour spécifier la route vers la destination (Source Routing);
- *Padding (taille variable)* : Une suite d'octets pour compléter l'entête (car la taille de l'entête doit être multiple de 4 octets) ;

3. Flags

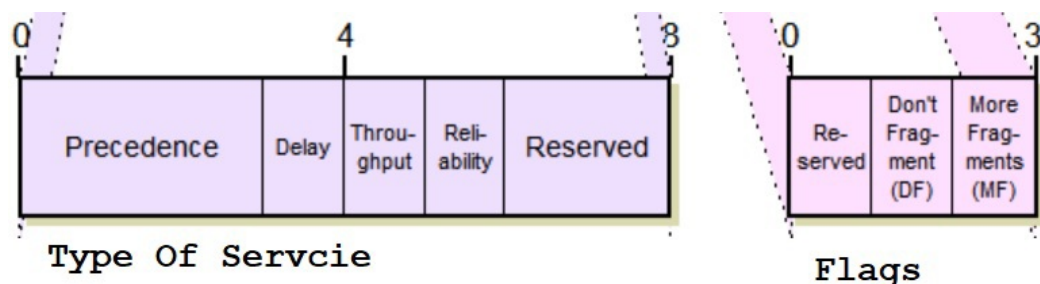


Figure IV.2. Flags de contrôle^{3*}

Flags :

1. Le premier bit est réserve (n'est pas utilisé) ;
2. Le deuxième bit : S'il est allumé signifie que ce paquet ne doit pas être fragmenté. Sinon (bit ==0), ce paquet peut être fragmenté ;

3. Le troisième bit : S'il est allumé indique qu'il y a d'autres fragments (*d'autres fragments qui vont suivre celui-ci*). Sinon (bit ==0), il y a pas d'autres fragments après celui-ci ;

Type of Service :

1. Les trois premiers bits indiquent la priorité du paquet appelé *Precedence* (000 == aucune priorité, 001 == paquet prioritaire, 010== immédiate, 011 == flash, 100== flash override, 101 == paquet critique, internet control : 110, 111 : network control) ;
2. Le 4em bit : Si ce bit est allumé, celui ci indique que *la transmission favorise que le délai soit minimal*. Sinon (bit ==0), la transmission est insensible au délai ;
3. Le 5em bit : Si ce bit est allumé, celui ci indique que *la transmission favorise le débit de transmission élevé*. Sinon (bit ==0), la transmission est insensible au débit ;
4. Le 6em bit : Si ce bit est allumé, celui ci indique que *la transmission favorise la fiabilité* (moins de paquets perdus). Sinon (bit ==0), la transmission est insensible à la fiabilité ;

4. Entête IPv6 principale (entête principale)

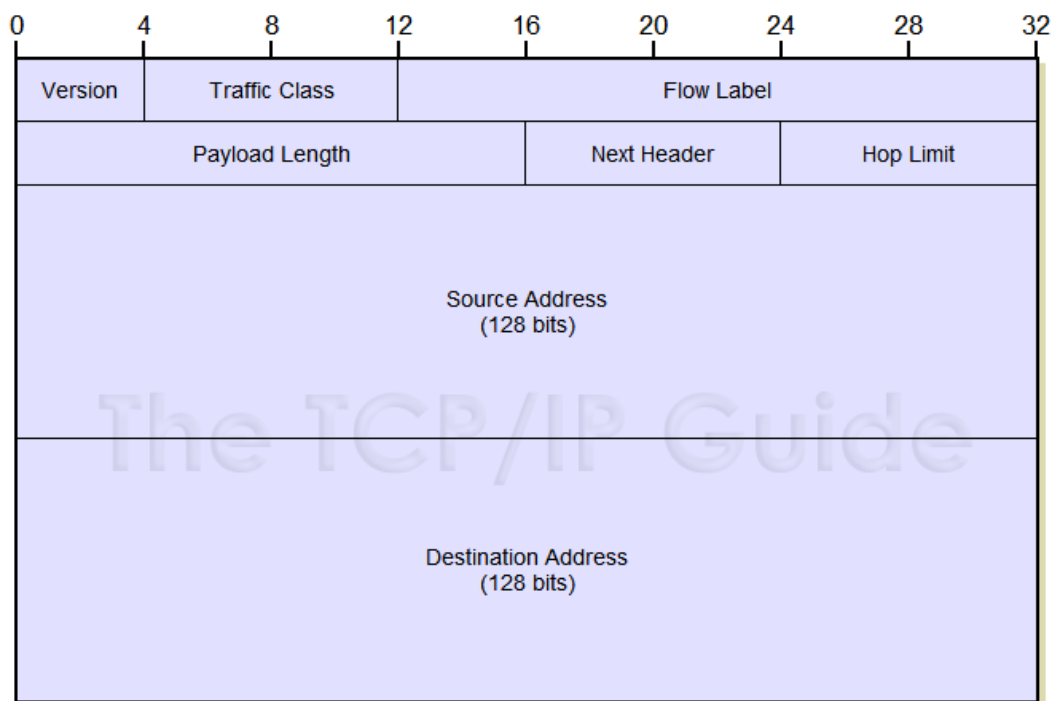


Figure IV.3. Entête IPv6^{3*}

- L'adresse de la source et destination sont de tailles 16 octets (128 bits) ;
- *Version* (la même signification que dans l'entête IPv4) prends une valeur 6 pour indiquer la version IPv6;
- *Traffic Class* (similaire au Type of Service dans le header IPv4) (1 octet) ;
- *Flow Label* (20 bits) : Les paquets appartenant au même trafic sont identifiés par le même identificateur du flux. L'identificateur de flux est utilisé pour assurer la QoS d'une manière uniforme par tous les routeurs appartenant au chemin de transmission ;
- *Payload Length* (2 octets) : Taille des données encapsulées (*payload* == *données encapsulées*);
- *Next Header* (1 octet) : Même que le champ *Protocol* dans l'entête IPv4 mais avec quelque changements ;
- *Hop Limit* (1 octet) : même signification que TTL ;

Next Header

- Le champ Options de IPv4 est implémenté sous forme d' Extension Header dans l'entête IPv6 ;
- Cette structure du paquet permet de limiter les parties que les routeurs doivent traiter. Ainsi, l'acheminement devient plus rapide ;
- Avec le protocole IPv6, seule la source est responsable de la fragmentation pour libérer les routeurs de cette charge. Pour effectuer ceci, la source découvre le plus petit Maximum Transmission Unit (MTU) dans la route et fragmente le paquet selon celui-ci ou bien utilise un MTU minimale supporté par défaut (1280 octets minimum pour IPv6). Cependant, si la route change au cours de la transmission et la fragmentation d'un paquet est nécessaire, la source doit redécouvrir le minimum MTU à nouveau ;

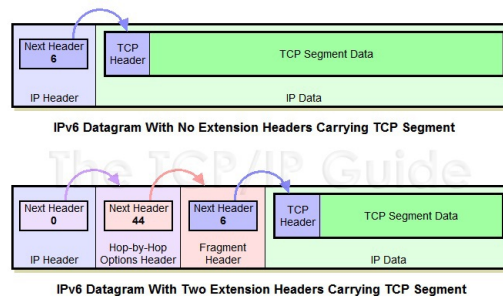


Figure IV.4. Next Header IPv6^{3*}

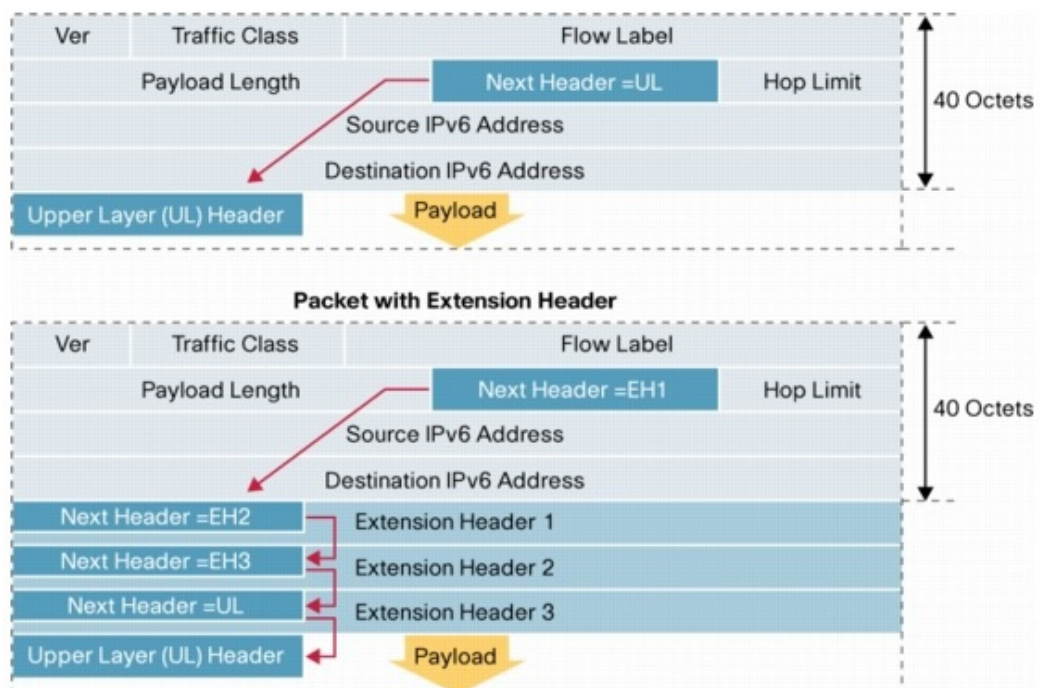


Figure IV.5. Chaînage des Extension Headers^{6*}

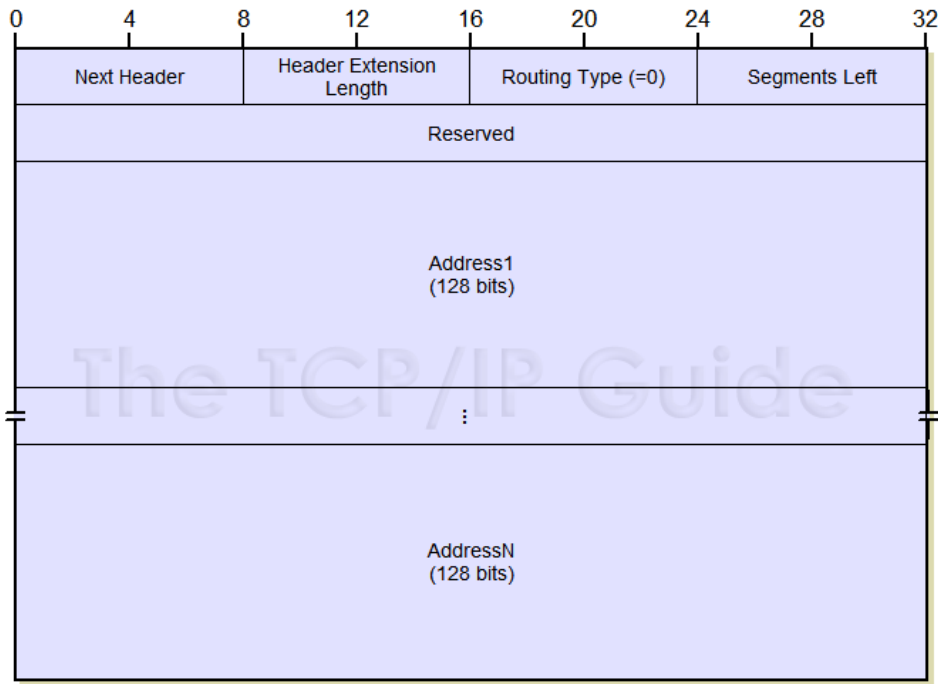


Figure IV.6. Format Routing EH^{3*}

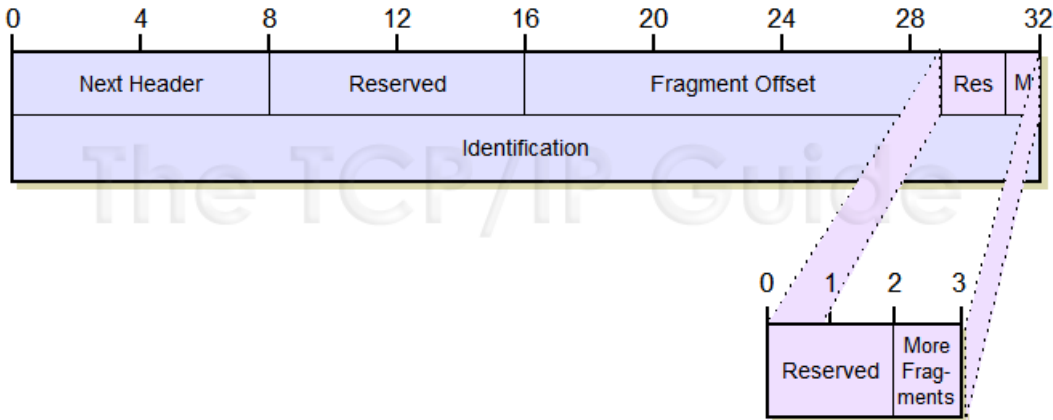


Figure IV.7. Format Fragment EH^{3*}

Chapitre V Adressage dans la Couche Couche Liaison

V

1. Couche Liaison

- Chaque interface permettant de connecter au réseau possède une adresse physique unique composée de 6 octets. Les trois premiers octets sont liés au fabricant de la carte (*Cisco, Huawei, Intel, DLink, etc*) et les trois derniers identifient la carte ;
- Cette adresse est appelée adresse MAC ou adresse physique ;
- Dans un réseau local, les machines sont adressées par leurs adresses MAC (c.à.d si un paquet est reçu et l'adresse IP du destinataire se situe dans le réseau local, l'adresse MAC du destinataire sera utilisée pour atteindre celui-ci);

Exemple

- Si un routeur souhaite livrer un paquet à une machine dans le réseau local, le routeur commence par déterminer l'adresse MAC de la machine avant ;
- Une fois que l'adresse MAC est déterminée, le paquet est transmis au destinataire ;

Dans Figure 7, les trois premiers octets de l'adresse MAC de la source et celle du récepteur sont remplacés par les noms des fabricants (*48 :51 :b7 est réservé pour Intel, 00 :66 :4b est réservé pour Huawei, pour un autre exemple 00:00:0C == Cisco et de 00:00:01 jusqu'à 00:00:08 Xerox*). Il existe des sites pour retrouver le nom du fabricant d'une carte réseau (<https://macvendors.com/> par exemple) ;

105...	136.197867	192.168.1.4	216.58.198.14	QUIC	73 Payload (Encrypted), PKN: 126, CID: 9964742710263775473
105...	136.907757	192.168.1.4	239.255.255.250	SSDP	222 M-SEARCH * HTTP/1.1
105...	137.789057	IntelCor_d4:c7:1d	HuaweiTe_6d:c1:aa	ARP	42 Who has 192.168.1.1? Tell 192.168.1.4
105...	137.792648	HuaweiTe_6d:c1:aa	IntelCor_d4:c7:1d	ARP	42 192.168.1.1 is at 00:66:4b:6d:c1:aa
105...	137.907762	192.168.1.4	239.255.255.250	SSDP	222 M-SEARCH * HTTP/1.1

>	Frame 3119: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
▼	Ethernet II, Src: HuaweiTe_6d:c1:aa (00:66:4b:6d:c1:aa), Dst: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)
>	Destination: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)
>	Source: HuaweiTe_6d:c1:aa (00:66:4b:6d:c1:aa)
>	Type: ARP (0x0806)
>	Address Resolution Protocol (request)

0000	48 51 b7 d4 c7 1d 00 66 4b 6d c1 aa 08 06 00 01	HQ.....f Km.....
0010	08 00 06 04 00 01 00 66 4b 6d c1 aa c0 a8 01 01	f Km.....
0020	00 00 00 00 00 00 c0 a8 01 04	

Figure V.1. Adresses MAC

Type d'adresse MAC

Dans un LAN, trois types d'adresses MAC sont rencontrées :

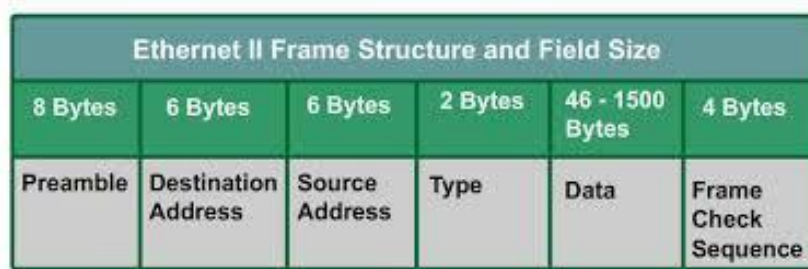
- *Adresse Broadcast* : Destiné à toutes les machines dans le réseau local au même temps (*tous les octets de l'adresse MAC == ff*) ;
- *Adresse Multicast* : Destiné à un groupe de machine spécifique (*valeur du premier octet = 01*) ;
- *Adresse Unicast* : Adresse physique d'une seule machine ;

Remarque : Adresse du transmetteur

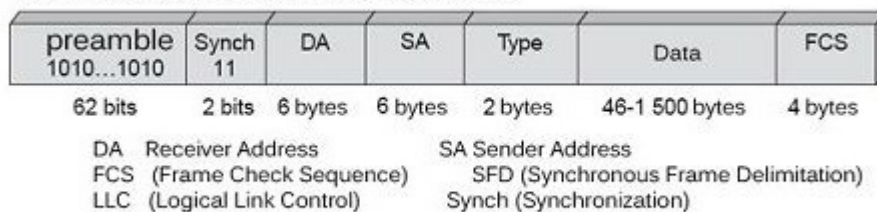
L'adresse du transmetteur est toujours une adresse Unicast (une trame ne peut pas être générée par plusieurs machines au même temps).

2. Trame Ethernet

- Après avoir insérer l'adresse MAC du/des destinataire(s) visé(s), la trame est émise sous forme de signal (électrique ou onde radio) par la carte réseau.
- Selon le type de la topologie physique du réseau et le médium de transmission, un mécanisme pour éviter la collision (*collision avoidance*) peut être implémenté afin d'éviter que les signaux émis se chevauchent.



Format of the Former Ethernet Frame



Format of the two types of Ethernet Frames

Figure V.2. Format trame envoyée sur le medium (Ethernet Type II)^{5*}

- *Preamble* (8 octets): une suite de 1 et 0 à qui s'alternent pour donner du temps au récepteur pour commencer la réception de la trame. Les deux derniers bits seront tout les deux allumés (*Synch == 11*) pour indiquer que la transmission va commencer ;
- Adresse MAC de la destination (6 octets);
- Adresse MAC de la source (6 octets);
- *Type* (appelé aussi Control) (2 octets) : Type du protocole encapsulé (IPv4, IPv6, ARP, etc) ;
- Données (entre 46 et 1500 octets) ;

- *Frame Check Sequence [FCS]* (4 octets) : Somme de contrôle de la trame entière (pas l'entête seulement) pour détecter les erreurs de transmission;

3. ARP (Address Resolution Protocol)

- Pour communiquer dans le réseau local, il est nécessaire d'obtenir l'adresse MAC du destinataire ;
- Le protocole ARP permet de rechercher l'adresse MAC correspondante à une adresse IP appartenant au réseau local;

Un message ARP contient les champs suivants :

1. *Hardware Type* (2 octets) : type matériel utilisé pour communiquer en réseau local ;
2. *Protocol Type* (2 octets) : type de l'adresse utilisée pour la couche 3, généralement IPv4 (2048 indique IPv4, ou bien 0800 hexadécimal);
3. *Hardware Address Size* (1 octets) :taille de l'adresse physique , 6 pour indiquer qu'il s'agit d'une adresse MAC ;
4. *Protocol Address Size* (1 octets) :taille de l'adresse de la couche supérieure, 4 pour indiquer qu'il s'agit d'une adresse IPv4 ;
5. *OPcode* (2 octets): Indique le type de l'opération (1 pour requête de recherche, 2 pour indiquer que c'est une réponse à une requête, il existe d'autres moins utilisées) ;
6. *Adresse MAC de la source* ;
7. *Adresse IP de la source* ;
8. *Adresse MAC de la destination* ;
9. *Adresse IP de la destination* ;

4. ARP Request (recherche d'une adresse MAC dans le réseau local)

- Un transmetteur commence par vérifier ça table ARP pour voir s'il y a une adresse MAC correspondante à l'adresse IP du destinataire. Les entrées de la table ARP ont une durée de vie limitée (sur Windows, l'entrée ARP expirent après 2 minutes).
- Lorsque une machine souhaite retrouver l'adresse MAC d'un destinataire, celle-ci transmet un message de type ARP request (*OPcode == 1*) ;
- Cette requête doit être diffusée dans tout le réseau, ainsi l'adresse destination dans la trame sera l'adresse de diffusion (*ff:ff:ff:ff:ff:ff*) et l'adresse source prendra l'adresse MAC de la machine qui souhaite rechercher l'adresse MAC ;
- Dans ce type de message trois adresses sont connues pour la source (l'adresse MAC de la source, l'adresse IP de la source et l'adresse IP de la destination). Ces valeurs sont inscrites dans les champs associés du message ARP [*champ 6,7,9 du message ARP*]. La seule information manquante est l'adresse MAC du destinataire [*champ 8 du message ARP*]. Ainsi, la valeur de ce champ est remplie par des 0 ;

No.	Time	Source	Destination	Protocol	Length	Info
1224	46.895297	IntelCor_d4:c7:1d	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.2
1225	46.896981	HuaweiTe_6d:c1:aa	IntelCor_d4:c7:1d	ARP	42	192.168.1.1 is at 00:66:4b:6d:c1:aa
1226	47.790534	IntelCor_d4:c7:1d	Broadcast	ARP	42	Who has 192.168.1.2? Tell 0.0.0.0
1229	48.787992	IntelCor_d4:c7:1d	Broadcast	ARP	42	Gratuitous ARP for 192.168.1.2 (Request)
1230	48.825371	IntelCor_d4:c7:1d	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.2

> Frame 1230: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0

> Ethernet II, Src: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

✓ Address Resolution Protocol (request)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: request (1)

Sender MAC address: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)

Sender IP address: 192.168.1.2

Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)

Target IP address: 192.168.1.1

Figure V.3. ARP request

5. ARP Reply

Lorsque la machine possédant l'adresse IP est atteinte par la requête ARP, celle-ci répond en insérant son adresse MAC. Cette réponse aura *Opcode* == 2 pour indiquer qu'il s'agit d'un "ARP reply" ;

No.	Time	Source	Destination	Protocol	Length	Info
1226	47.790534	IntelCor_d4:c7:1d	Broadcast	ARP	42	Who has 192.168.1.2? Tell 0.0.0.0
1229	48.787992	IntelCor_d4:c7:1d	Broadcast	ARP	42	Gratuitous ARP for 192.168.1.2 (Request)
1230	48.825371	IntelCor_d4:c7:1d	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.2
1231	48.827082	HuaweiTe_6d:c1:aa	IntelCor_d4:c7:1d	ARP	42	192.168.1.1 is at 00:66:4b:6d:c1:aa
1241	48.904849	HuaweiTe_6d:c1:aa	Broadcast	ARP	60	Who has 192.168.1.2? Tell 192.168.1.1

Frame 1231: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0

Ethernet II, Src: HuaweiTe_6d:c1:aa (00:66:4b:6d:c1:aa), Dst: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)

Address Resolution Protocol (reply)

Hardware type: Ethernet (1)

Protocol type: IPv4 (0x0800)

Hardware size: 6

Protocol size: 4

Opcode: reply (2)

Sender MAC address: HuaweiTe_6d:c1:aa (00:66:4b:6d:c1:aa)

Sender IP address: 192.168.1.1

Target MAC address: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)

Target IP address: 192.168.1.2

Figure V.4. ARP reply

6. Gratuitous ARP

Une machine peut tester si son adresse IP est utilisée par une autre machine avec un message ARP Request. Ce type de message est appelé Gratuitous ARP. Dans Figure 11, la machine 192.168.1.2 transmet une requête ARP avec ça propre adresse dans le champ destination (192.168.1.2) pour vérifier s'il y a une autre machine possédant la même adresse IP ;

arp						
No.	Time	Source	Destination	Protocol	Length	Info
1223	46.886361	HuaweiTe_6d:c1:aa	IntelCor_d4:c7:1d	ARP	42	192.168.1.1 is at 00:66:4b:6d:c1:aa
1224	46.895297	IntelCor_d4:c7:1d	Broadcast	ARP	42	Who has 192.168.1.1? Tell 192.168.1.2
1225	46.896981	HuaweiTe_6d:c1:aa	IntelCor_d4:c7:1d	ARP	42	192.168.1.1 is at 00:66:4b:6d:c1:aa
1226	47.790534	IntelCor_d4:c7:1d	Broadcast	ARP	42	Who has 192.168.1.2? Tell 0.0.0.0
1229	48.787992	IntelCor_d4:c7:1d	Broadcast	ARP	42	Gratuitous ARP for 192.168.1.2 (Request)

> Frame 1229: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
 > Ethernet II, Src: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
 > Address Resolution Protocol (request/gratuitous ARP)
 Hardware type: Ethernet (1)
 Protocol type: IPv4 (0x0800)
 Hardware size: 6
 Protocol size: 4
 Opcode: request (1)
 [Is gratuitous: True]
 Sender MAC address: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)
 Sender IP address: 192.168.1.2
 Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
 Target IP address: 192.168.1.2

Figure V.5. Gratuitous ARP

Remarque

- Il existe un protocole RARP (Reverse ARP) qui offre la fonctionnalité inverse de ARP (à partir de l'adresse MAC d'une machine, ce dernier retrouve l'adresse IP).
- Un message ARP request (Opcode=1) avec l'adresse IP source = 0.0.0.0 est dit ARP Probe. Ce type de message est généré par une machine à fin de vérifier si l'adresse IP recherchée n'est pas utilisée. Si aucune machine ne répond avec un ARP response, la machine qui a sollicité le réseau peut utiliser cette adresse pour opérer.
- Après la récupération de l'adresse MAC d'un destinataire avec ARP, la machine continue périodiquement à transmettre un ARP request avec l'adresse MAC destination dans la trame égale à celle reçue (au lieu de l'adresse de diffusion). Ceci est fait pour tester si l'adresse MAC est toujours valide et la machine n'a pas quitter le réseau.

Chapitre VI Internet Control Message Protocol

VI

1. Diagnostic du réseau IP avec Internet Control Message Protocol (ICMP)

- Internet Control Message Protocol (ICMP) est un protocole permettant de diagnostiquer le fonctionnement du réseau et de signaler des erreurs de routage ;
- ICMP fonctionne au niveau de la Couche Internet du modèle TCP / IP ;
- Plusieurs commandes utilisent ICMP pour fonctionner (ping, tracert / traceroute).
- Les messages générés par ce protocole sont encapsulés par un entête IP. Si dans un message IPv4 la *valeur du champs Protocol est égale à 1*, le message encapsulé est un message ICMP ;
- La plus part des machines fonctionnant sur Internet sont configurées pour répondre aux messages ICMP (comme ping par exemple);

2. Structure du message ICMP

- La structure du message ICMP change selon la valeur du champ *type*;
- Les champs suivants qui peuvent être présents dans un message ICMP sont :
 1. *Type [présent dans tous les messages ICMP]*: Valeur numérique qui indique le type du message ICMP. Il existe 16 types de messages ICMP mais ceux fréquemment rencontrés sont :
 - 0—Echo reply
 - 3—Destination unreachable
 - 5—Redirect
 - 8—Echo request
 - 11—Time Exceeded
 2. *Code [présent dans tous les messages ICMP]*: La valeur du code prend souvent 0. Si ce champ prend une valeur différent de 0, cela indique que le message ICMP représente une variation d'un type de message (*champ I*). Pour les messages ICMP qui n'ont pas de variantes, le champ *code* prends toujours la valeur 0 ;
 3. *Checksum [présent dans tous les messages ICMP]*: Valeur de contrôle du message ICMP entier ;
 4. *Identifiant* : N'est pas présent dans tous les types de messages ICMP. Cette valeur peut faire référence à un message ICMP précédant qui a provoqué l'envoi de ce message ICMP ;
 5. *Numéro de séquence* : Comme l'identifiant, la valeur de ce champ est utilisée pour faire la correspondance entre le *Echo Request* et le *Echo Reply* ;
 6. *Copie de l'entête IP + copie de 64 bits des données encapsulées* (8 octets) : Ce champs est présent dans certains types de message ICMP pour faire référence au message qui a provoqué la génération de ce dernier ;
 7. *Payload*: Les données enveloppées par le message ICMP ;

3. Echo Request (Type ==8) et Echo Reply (Type ==0)

- La commande *ping* présente sur Windows et Linux génère un message *ICMP Echo Request (Type == 8)* ;
- Le destinataire, qui correspond à l'adresse IP fournie, répond avec un message *ICMP Echo Reply (Type == 0)*;
- Le payload d'un message Echo Request contient une suite d'octets. Les octets insérés dans ce champ dépendent du système d'exploitation. Par exemple : Windows génère un payload de 32 octets contenant des caractères alphabétiques (de 'a' jusqu'à 'z' et ensuite de 'a' jusqu'à 'i'), Ubuntu génère une suite de 48 octets avec une suite de caractères différente) ;
- Le Echo Reply correspondant à un Echo Request aura la même valeur d'identification, le même numéro séquence et la même suite d'octets constituant le payload ;

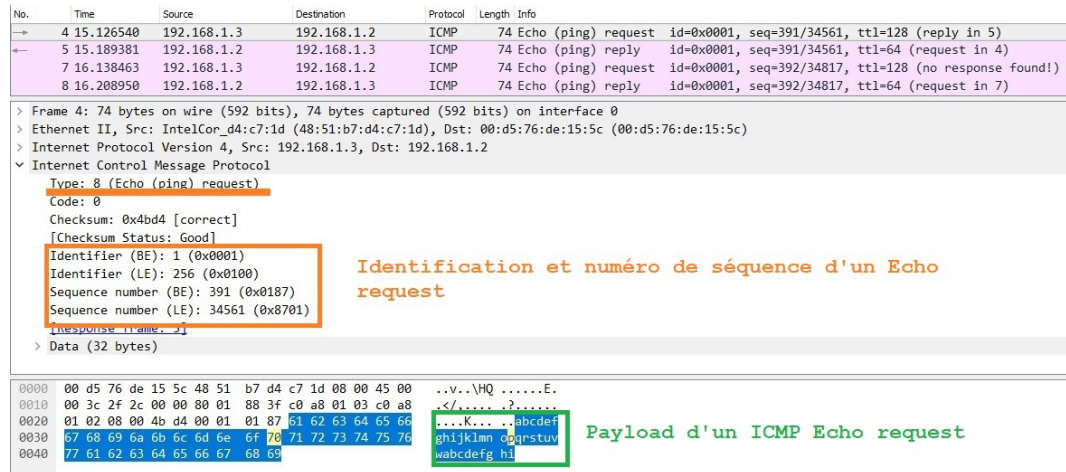


Figure VI.1. Echo Request

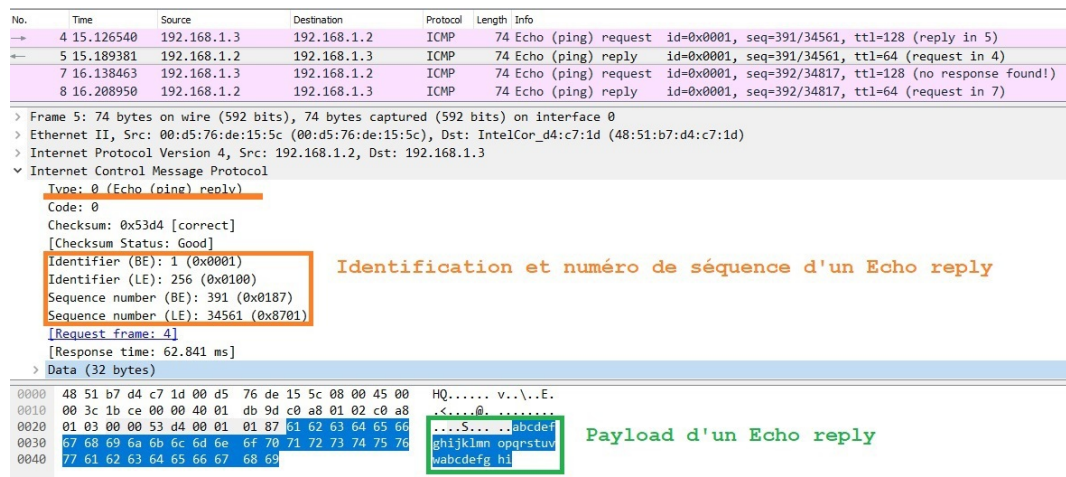


Figure VI.2. Echo Reply

Remarque : Identification et numéro de séquence

Wireshark affiche l'identification et le numéro de séquence d'un message ICMP sous deux formats (BE == Big Endian , LE == Little Endian).

4. ICMP Time Exceeded (Type ==11)

- Dans le cas où un paquet encapsulé avec un entête IP est supprimé à cause de l'expiration du TTL (TTL == 0), le routeur qui supprime le paquet transmet un *message ICMP avec Time Exceeded (Type == 11)* vers la source du paquet supprimé ;
- Ceci informe la source que le paquet a potentiellement traversé une boucle de routage (routing loop) ;
- Contrairement à Echo reply / request, un message ICMP Time Exceeded ne contient pas le champ identification / numéro de séquence. Ce type de message contient une copie du message qui a expiré (TTL a atteint 0) ;
- Le deuxième cas d'utilisation pour Time Exceeded est lorsque un fragment est perdu/supprimé ainsi le reste des fragments ne peuvent pas être livrés à la couche transport. Dans ce cas, un message *Time Exceeded (Type == 11)* avec code =1 est envoyé (Temps de ré-assemblage des fragments du datagramme dépassé) ;

 Remarque : TTL

- Chaque fois qu'un paquet est retransmis par un routeur, la valeur du TTL est décrémentée par au moins 1. Dans certaines situations, le TTL est décrémenté par une valeur supérieure à 1;
- La commande *Tracert* (traceroute sur Linux) utilise ICMP Time Exceeded pour retrouver les sauts sur le chemin vers la destination. La source transmet des messages *ICMP Echo Request avec des TTL incrémentés* de 1 jusqu'à ce que la source est atteinte. La transmission d'un *echo request* avec un TTL ==N permet de traverser N sauts seulement (car la machine positionnée dans le saut numéro N supprimera le paquet puisque le TTL initial a été décrémenté N fois au moins pour l'atteindre). Chaque fois qu'une machine sur la route supprime le message à cause de l'expiration du TTL, elle renvoie un ICMP Time Exceeded vers la source. Ainsi, la source détermine le générateur du ICMP Type ==11 est positionnée sur la route à la N^{em} position ;
- Lorsque l'administrateur réseau souhaite qu'une machine ne soit pas visible dans la route tracée par cette commande, il suffit de configurer celle-ci pour ne pas décrémenter le TTL ;

[illegible]

No.	Time	Source	Destination	Protocol	Length	Info
71	29.564538	192.168.1.4	172.217.19.131	ICMP	106	Echo (ping) request id=0x0001, seq=80/20480, ttl=4 (no response found!)
72	29.592381	172.28.16.13	192.168.1.4	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
83	35.188809	192.168.1.4	172.217.19.131	ICMP	106	Echo (ping) request id=0x0001, seq=81/20736, ttl=5 (no response found!)
84	35.213918	172.28.16.14	192.168.1.4	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)

> Frame 72: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface 0
 > Ethernet II, Src: HuaweiTe_6d:c1:aa (00:66:4b:6d:c1:aa), Dst: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)
 > Internet Protocol Version 4, Src: 172.28.16.13, Dst: 192.168.1.4

▼ Internet Control Message Protocol
 Type: 11 (Time-to-live exceeded)
 Code: 0 (Time to live exceeded in transit)
 Checksum: 0xf4ff [correct]
 [Checksum Status: Good]

> Internet Protocol Version 4, Src: 192.168.1.4, Dst: 172.217.19.131
 ▼ Internet Control Message Protocol
 Type: 8 (Echo (ping) request)
 Code: 0
 Checksum: 0xf7ae [unverified] [in ICMP error packet]
 [Checksum Status: Unverified]
 Identifier (BE): 1 (0x0001)
 Identifier (LE): 256 (0x0100)
 Sequence number (BE): 80 (0x0050)
 Sequence number (LE): 20480 (0x5000)

ICMP Type 11 avec l'adresse du 4em saut

Copie du message original qui a engendré le message ICMP Type 11

```
(c) 2018 Microsoft Corporation. Tous droits réservés.

C:\Users\DVR>tracert google.dz

Détermination de l'itinéraire vers google.dz [172.217.19.131]
avec un maximum de 30 sauts :

  1  1 ms  <1 ms  <1 ms  192.168.1.1
  2  16 ms  15 ms  15 ms  192.168.1.1
  3  22 ms  20 ms  21 ms  192.168.1.1
  4  28 ms  28 ms  28 ms  172.28.16.13
  5  25 ms  25 ms  25 ms  172.28.16.14
  6  25 ms  24 ms  24 ms  172.17.116.16
  7  43 ms  43 ms  42 ms  72.14.205.138
  8  48 ms  49 ms  48 ms  108.170.252.225
  9  51 ms  51 ms  52 ms  66.249.94.127
 10  42 ms  42 ms  42 ms  par03s12-in-f131.1e100.net [172.217.19.131]

Itinéraire déterminé.

C:\Users\DVR>
```

Figure VI.4. Fonctionnement de la commande Tracert

Exemple : Boucle de routage

Par exemple, Routeur 3 a une route manuellement configurée vers le réseau 192.168.1.0 et Routeur 2 est configuré avec le Routeur 3 comme route par défaut. Si la route vers 192.168.1.0 est supprimée de Routeur 2 à cause d'une panne, lorsque Routeur 3 souhaite communiquer avec 192.168.1.0, une boucle de routage se formera : car Routeur 3 utilise Routeur 2 pour atteindre 192.168.1.0, et Routeur 2 n'a pas de route vers 192.168.1.0 et donc utilisera la route par défaut (Routeur 3). Ainsi, Routeur 2 et Routeur 3 s'échangeront le message destiné au réseau 192.168.1.0 jusqu'à ce que le TTL du paquet atteindra 0 et un message ICMP Type 11 est transmis vers la source de cette transmission ;

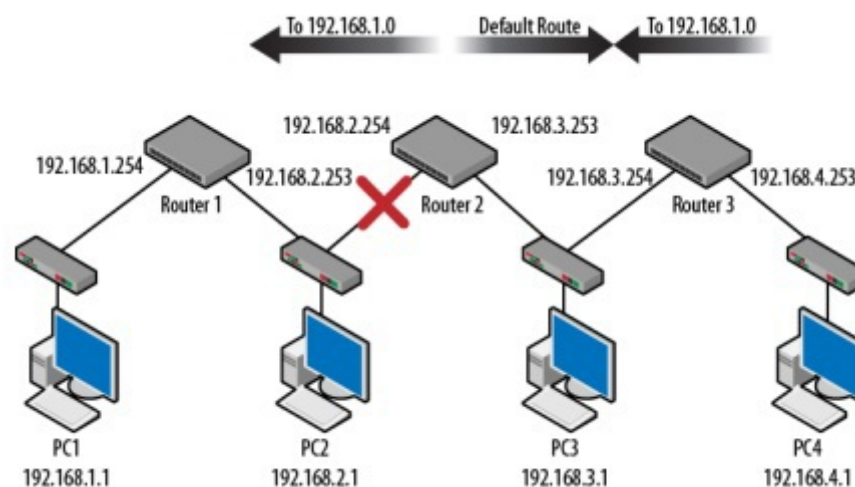


Figure VI.5. Boucle de routage^{5*}

5. ICMP Destination Unreachable (Type ==3)

Un message ICMP de type 3 est généré quand le paquet ne trouve pas une route correspondante à la destination ou ne peut pas être transmis vers celle-ci. Les codes (*variations*) suivants possibles pour ce type de message sont :

0—*Net unreachable* : Le réseau n'est pas atteignable ;

1—*Host unreachable* : La machine n'est pas atteignable ;

2—*Protocol unreachable* : Le protocole de transport utilisé n'est pas supporté ;

3—*Port unreachable* : La destination n'a pas de service correspondant au numéro de port et il n'existe pas un mécanisme pour informer la source (*comme c'est le cas pour UDP !*);

4—*Fragmentation needed and DF (do not fragment) set* : Le paquet ne peut pas être transmis car Do Not Fragment == 1 et la fragmentation a été requise ;

5—*Source route failed* : Le message original utilise Source Routing et cette option est bloquée par le routeur ;

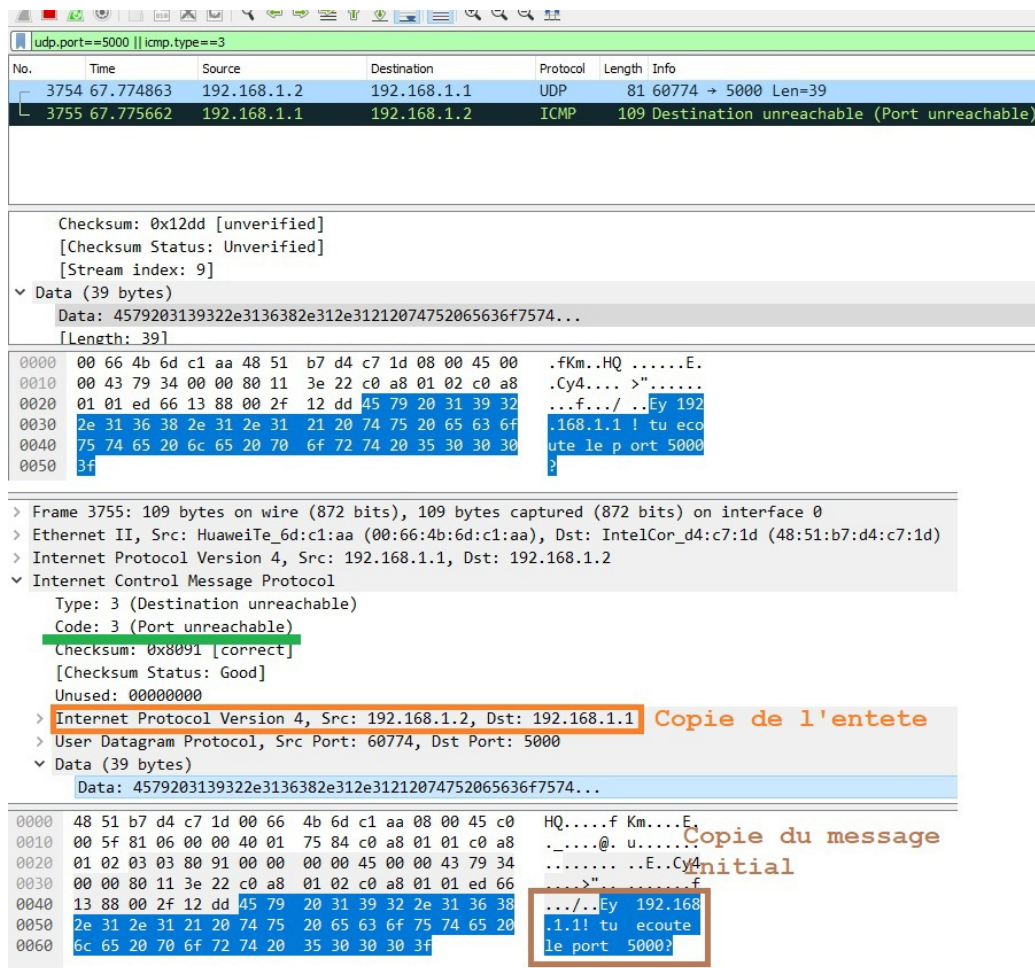
13—*Communication administratively filtered* : Le paquet a été bloqué à cause d'un filtre administrative basé sur le contenu (par exemple le firewall [*pare-feu*] est configuré pour bloquer les paquets destinés au port 137) ;

No.	Time	Source	Destination	Protocol	Length	Info
23259	294.653027	192.168.1.2	10.103.13...	TCP	66	51363 → 23 [SYN] Seq=0 Win=17520 Len=0 MSS=1460 WS=256 SACK_PERM=1
23260	294.670017	10.103.13...	192.168.1.2	ICMP	70	Destination unreachable (Communication administratively filtered)
23270	297.653342	192.168.1.2	10.103.13...	TCP	66	[TCP Retransmission] 51363 → 23 [SYN] Seq=0 Win=17520 Len=0 MSS=14

> Frame 23260: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface 0
> Ethernet II, Src: HuaweiTe_6d:c1:aa (00:66:4b:6d:c1:aa), Dst: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)
> Internet Protocol Version 4, Src: 10.103.13.45, Dst: 192.168.1.2
▼ Internet Control Message Protocol
Type: 3 (Destination unreachable)
Code: 13 (Communication administratively filtered)
Checksum: 0x38d4 [correct]
[Checksum Status: Good]
Unused: 00000000
> Internet Protocol Version 4, Src: 192.168.1.2, Dst: 10.103.13.45
> Transmission Control Protocol, Src Port: 51363, Dst Port: 23

Figure VI.6. Communication administratively filtered

Chacun de ces messages ICMP type 3 contient une copie de l'entête et du payload [*champ 7 du message ICMP*] du message que le routeur a échoué de livrer.



udp.port==5000 || icmp.type==3

No.	Time	Source	Destination	Protocol	Length	Info
3754	67.774863	192.168.1.2	192.168.1.1	UDP	81	60774 → 5000 Len=39
3755	67.775662	192.168.1.1	192.168.1.2	ICMP	109	Destination unreachable (Port unreachable)

Checksum: 0x12dd [unverified]
[Checksum Status: Unverified]
[Stream index: 9]
Data (39 bytes)
Data: 4579203139322e3136382e312e31212074752065636f7574...
[Length: 39]

0000 00 66 4b 6d c1 aa 48 51 b7 d4 c7 1d 08 00 45 00 .fKm..HQ.....E.
0010 00 43 79 34 00 00 80 11 3e 22 c0 a8 01 02 c0 a8 .Cy4....>.....
0020 01 01 ed 66 13 88 00 2f 12 dd 45 79 20 31 39 32 ...f.../..Ey 192
0030 2e 31 36 38 2e 31 2e 31 21 20 74 75 20 65 63 6f .168.1.1! tu eco
0040 75 74 65 20 6c 65 20 70 6f 72 74 20 35 30 30 30 ute le p ort 5000
0050 3f

> Frame 3755: 109 bytes on wire (872 bits), 109 bytes captured (872 bits) on interface 0
> Ethernet II, Src: HuaweiTe_6d:c1:aa (00:66:4b:6d:c1:aa), Dst: IntelCor_d4:c7:1d (48:51:b7:d4:c7:1d)
> Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.2
> Internet Control Message Protocol
Type: 3 (Destination unreachable)
Code: 3 (Port unreachable)
Checksum: 0x8091 [correct]
[Checksum Status: Good]
Unused: 00000000
> Internet Protocol Version 4, Src: 192.168.1.2, Dst: 192.168.1.1 **Copie de l'entete**
> User Datagram Protocol, Src Port: 60774, Dst Port: 5000
Data (39 bytes)
Data: 4579203139322e3136382e312e31212074752065636f7574...
0000 48 51 b7 d4 c7 1d 00 66 4b 6d c1 aa 08 00 45 c0 HQ.....f Km....E.
0010 00 5f 81 06 00 00 40 01 75 84 c0 a8 01 01 c0 a8@. u.....
0020 01 02 03 03 80 91 00 00 00 00 45 00 00 43 79 34E..Cy4
0030 00 00 80 11 3e 22 c0 a8 01 02 c0 a8 01 01 ed 66>.....f
0040 13 88 00 2f 12 dd 45 79 20 31 39 32 2e 31 36 38 .../..Ey 192.168
0050 2e 31 2e 31 21 20 74 75 20 65 63 6f 75 74 65 20 .1.1! tu eco
0060 6c 65 20 70 6f 72 74 20 35 30 30 30 3f ute le port 5000?

Figure VI.7. Message ICMP Type ==3 Code ==3

Remarque : Traceroute sur Linux

Alors que `tracert` sur Windows utilise *echo request* et *ICMP TTL Exceeded*, la commande `Traceroute` sur Linux génère des *datagrames UDP* avec un *TTL* progressivement incrémenté et attend de recevoir le message *ICMP TTL Exceeded* par les machines intermédiaires. La destination finale répondra généralement avec un *ICMP Destination Unreachable (Type 3) Port Unreachable (Code 3)*.

6. ICMP Redirect (Type ==5)

Un message *ICMP Redirect (Type ==5)* est généré pour informer la source d'un paquet que ça destination est joignable par un plus court chemin *dans le même réseau*. Ainsi, le routeur qui génère un paquet *ICMP Redirect* envoie la nouvelle route vers la source. Ensuite, la source ajoute une nouvelle entrée dans ça table de routage vers la destination souhaitée avec le prochain saut égale à la route spécifiée par le *ICMP Redirect*.

☞ *Exemple : Exemple scénario ICMP Redirect*

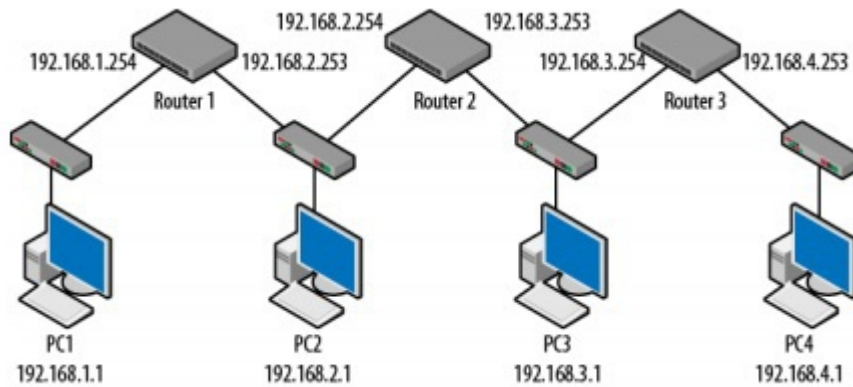


Figure VI.8. Exemple ICMP Redirect^{5*}

- Dans l'exemple présenté dans Figure 7, PC3 est configuré avec Routeur 2 comme route par défaut et ne connaît pas de route vers le réseau 192.168.4.0 (où PC4 se trouve). En outre, *Routeur 2 connaît toutes les routes vers les réseaux atteignables*. Ainsi, quand PC 3 souhaite transmettre des paquets vers PC4, celui-ci utilise Routeur 2.
- Routeur 2 détermine que PC3 possède une route plus courte vers PC4 (car *pour Routeur 2 le prochain saut vers 192.168.4.0 est Routeur 3 && Routeur 3 et PC3 sont dans le même réseau*). Ainsi, *Routeur 2 envoie un message ICMP Redirect vers PC3 pour lui dire qu'il peut rejoindre PC4 à travers Routeur 3* ;

Si la route est explicitement définie par la source (*Source Routing est appliqué*), le routeur ne génère pas de ICMP Redirect même s'il y a une meilleure route ;

Chapitre VII Routage et Protocoles de routage

VII

1. Routage

Le routage IP consiste à traiter l'entête IP pour savoir sur quelle interface transmettre le paquet. Un système implémentant le modèle TCP/IP peut avoir plusieurs interface réseau (interfaces RJ45, Bluetooth, Wifi, Satellite, Fibre etc) mais doit avoir au moins une interface réseau *loopback*. Celle-ci est un moyen pour que une machine communique avec elle même.

Supposant qu'une trame est reçue qui encapsule un paquet IP sur une interface réseau. La figure suivante résume le traitement effectué sur l'entête IP afin d'acheminer celui-ci :

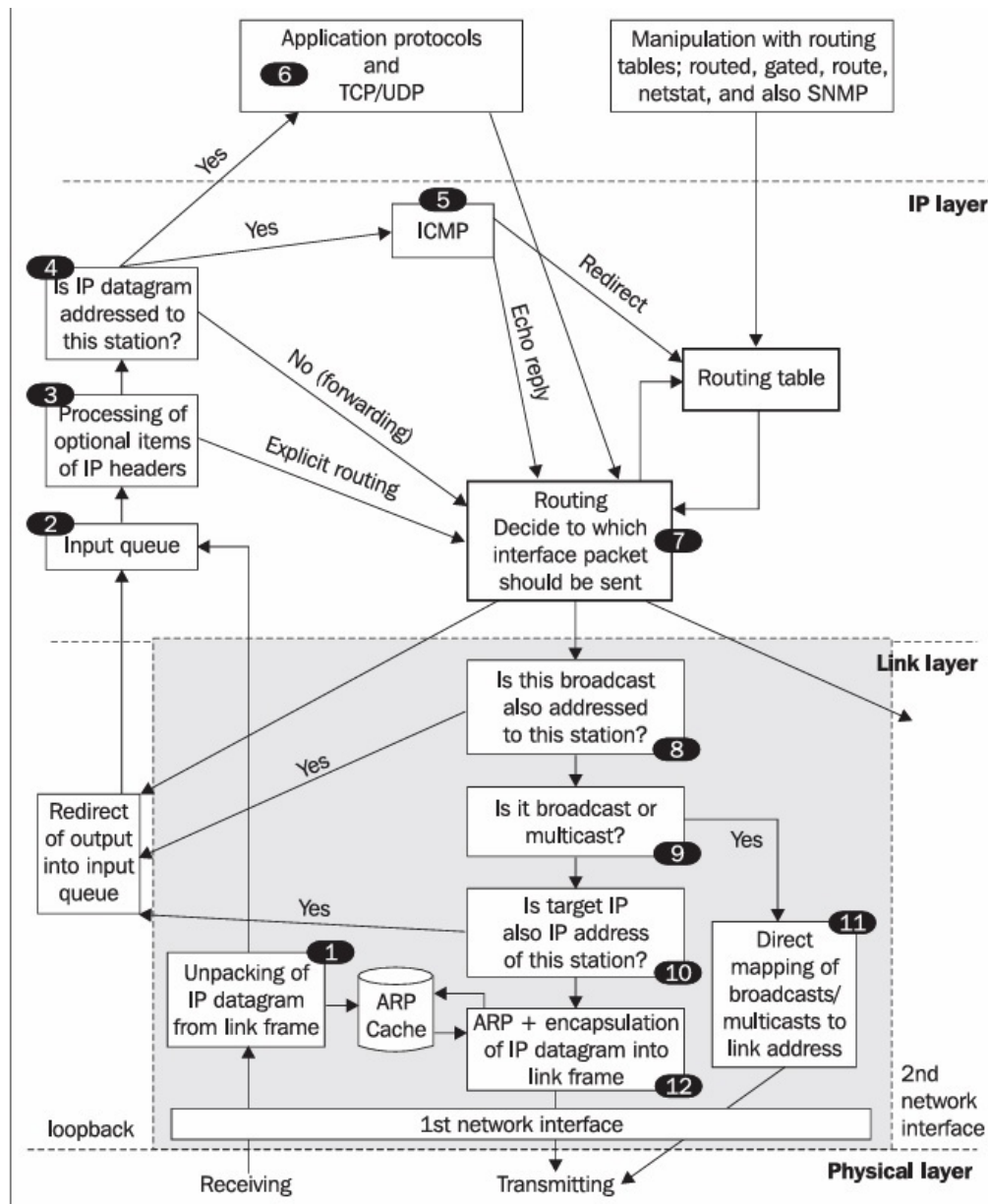


Figure VII.1. Traitement de l'entête IP pour acheminement^{7*}

1. Tout d'abord, le paquet IP est extrait de la trame.
2. Le paquet IP est placé dans une file d'attente avant d'être traité.
3. Le paquet ensuite est relevé de la file d'attente. Le premier traitement effectué consiste à vérifier si le champ IP Options exige le Source Routing. Si oui, la machine retransmet le paquet vers le prochain saut seulement si l'adresse de la machine actuelle appartient au chemin contenu dans l'option Source Routing.
4. Si l'adresse IP destination correspond à l'une des adresses des interfaces de la machine actuelle, le paquet est transféré à l'interface *loopback*. La machine reconnaîtra que le paquet est destiné à elle même en examinant ça table de routage. Celle-ci contiendra un chemin exacte (masque égale à 255.255.255.255) à chaque interface de la machine actuelle.
5. Si le paquet est destiné à la machine et le contenu est encapsulé avec ICMP, le paquet est traité par la couche IP et une réponse est transféré au processus de routage.
6. Si le paquet est destiné à la machine et le contenu encapsulé est TCP/UDP, le contenu est transféré à la couche supérieure.

7. L'étape de routage/acheminement consiste simplement à trouver l'interface de sortie et le prochain saut pour relayer le paquet. Pour chaque paquet placé pour le routage, le processus de routage prend l'adresse IP destination et applique le masque (*ADDING / XOR logique*) de chaque adresse réseau destination contenue dans la table de routage. Chaque résultat du *ADDING* est comparé avec l'adresse réseau destination correspondantes. Cette opération est exécutée d'une manière optimale en terme de complexité grâce à une structure de donnée appelée Trie. Le résultat le plus exacte (longest match entre le résultat du *ADDING* et adresse réseau destination) est pris et l'interface de sortie / prochain saut correspondants sont pris pour l'acheminement du paquet.
8. Le processus de routage ensuite vérifie si l'adresse destination est une adresse broadcast/diffusion dans le réseau local. Si c'est le cas une copie du message est transféré au *loopback*.

Le reste du processus affiché dans la figure consiste à trouver l'adresse MAC correspondante à l'adresse IP du prochain saut. Si l'adresse MAC est unicast, ARP et la table ARP sont utilisés pour trouver l'adresse MAC du prochain saut dans le réseau local.

Remarque

- La machine recevant un paquet IP peut mettre à jour l'entrée de la table ARP associée à la source du paquet.
- L'option Source Routing est généralement désactivée pour des raisons de sécurité claires. L'activation de cette option permet de créer facilement des attaques DOS (plusieurs trafics volumineux peuvent être dirigés à travers le même routeur pour le faire tomber en panne).
- Lorsque une machine transfère un paquet au *loopback*, le contenu du paquet est passé à la couche / processus correspondant (ICMP, TCP ou UDP).
- La table de routage peut être affichée par Windows avec la commande *route print* dans le cmd.

```
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\ilyas>route print

=====
Liste d'Interfaces
 8...74 56 3c 9b 91 ba .....Realtek PCIe GbE Family Controller #3
10...48 ee 0c c9 75 a7 .....Microsoft Wi-Fi Direct Virtual Adapter
 9...48 ee 0c c9 75 a0 .....Microsoft Wi-Fi Direct Virtual Adapter #2
 6...48 ee 0c c9 75 a5 .....D-Link DWA-140 Wireless N USB Adapter(rev.B3)
 1.....Software Loopback Interface 1
=====

IPv4 Table de routage
=====
Itinéraires actifs :
Destination réseau    Masque réseau    Adr. passerelle    Adr. interface    Métrique
 0.0.0.0              0.0.0.0          192.168.1.1        192.168.1.3       50
127.0.0.0             255.0.0.0        On-link            127.0.0.1         331
127.0.0.1             255.255.255.255  On-link            127.0.0.1         331
127.255.255.255       255.255.255.255  On-link            127.0.0.1         331
192.168.1.0           255.255.255.0    On-link            192.168.1.3       306
192.168.1.3           255.255.255.255  On-link            192.168.1.3       306
192.168.1.255         255.255.255.255  On-link            192.168.1.3       306
224.0.0.0             240.0.0.0        On-link            127.0.0.1         331
224.0.0.0             240.0.0.0        On-link            192.168.1.3       306
255.255.255.255       255.255.255.255  On-link            127.0.0.1         331
255.255.255.255       255.255.255.255  On-link            192.168.1.3       306
=====
Itinéraires persistants :
Aucun

IPv6 Table de routage
=====
Itinéraires actifs :
If Metric Network Destination Gateway
 1 331 ::1/128 On-link
 6 306 fe80::/64 On-link
 6 306 fe80::11ac:a1f:296d:c2c4/128 On-link
 1 331 ff00::/8 On-link
 6 306 ff00::/8 On-link
=====
Itinéraires persistants :
Aucun

C:\Users\ilyas>
```

Figure VII.2. Résultat de rout print

2. Protocoles de routage

Catégories des protocoles de routage

Le protocole de routage est un processus qui crée et met à jour la table de routage. La table de routage peut aussi être créé manuellement en ajoutant des entrées à la table de routage avec la commande `route add`.

Dans les protocoles de routage, dit aussi routage dynamique, les routeurs exécutent un algorithme de routage pour découvrir les routes existantes et diffusent celles-ci aux routeurs adjacents.

Deux types de protocole de routage principaux existent:

- *Distance Vector (Vecteur de Distances)*: dans ce type de protocole, les routeurs échangent une liste de réseaux accessibles avec les distances (nombre de sauts) correspondants pour chaque réseau destination. Par la suite, le routeur choisit la route avec le nombre de sauts le plus petit. Ce type de protocole est généralement basé sur l'algorithme de Belman-Ford.

- *Link State (Etat des liens)*: Au lieu de choisir le chemin avec la distance la plus courte, les protocoles Link State basent le choix de la route sur la qualité du lien. La qualité du lien peut être calculée à partir d'un ou plusieurs paramètres (Bande passante du lien, MTU, Round Trip, Latence, reliability). Ce type de protocole est généralement basé sur l'algorithme Dijkstra.

Les protocoles de routage classifiés comme Classfull ne transmettent pas les masques réseaux dans les messages de routage (par exemple: RIP) et se basent sur les classes réseaux pour déduire le masque réseau. Contrairement à cette classe de protocole, les protocoles de routage Classless transmettent les masques réseaux correspondants à chaque réseau destination et ainsi, supportent le subnetting avec masques de tailles variables (comme OSPF, RIPv2);

Distance Vector (Vecteur de Distances)

Dans ce type de protocole, des messages de routage sont transmis périodiquement contenant la table de routage complète (chaque 30 secondes). Parallèlement, chaque routeur reçoit les tables de routage de ces routeurs voisins et applique un algorithme pour déterminer les plus courts chemins pour atteindre chaque réseau (par exemple Bellman-Ford). Ainsi, le prochain message de routage contiendra les changements détectés

Si une route n'est pas reçue pour trois intervalles consécutives, celle-ci est supprimée de la table de routage et ne sera pas diffusée dans le prochain message de routage. Pour que un routeur annonce que un réseau n'est plus accessible, celui-ci arrête de diffuser la route correspondante dans ses messages de routage.

Un désavantage de ce type de protocole est la lenteur de la convergence (les informations de routage ne sont pas diffusées rapidement à tous les routeurs).

Par exemple, : RIP, RIPv2, IGRP sont des protocoles de routage Vecteur de Distances ;

Exemple : Convergence des protocoles Vecteur de distances

En cas de panne dans le routeur A, B attendra 3 intervalles (3 x 90sec) avant de supprimer les routes dans les quelles A est le prochain saut;

Par la suite C sera notifié après 120 sec et affichera ce changement dans le message de routage prochain (120 sec +30). La même chose pour D;

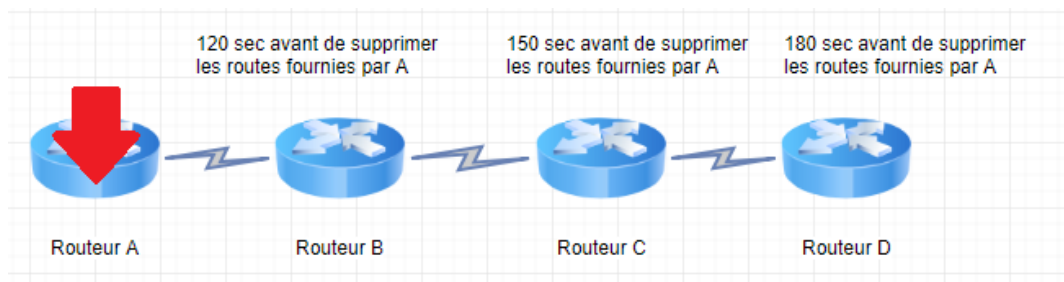


Figure VII.3. Exemple de routage Vecteur de distance

Link State (État des liens)

Par exemple, : OSPF, IS-IS sont classifiés comme des protocoles de routage Etat des liens ;

Les routeurs transmettent des messages HELLO dans des intervalles de 10sec à tous les routeurs voisins. Selon les réponses à son HELLO, un routeur diffuse une mise à jour dans tout le réseau pour informer les autres routeurs des états des liens avec ses réseaux voisins;

À la réception des messages de mise à jour, un routeur crée une carte topologique complète du réseau. Afin de créer un arbre (Spanning Tree), connectant le réseau entier, les routeurs appliquent généralement l'algorithme de Dijkstra pour trouver les meilleurs chemins vers toutes les destinations atteignables;

Exemple : Déroulement du protocole État des liens

L'exemple suivant montre comment un protocole d'État des liens se déroule.

- Un routeur A envoie périodiquement un message HELLO ;

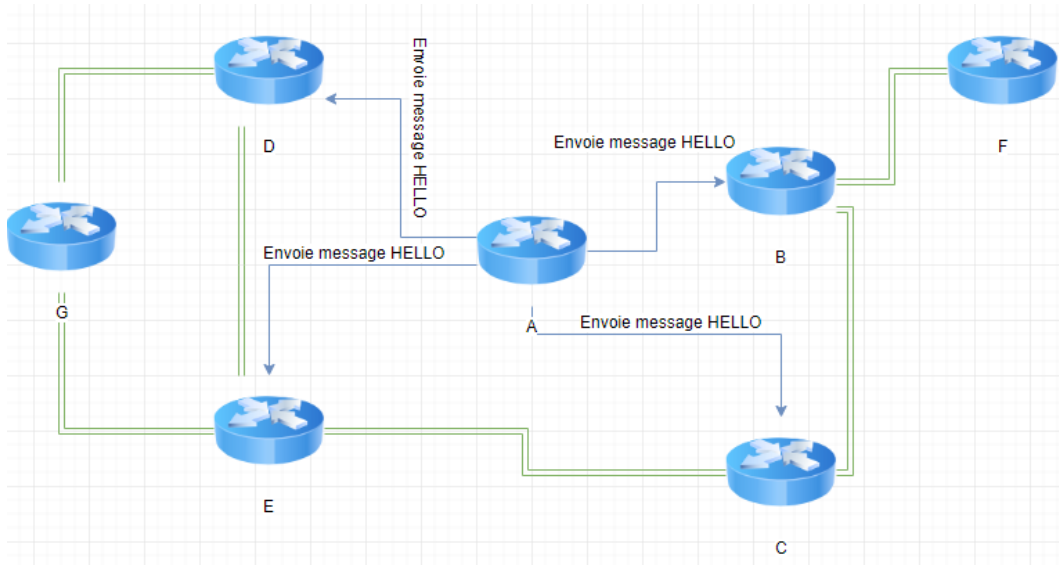


Figure VII.4. Exemple de routage État des liens (a)

- Les routeurs adjacents répondent avec des messages qui permettent au routeur A de mesurer les états /qualités des liens avec les réseaux voisins;

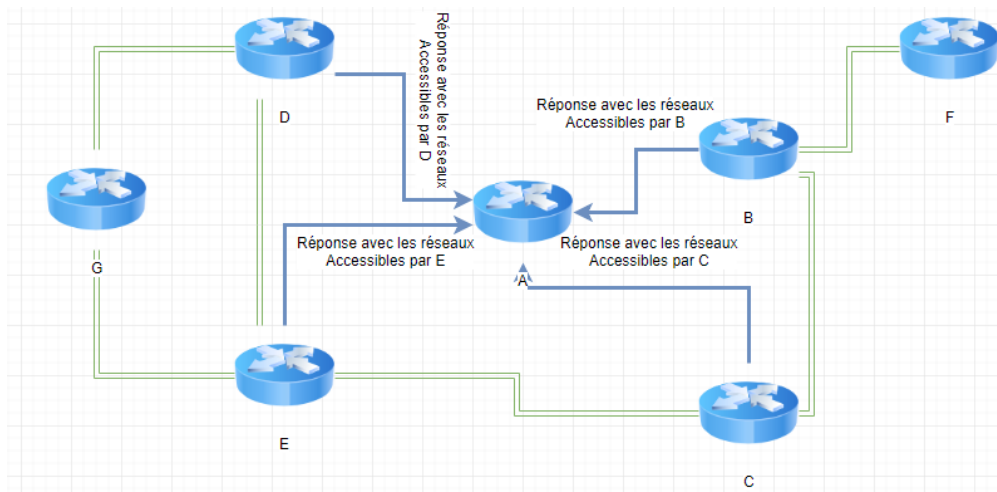


Figure VII.5. Exemple de routage État des liens (b)

- Le routeur A diffuse un message contenant les états des liens avec les réseaux voisins *dans tout le réseau* afin que les autres routeurs puissent créer un graphe du réseau et appliquer l'algorithme Dijkstra ;

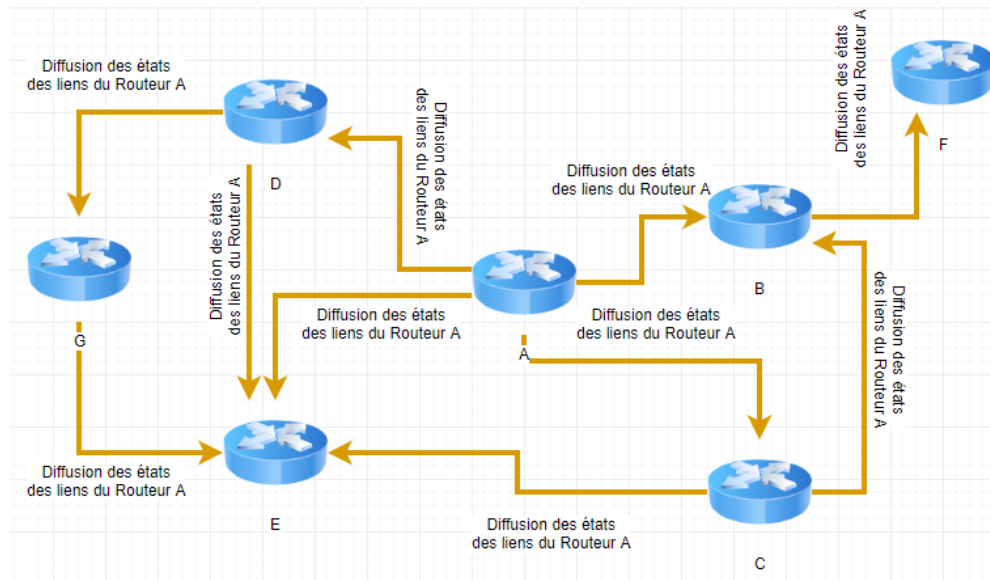


Figure VII.6. Exemple de routage État des liens (c)

Interior Routing Protocol vs Exterior Routing Protocol

Les protocoles mentionnés précédemment sont des protocoles IGP (RIP, RIPv2, OSPF, etc) permettant d'assurer la création des tables de routage dans des zones limitées. Les fournisseurs d'accès réseau utilisent ces protocoles à l'intérieur de leurs réseaux pour mettre à jours les tables de routage des routeurs. D'autre part, les tables de routage sont échangées entre les réseaux de différents fournisseurs avec un protocole BGP (Border Gateway Protocol) qui est classifié comme EGP (Exterior Gateway Protocol).

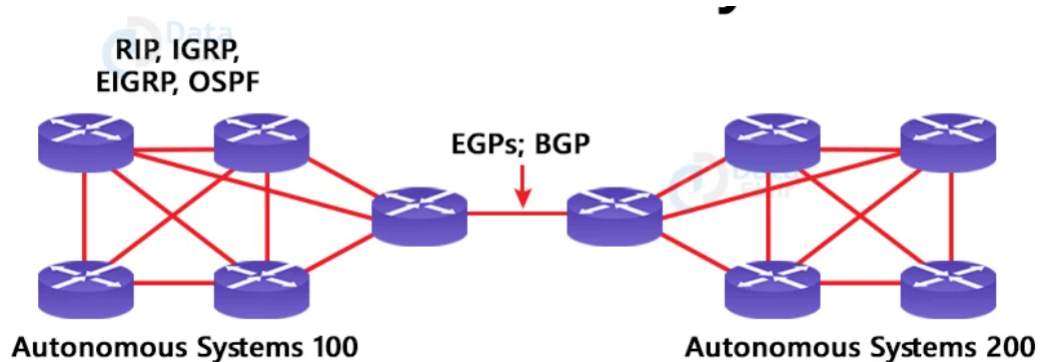


Figure VII.7. Border Gateway Protocol^{8*}

Chapitre IIX Protocoles Applicatif TELNET

VIII

1. TELNET (TELEtype NETwork)

- Lors des débuts d'internet, l'accès à distance (*remote login*) été l'une des premières applications souhaitées pour permettre aux utilisateurs de contrôler des machines à distance. Par exemple, les serveurs peuvent exécuter plusieurs programmes de divers utilisateurs simultanément. Mais, un seul utilisateur peut être présent physiquement devant le serveur;
- L'idée de base est simple : concevoir un protocole permettant à l'utilisateur de connecter à une machine distante et de contrôler celle-ci. Comme si l'utilisateur été physiquement assis devant la machine pour taper ses commandes ;
- Le problème rencontré à cette époque été l'hétérogénéité entre l'encodage des fabricants de logiciels / matériel (surtout pour les systèmes d'exploitation). Par exemple, une interruption de processus sur Windows est représentée par la séquence `CTRL+ALT+SUPPR`, alors que interruption de processus sur Linux est représentée par `CTRL+C`;
- TELNET est un simple protocole applicatif qui a été développé pour résoudre ce problème ;

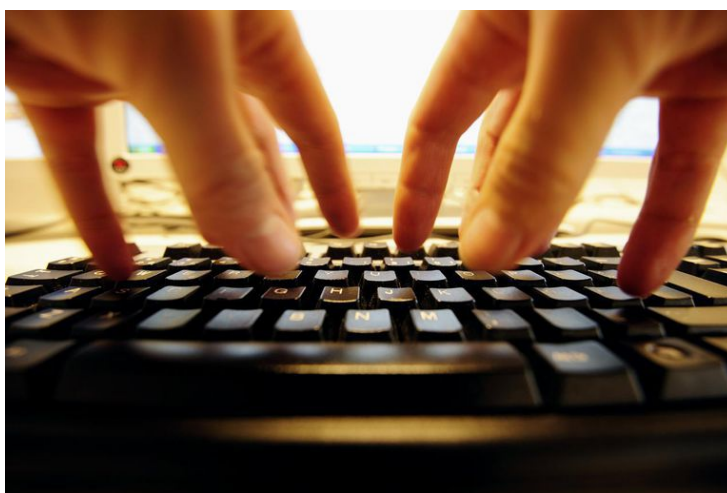


Figure IIX.1. Telnet

2. Fonctionnement

TELNET fonctionne avec l'architecture client /serveur et utilise TCP pour assurer la livraison des données entre les deux parties (commandes transmis du client vers le serveur, résultats d'exécution transmis du serveur vers le client). Généralement, ce protocole utilise le port TCP 23 ;

Le protocole TELNET repose sur deux éléments :

- *NVT (Network Virtual Terminal)* et commandes de contrôle unifiés;
- Négociation des options ;

3. NVT (Network Virtual Terminal)

- Vu l'hétérogénéité des systèmes d'exploitations, TELNET utilise un clavier virtuel standard pour encoder les échanges entre le client et le serveur. Cette norme est appelée NVT ;
- Lorsque le client saisie ça commande, cette commande est convertie vers l'équivalent NVT avant d'être transmis vers le serveur. Ensuite, le serveur converti les données reçus sous format NVT vers son format spécifique pour les traiter;
- Ceci évite que tous les clients et serveurs TELNET ne soit conscients des encodages possibles de d'autres systèmes d'exploitation(ce qui complique énormément l'implémentation de ce protocole);

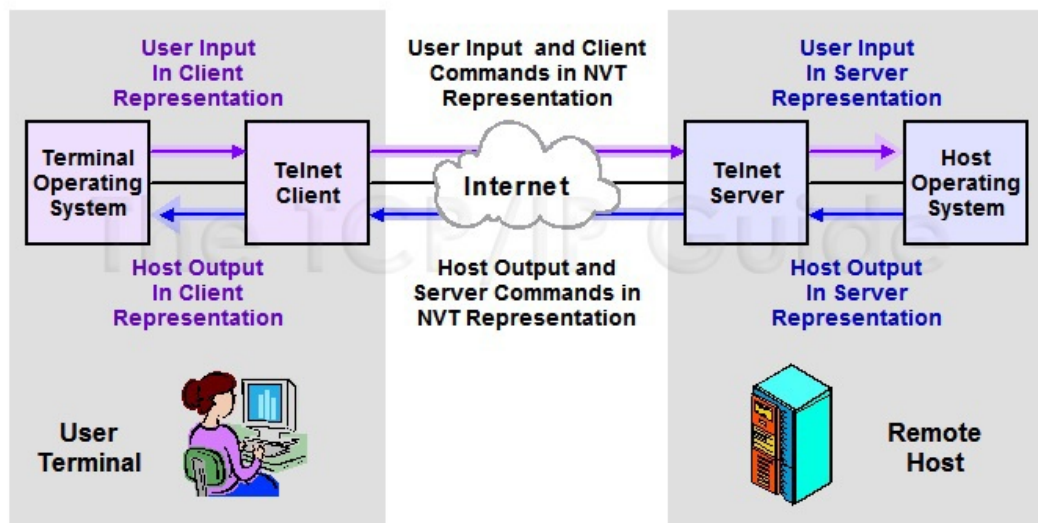


Figure IIX.2. NVT^{3*}

NVT, qui doit être utilisé par toute application TELNET est basé sur le code ASCII :

- 95 caractères affichables [du 32 jusqu'à 126] sur écran (dits "*printables*") ;
- NVT spécifie comment trois caractères de contrôle qui doivent être interprétés de manière unifiée par n'importe quel Serveur / Client TELNET [*NULL*, *LF*, *CR*];
- Cinq autres caractères sont optionnels, et si implémentés, ils doivent être conformes à la spécification de NVT ;

Pour se déplacer au début de la ligne suivante, la séquence *CR +LF* est utilisée ;

ASCII Value (Decimal)	ASCII Character Code	ASCII Character	Description	Support Optional / Mandatory
0	NUL	Null	No operation (no effect on output).	Mandatory
7	BEL	Bell	Produces an audible or visible signal on the output without moving the print head. This notification may be used to get the user's attention, as in the case of an error.	Optional
8	BS	Back Space	Moves the print position one character to the left.	Optional
9	HT	Horizontal Tab	Moves the printer to the next horizontal tab stop. The standard does not specify how devices agree on tab stop positions, this can be negotiated using Telnet options.	Optional
10	LF	Line Feed	Moves the printer to the next line, keeping the print position the same.	Mandatory
11	VT	Vertical Tab	Moves the print line to the next vertical tab stop. As with the "HT" character, devices must use an option to come to an agreement on vertical tab stop positions.	Optional
12	FF	Form Feed	Moves the printer to the top of the next page (or on a display, clears the screen and positions the cursor at the top.)	Optional
13	CR	Carriage Return	Moves the printer to the left margin of the current print line.	Mandatory

Figure IIX.3. Caractères normalisés^{3*}

- Pour contrôler le déroulement de la session TELNET, NVT définit des caractères virtuels qui représentent des commandes ;
- Pour qu'un octet soit interprété comme une commande, celui-ci doit être précédé par un octet IAC (*Interpret As Command* == *Code FF ou 255*) ;
- Pour transmettre un octet avec la valeur FF sans qu'il soit interprété comme IAC, deux octets de valeur FI doivent être transmis ;

Command Byte Value (Decimal)	Command Code	Command	Description
240	SE	Subnegotiation End	Marks the end of a Telnet option subnegotiation, used with the SB code to specify more specific option parameters. See the topic on Telnet options for details.
241	NOP	No Operation	Null command; does nothing.
242	DM	Data Mark	Used to mark the end of a sequence of data that the recipient should scan for urgent Telnet commands. See the topic on Telnet interrupt handling for more information.
243	BRK	Break	Represents the pressing of the "break" or "attention" key on the terminal.
244	IP	Interrupt Process	Tells the recipient to interrupt, abort, suspend or terminate the process currently in use.
245	AO	Abort Output	Instructs the remote host to continue running the current process, but discard all remaining output from it. This may be needed if a program starts to send unexpectedly large amounts of data to the user.
246	AYT	Are You There	May be used to check that the remote host is still "alive". When this character is sent the remote host returns some type of output to indicate that it is still functioning.
247	EC	Erase Character	Instructs the recipient to delete the last undeleted character from the data stream. Used to "undo" the sending of a character.
248	EL	Erase Line	Tells the recipient to delete all characters from the data stream back to (but not including) the last end of line ("CR"+"LF") sequence.
249	GA	Go Ahead	Used in Telnet half-duplex mode to signal the other device that it may transmit.
250	SB	Subnegotiation	Marks the beginning of a Telnet option subnegotiation, used when an option requires the client and server to exchange parameters. See the topic on Telnet options for a full description.
251	WILL	Will Perform	In Telnet option negotiation , indicates that the device sending this code is willing to perform or continue performing a particular option.
252	WON'T	Won't Perform	In Telnet option negotiation , indicates that the device sending this code is either not willing to perform a particular option, or is now refusing to continue to perform it.
253	DO	Do Perform	In Telnet option negotiation , requests that the other device perform a particular option or confirms the expectation that the other device will perform that option.
254	DON'T	Don't Perform	In Telnet option negotiation , specifies that the other party not perform an option, or confirms a device's expectation that the other party not perform an option.
255	IAC	Interpret As Command	Precedes command values 240 through 254 as described above. A pair of IAC bytes in a row represents the data value 255.

Figure IIX.4. Commandes Telnet^{3*}

4. Négociation d'options

- Le client et serveur peuvent négocier des options supplémentaires au NVT standard ;
- Pour qu'une option soit utilisée, les deux parties doivent être d'accord sur celle-ci ;

- Par exemple : TELNET est un protocole half-duplex où le client et le serveur doivent attendre un signal *Go Ahead* pour transmettre vers l'autre. Il existe une option définie appelée *SUPPRESS-GO-AHEAD* (code == 3) pour que la communication soit simultanée. Alors si le client souhaite utiliser cette option il envoie la séquence suivante :

IAC WILL SUPPRESS-GO-AHEAD (suite de codes == 255 251 3) ;

Si le serveur accepte cette option, celui-ci répondra avec : *IAC DO SUPPRESS-GO-AHEAD* (suite de codes == 255 253 3). Sinon, si l'option est refusée : *IAC DONT SUPPRESS-GO-AHEAD* (suite de codes == 255 254 3) ;

Option Number	Option Code	Option Name	Description	Defining RFC
0	<i>TRANSMIT-BINARY</i>	Binary Transmission	Allows devices to send data in 8-bit binary form instead of 7-bit ASCII.	856
1	<i>ECHO</i>	Echo	When you press a key on a terminal, you also expect to see the character you entered appear on the terminal screen as output; this is called <i>echoing</i> the input. Echoing is a significant issue in terminal applications because it can be implemented in a number of different ways. This option allows devices to negotiate any of a variety of different echo modes.	857
3	<i>SUPPRESS-GO-AHEAD</i>	Suppress Go Ahead	Allows devices not operating in half-duplex mode to no longer need to end transmissions using the Telnet <i>Go Ahead</i> command.	858
24	<i>TERMINAL-TYPE</i>	Terminal Type	Allows the client and server to negotiate the use of a specific terminal type. If they agree, this allows the output from the server to be ideally customized to the needs of the particular terminal the user is working on.	1091
34	<i>LINEMODE</i>	Linemode	Allows the client to send data one line at a time instead of one character at a time. This improves performance by replacing a large number of tiny TCP transmissions with a smaller number of larger ones.	1184

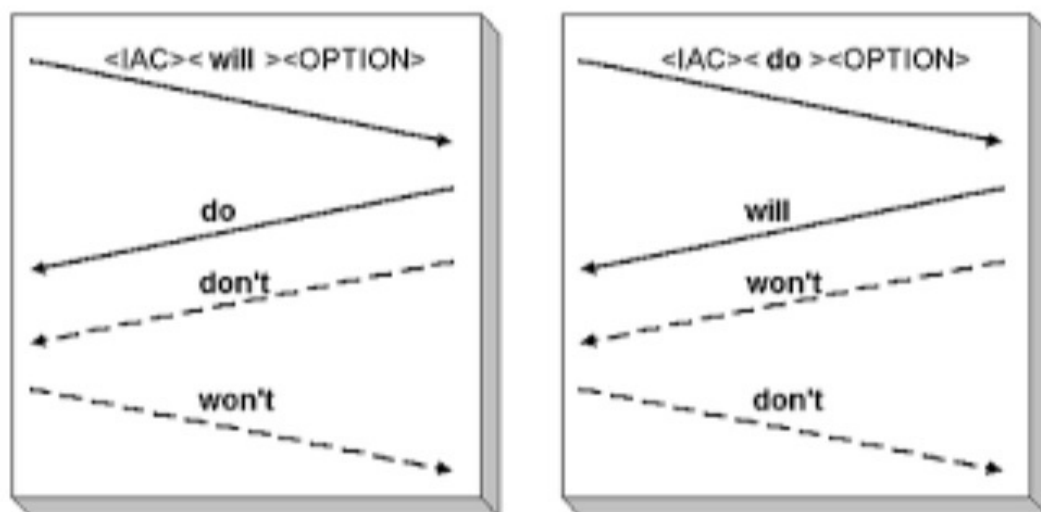


Figure IIX.5. Exemple d'échanges Telnet^{3*}

Chapitre IX Protocoles Applicatif FTP

IX

1. FTP (File Transfer Protocol)



FTP est une solution de partage et transfert de fichiers (par exemple : archivage, stockage distribué, etc). Comme TELNET, le développement de ce protocole a fait face aux différents représentations de fichiers et des hétérogénéités des systèmes d'exploitation.

FTP est bâti sur les fonctionnalités suivantes :

- *Authentification des utilisateurs* : Les utilisateurs doivent être authentifiés et selon leurs identités, les opérations permises pour ces derniers seront définies (par exemple : certains utilisateurs auront les droits d'écriture alors que d'autres seront limités à la lecture);
- *Mode interactif / automatique* : FTP été conçu pour permettre aux utilisateurs un accès interactif via l'ensemble de commandes FTP, ainsi qu'un accès automatique aux applications.
- *Spécification du format du transfert* ;

2. Fonctionnement FTP

- FTP utilise TCP (*port 21 et 20*) au niveau de la couche transport et repose sur une architecture client / serveur ;
- Comme d'autres protocoles applicatifs (HTTP par exemple), FTP crée un processus esclave pour traiter une nouvelle connexion ;
- Contrairement à la plus part des protocoles applicatifs, ce protocole utilise deux connexions TCP :

1. *Une connexion permettant la transmission des commandes de contrôles* : La première connexion TCP (plus précisément une connexion TELNET) établie entre le client et le serveur est appelée *connexion de contrôle*. Cette connexion est faite d'un port quelconque (dynamiquement alloué comme d'habitude) de la part du client vers le port 21 (qui est le port réservé pour FTP). Cette connexion primaire est maintenue jusqu'à la fin de la session FTP et permet de transmettre des commandes FTP depuis le client vers le serveur. De l'autre côté, le serveur interprète les commandes FTP et transmet un code de réponse vers le client ;
 2. *Une connexion de transmission de données* : La deuxième connexion permet de transmettre les données (par exemple lors de la demande d'obtention d'un fichier par client). Cette connexion par contre est temporaire et sera clôturée après la fin de la transmission des données. Typiquement, le port utilisé pour la transmission des données du côté serveur est le port 20 qui est réservé pour FTP ;
- Deux modes de transmission de données sont possibles :
 1. *Mode actif* : Dans ce mode, le client transmet le numéro de port, par lequel il souhaite recevoir les données / fichier, vers le serveur par la connexion de contrôle. Par la suite, le serveur établie lui même la connexion TCP avec le client via ce port (ce qui sort un peu de la tradition des protocoles applicatifs). Par la suite, la transmission du fichier commence immédiatement ;
 2. *Mode Passif* : Dans le mode passif le client envoie une commande PASV sur la connexion de contrôle pour demander au serveur le numéro de port qui doit être utilisé pour la transmission de données. Par la suite, le client connecte au port spécifié par le serveur et ce dernier commence directement la transmission du fichier demandé;
 - L'utilisateur peut spécifier que le transfert du fichier doit se faire en mode binaire (*copie identique octet par octet comme les images et fichiers .exe*) ou mode ASCII pour que le système d'exploitation du transmetteur converti la représentation vers une représentation normalisée avant de transmettre le contenu.

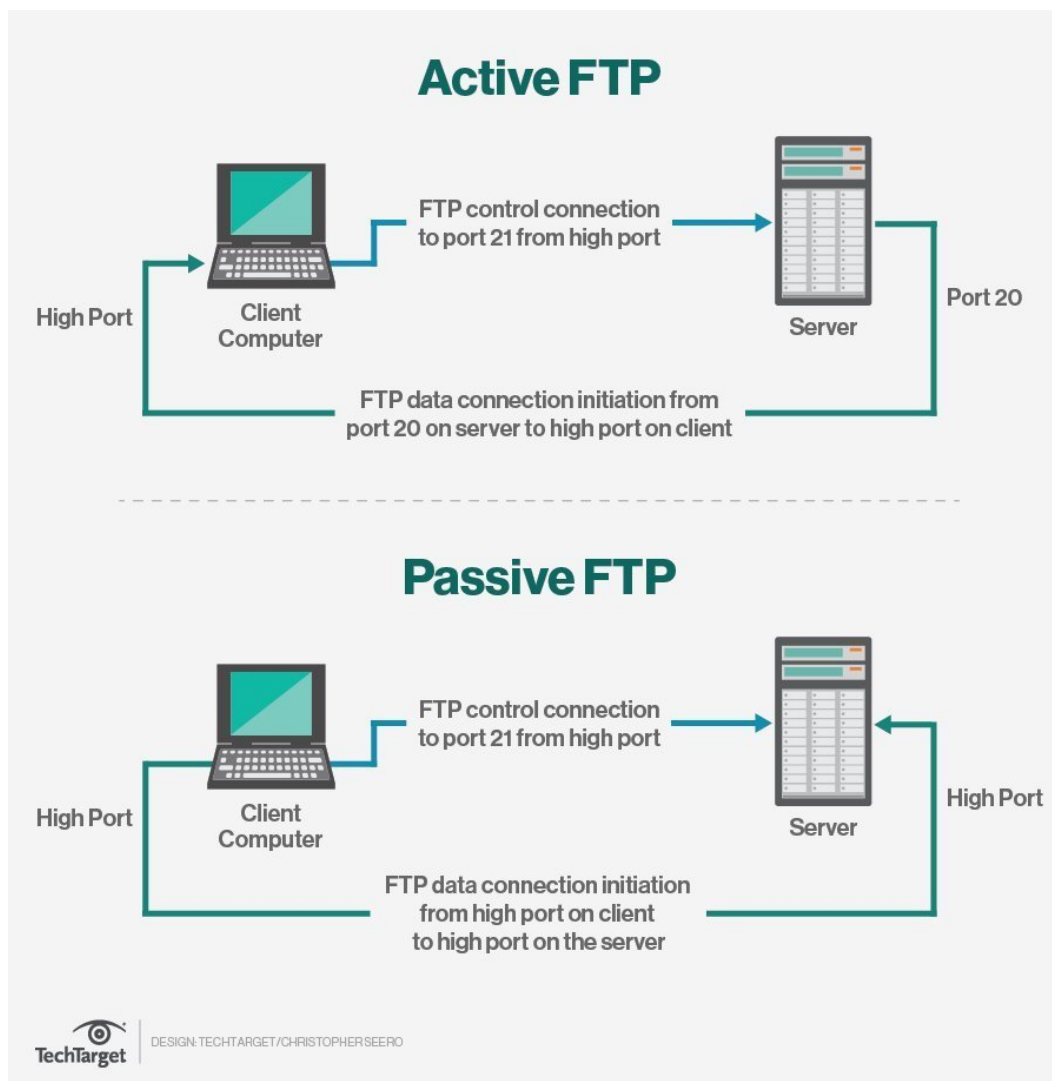


Figure IX.1. Modes FTP^{9*}

⚠ Attention : Mode ASCII et mode binaire

Attention, l'utilisation du mode ASCII pour transmettre un fichier binaire (image par exemple) donne un fichier inutilisable du côté du récepteur. Car la conversion de la représentation interne vers la représentation standard change le contenu du fichier.

📦 Complément : Mode actif et TELNET

- Le mode actif nécessite que le serveur établie la connexion vers le client (c.à.d le serveur envoie un segment TCP avec SYN allumé vers le client). Les connexions entrantes sont souvent bloquées par le pare-feu car ils sont considérés comme une tentative d'attaque;
- FTP utilise TELNET pour standardiser la représentation des commandes (NVT) mais n'utilise pas la négociation d'options TELNET. Ce-ci permet de transmettre les commandes dans un format que le serveur peut interpréter ;

3. Liste des commandes FTP et réponses serveurs

Les commandes FTP sont regroupées dans *trois catégories* :

- Commandes d'accès (*USER, PASS, CWD, QUIT, CDUP, REIN*);
- Commandes de spécification des paramètres (*PORT, PASV, TYPE, MODE*);
- Commandes du service FTP (*RETR, STOR, ALLO, REST, DELE, MKD, ABOR, NOOP, STAT*);

De l'autre part, selon le déroulement de la commande, *le serveur répond avec un code de trois chiffres et un message explicatif*. En outre, chaque chiffre dans la réponse du serveur a une signification :

- Le premier chiffre indique le succès ou l'échec (permanent ou temporaire) de la commande. Ce premier chiffre peut prendre de 1 à 5 ;
- Le deuxième chiffre divise les catégories de réponses définies par le premier chiffre en cinq sous-catégories pour donner plus de précision sur la réponse. La valeur de ce chiffre indique la réponse concerne quoi (exemple *si le deuxième chiffre est égale à 3, ceci indique que la réponse est relative à l'authentification*, si la valeur est égale à 2, *la réponse est relative à la syntaxe de la commande*) ;
- Le dernier chiffre donne dix codes pour chaque sous-catégorie ;

4. TFTP (Trivail FTP)

- TFTP est un protocole de transmission de fichiers simplifié qui ne prend compte ni de l'authentification, ni de l'accès interactif ;
- Le client initie la transmission de fichier par un simple message contenant le nom du fichier et le type d'opération (*OpCode == 1* pour dire que le client souhaite recevoir le fichier, *OpCode == 2* pour indiquer que le client souhaite transmettre un fichier vers le serveur) ;
- La valeur du champ *Mode* indique le mode de transmission (ASCII ou binaire) ;
- TFTP transmet les morceaux du fichier en taille de 512 octets. Ainsi, la réception d'un morceau de données de taille inférieur à 512 signale automatiquement la fin du fichier. En outre, les messages contenant des parties du fichier transféré ont *OpCode == 3* ;
- Contrairement à FTP, TFTP utilise UDP et assure l'acquittement des messages au niveau de la couche application. *OpCode == 4* signale un message ACK pour le numéro du bloque indiqué dans le message. En utilisant UDP, TFTP peut diffuser le contenu dans le réseau locale ;
- *OpCode == 5* signale une erreur de transmission ;

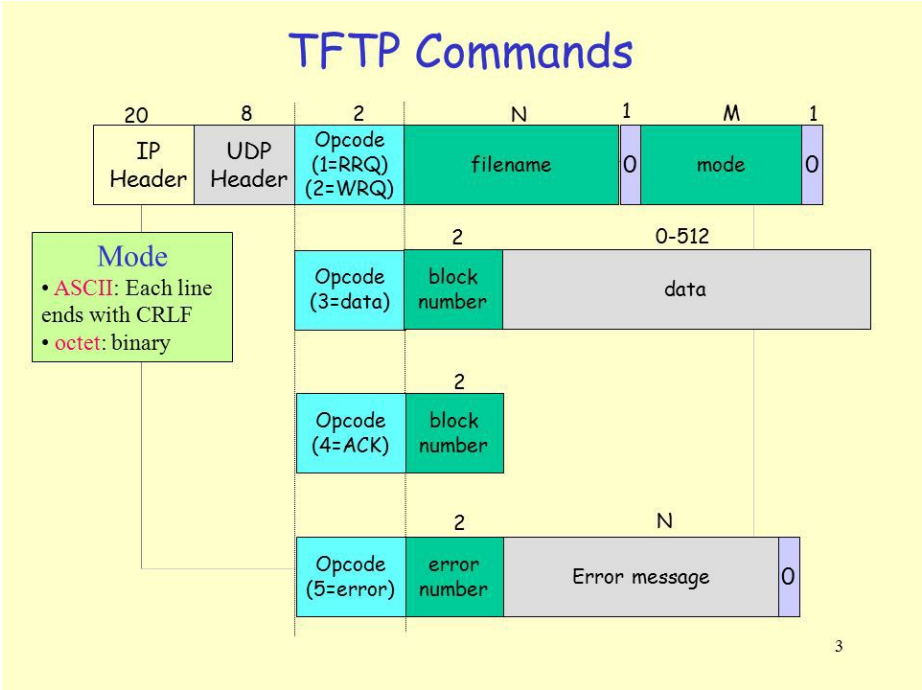


Figure IX.2. Protocole TFTP

Chapitre X Protocoles Applicatif DNS

X

1. Domain Name System

- Les machines utilisent les adresses IP pour s'adresser à d'autres machines et accéder à leurs services ;
- Pour un humain, les adresses machines sont souvent difficiles à mémoriser ;
- Si le site change d'adresse IP, tous les utilisateurs doivent être notifiés de ça nouvelle adresse ;
- Ainsi, un mécanisme pour nommer des machines accessibles sur Internet avec des noms faciles à mémoriser et convertir les noms vers des adresses IP a été mis en place ;
- Pour cet objectif, *DNS (Domain Name System)* a été conçu ;
- DNS permet de retrouver l'adresse IP d'une machine à partir d'un nom de domaine. En outre, DNS fonctionne dans le sens inverse (récupérer le nom de la machine à partir de l'adresse IP) ;
- A l'époque d'ARPnet, les noms des ordinateurs avec leurs adresses IP correspondantes été enregistrées dans un fichier "*hosts.txt*". DNS utilise le même principe sauf que ce-ci ne s'applique pas à un réseau de la taille d'Internet ;
- DNS est conçu avec une architecture distribuée et hiérarchique :
- Chaque client utilise un programme appelé "*Name Resolver*" qui permet de transmettre les requêtes DNS vers le Serveur DNS local selon ça configuration. Le client doit être configuré avec au moins l'adresse d'un Serveur DNS.
- Le serveur DNS est une machine capable à résoudre les requêtes DNS et enregistre le résultat des requête dans son cache.

2. Nom de domaines

- Un nom de domaine est structuré d'une forme hiérarchique similaire aux chemins dans système de fichiers.
- Pour un chemin: C:\Preparation\Reseaux\CoursTelnet : Celui-ci désigne le répertoire *CoursTelnet* contenu dans le répertoire *Reseaux* qui est lui même contenu dans le répertoire *Preparation*. Et puis, le répertoire *Preparation* est contenu dans le Disque C.
- Le même principe s'applique sur un nom de domaine :
- Supposant l'existence d'un domaine : *CoursTelnet.Reseaux.Preparation.com*. Ce nom de domaine désigne le domaine *CoursTelnet* contenu dans le domaine *Reseaux* qui est lui même contenu dans un domaine *Preparation*. De même, le domaine *Preparation* est contenu dans le domaine de niveau haut *com*.
- Comme dans un système de fichiers, deux sous domaines du même domaine ne peuvent pas avoir le même nom.
- Les noms des domaines sont insensibles à la case (*Reseaux.edu* et le même que *RESEAUX.edu*) et doivent contenir que des caractères alphabétiques, des chiffres et "-" seulement.
- Le nom de domaine ne doit pas dépasser 253 caractères.

- Le domaine le plus à droite est appelé Top Level Domain (TLD). Les domaines TLD sont gérés par *ICAN* (*Internet Corporation for Assigned Names and Numbers*) et *IANA* (*Internet Assigned Numbers Authority*).
- Il existe actuellement plus de 200 TLD. Parmi ceux, des domaines génériques (.com , .org, .gov, .edu, .mil, etc) et des domaines nationaux (.fr, jp, .dz, etc).
- Pour obtenir un domaine du second niveau, il suffit de contacter le registraire du TLD souhaité pour vérifier si le nom est disponible. Une fois le nom est disponible, le client paie pour acheter le nom de domaine.

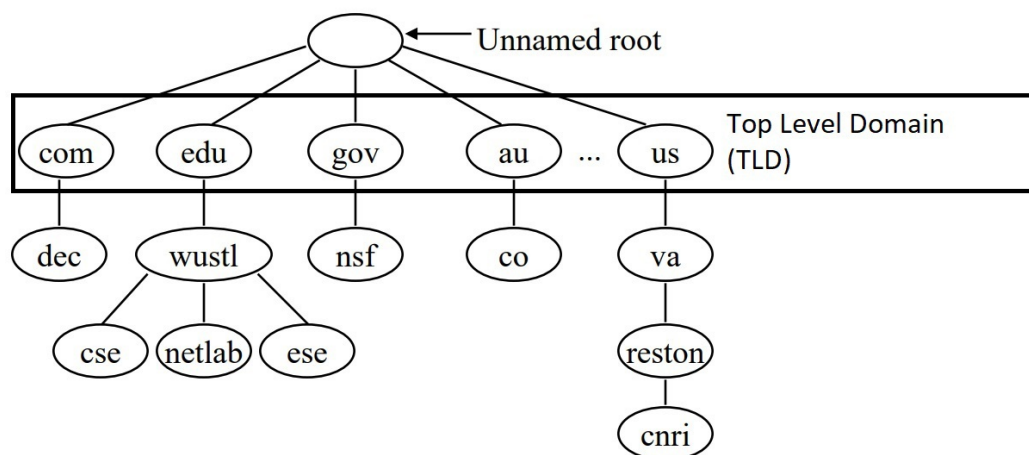


Figure X.1. Structures des Noms de domaines

3. Enregistrements DSN (resource records)

- Pour fonctionner, un serveur DNS contient une base de donnée sous forme de texte clair. Chaque entrée dans cette BDD est de la forme :
(Nom de domaine, Durée de vie, Classe, Type, Valeur)
- Le champ *Nom de domaine* prend un nom d'un domaine ou sous domaine alors que *Durée de vie* reflète la stabilité de l'information (valeur maximale == 86400 pour indiquer une journée de durée de vie). La valeur de Classe prend généralement IN (Internet).
Type peut prendre : A (Adresse IPv4), AAAA (Adresse IPv6), MX (Mail eXchanger), CNAME, etc et Valeur pointe vers une adresse IP ou un autre domaine.

Exemple

- Par exemple on souhaite enregistrer notre nom de domaine M1Info au TLD .dz situé à l'adresse publique 130.37.40.1. Pour accomplir ceci, le serveur DNS du domaine dz doit ajouter une entrée : `M1Info.dz. 86400 IN A 130.37.40.1`
- L'entrée de type A est celle la plus utilisée et indique simplement l'adresse IP associée à un nom de domaine. Dans cet exemple, l'adresse IP 130.37.40.1 est associée au domaine M1Info ;
- L'entrée de type MX indique le nom d'une machine capable de recevoir les mails dans le domaine courant. Souvent, il existe plusieurs enregistrements MX pour donner des alternatifs en cas de panne ;
- CNAME pointe un nom de domaine vers un autre pour définir un alias ;
`Master1.Masters.dz. 86400 IN CNAME M1Info.Masters.dz.`

4. Organisation hiérarchique

- DNS fonctionne d'une manière distribuée. Un serveur DNS doit au moins connaître les adresses IP des DNS racines. Il existe plusieurs serveurs DNS racines (par exemple : a.root-servers.net 198.41.0.4) ;
- Quand c'est nécessaire de résoudre un nom de domaine, le "Name resolver" envoie une requête DNS vers le serveur DNS local ;
- Par la suite le serveur local essaye de résoudre le nom (selon les enregistrements qu'il possède ou son cache). Si la résolution n'est pas possible, la requête est transmise vers le serveur racine ;
- Le serveur racine n'est pas forcément capable de résoudre toute la requête mais connaîtra au moins le TLD. Ainsi, il peut répondre partiellement à la requête pour rediriger le serveur DNS local vers le serveur DNS TLD ;
- A la réception d'une réponse (partielle ou complète), le serveur DNS local enregistre la réponse dans son cache pour réutiliser cette réponse en cas de besoin ;
- Si la réponse n'est pas complète, elle comportera au moins l'adresse du domaine suivant dans la hiérarchie (qui aura l'adresse du domaine inférieure si celui-ci existe). Ainsi, le DNS local utilisera l'adresse du serveur DNS suivant dans la hiérarchie pour formuler sa requête ;
- Ce processus est répété jusqu'à ce que la requête est complètement résolue et la réponse complète est transmise vers le "Name Resolver" ;
- La transmission de requêtes DNS utilise le port 53 UDP du côté serveur ;

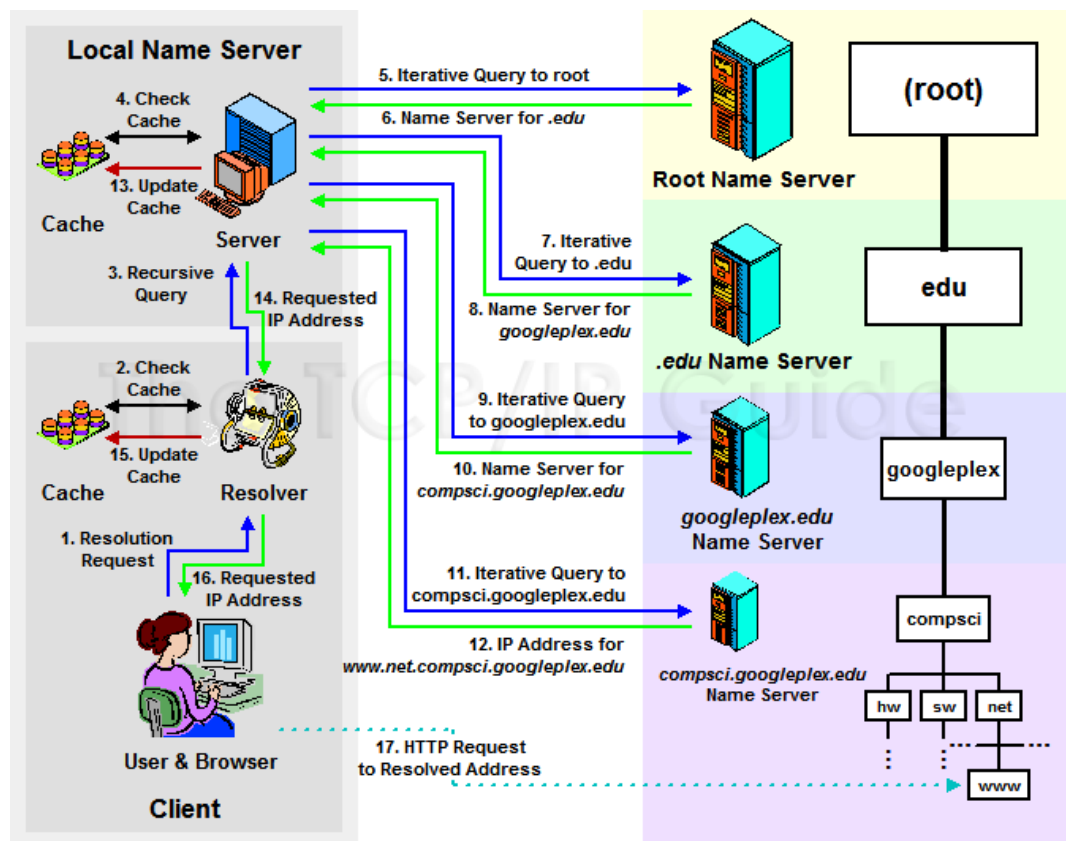


Figure X.2. Exemple de résolution récursive pour le nom `www.net.compsci.googleplex.edu`^{3*}

Chapitre XI Protocoles Applicatif HTTP

XI

1. Hypertext Transfer Protocol (HTTP)

- Actuellement le World Wide Web (WWW) est l'application la plus utilisée sur Internet (et la plus influente sur le monde) ;
- Le Web est une application tellement complexe et qui continue d'évoluer depuis sa conception ;
- Si *HTML* (*HyperText Markup Language*) est le langage de conception du contenu Web (documents hypertext), *HTTP* (*Hypertext Transfer Protocol*) est le protocole de la couche application permettant le transfert du contenu Web ;
- *Uniform Resource Locators* (*URLs*) est la méthode permettant d'identifier et référencer les ressources sur le Web d'une manière unique.

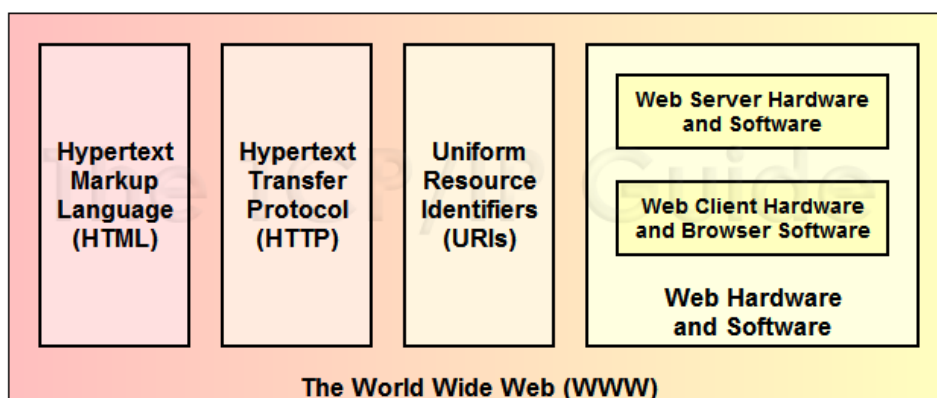


Figure XI.1 Composant du WWW^{3*}

2. Syntaxe URL HTTP

La syntaxe générale d'un URL HTTP *absolu* :

`http://<host>:<port>/<chemin-url>?<query>#<bookmark>`

- *<host>* : Nom de domaine ou adresse IP du serveur Web où la ressource est située ;
- *<port>* : Numéro de port TCP pour connecter au serveur Web. Si le port n'est pas spécifié, le port 80 sera utilisé ;
- *<chemin-url>* : Chemin qui pointe sur la ressource demandée ;
- *<query>* : Requête optionnelle ;
- *<bookmark>* : Permet d'accéder à une section spécifique de la page Web directement ;

La forme générale d'un URL est :

http://<host>/<chemin-url>

Par exemple :

https://www.eurosport.com/football/

Remarque

- Quand le numéro de port est spécifié, il est fortement possible que plusieurs serveurs Web existent sur la même adresse IP (le serveur Web utilise un numéro de port distinct pour chaque site hébergé) ;
- Les noms de domaines ne sont pas sensibles à la case. Par contre, les URI sont sensibles à la case.

3. Versions HTTP

- *HTTP / 0.9* : A été simplement conçu pour la transmission de documents hypertext seulement (aucun autre média supporté) ;
- *HTTP / 1.0* : Introduit le format des messages client et les formats des réponses serveur. Cette version a étendu le contenu transporté par le protocole vers d'autres types de médias (les images surtout) ;
- *HTTP / 1.1* : Les versions précédentes nécessitent l'initiation de session TCP pour chaque requête et la fermeture de la connexion après la livraison, ce qui diminue la performance. Ce problème a été résolu dans la version 1.1 par la transmission de plusieurs requêtes sur la même session. Cette version a aussi introduit les requêtes partielles (*entête Range :*) et d'autres fonctionnalités;
- *HTTP / 2.0 (2015)*: Compression des entêtes HTTP, et multiplexage des ressources transmis ;

Remarque

Si la version 0.9 ou 1.0 est spécifiée dans la requête HTTP, le serveur fermera automatiquement la connexion TCP après réponse.

4. Fonctionnement HTTP

- HTTP fonctionne d'une manière plus simple que FTP et SMTP.
- Une connexion TCP est établie depuis le client vers le port 80 du serveur initialement (*si un autre port que TCP 80 est utilisé par le serveur, celui la doit être spécifié dans l'URL*);
- Le client transmet ses requêtes HTTP sur la connexion ;
- Pour chaque requête reçue, le serveur répond par une ou plusieurs réponses HTTP (comme dans FTP il peut y avoir des réponses intermédiaires pour une requête);
- *Caching et proxies* : Comme dans le cours DNS, le caching des ressources récemment récupérées est employé pour répondre plus rapidement au requêtes de l'utilisateur ;

5. Requetes et Réponses HTTP

Syntaxe : Requête HTTP

Le format générale d'une requête HTTP est le suivant :

<request-line>

<general-headers>

<request-headers>

<entity-headers>

<empty-line>

[<message-body>]

[<message-trailers>]

- Seul <request-line> et le request-header *Host* sont obligatoires dans une requête HTTP;
- <request-line> : Indique l'action (dite Méthode) souhaitée par le client (généralement GET), l'URI de la ressource souhaitée, et la version de HTTP supportée par le client ;
- Généralement une requête ne contient pas la partie <message-body> (sauf si l'utilisateur souhaite transmettre quelque chose au serveur) ;



Figure XI.1. Structure requête HTTP^{3*}



Syntaxe : Réponse HTTP

Le format générale d'une réponse HTTP est le suivant :

<status-line>

<general-headers>

<response-headers>

<entity-headers>

<empty-line>

[<message-body>]

[<message-trailers>]

- <status-line> Indique la version utilisée par le serveur et un code qui représente le statut du traitement de la requête par le serveur (*les codes de réponses sont structurés d'une manière similaire à ceux de FTP*) et un message descriptif;
- <Entity-headers> : Contient des informations concernant le contenu transmis par le serveur (le nombre d'octets dans la page renvoyée par exemple);
- *Message Body* contient généralement la ressource demandée par le client (si la requête client ne peut pas être servie, Message Body contiendra une page d'erreur) ;

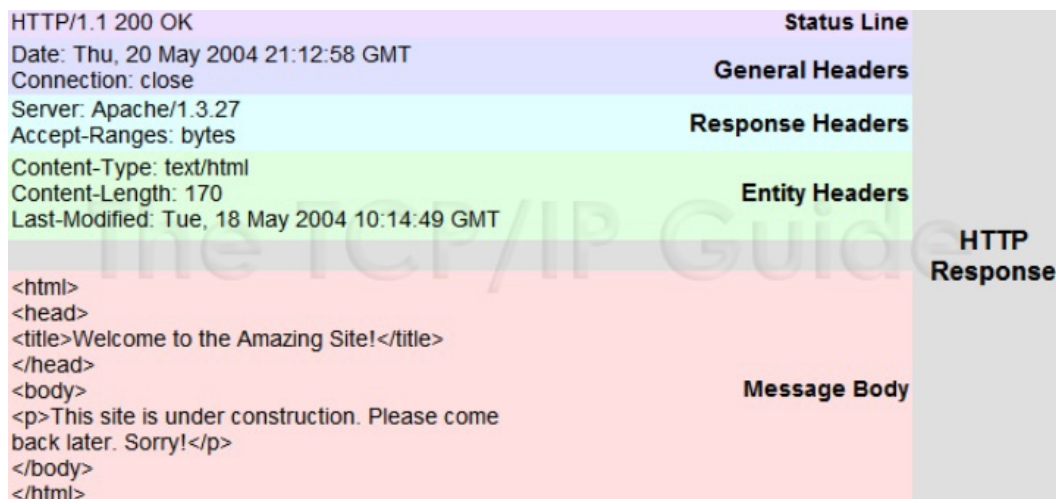


Figure XI.2. Structure réponse HTTP^{3*}

Fondamental

- Chaque entête dans un message HTTP se termine par *CRLF* (comme dans TELNET et FTP) ;
- Dans une réponse HTTP serveur, une séquence *CRLFCRLF* marque la fin des entêtes et le début du *Message Body* (la ressource que le client a demandé) ;
- Dans une requête client, *CRLFCRLF* indique la fin des entêtes HTTP;

6. Méthodes HTTP

- *GET* : Permet d'obtenir la ressource spécifiée par l'URI. Si la ressource désignée par l'URI est accessible, le serveur renvoie la ressource dans la partie Message Body de la réponse. Sinon (l'URI est incorrecte), le serveur renvoie un code d'erreur (404 par exemple) dans la réponse et un message descriptive. Si la requête GET contient un header *If-Modified-Since*, la ressource est retournée seulement si la ressource a été modifiée après la date indiquée par cet entête (Sinon, le serveur renvoie un code 304 sans la ressource demandée) ;
- *HEAD* : Similaire à GET mais demande l'obtention de la réponse serveur sans le contenu de la ressource (le serveur renvoie seulement la partie <entity headers> pour que le client apprend la description de la ressource demandée) ;
- *POST* : Permet la soumission des informations par le client à un programme dans le serveur afin que ces derniers soit traités ;
- *OPTIONS* : Permet d'obtenir les options de communication supportées par le serveur. Si le paramètre de cette méthode est un URI d'une ressource, la réponse sera relative à la ressource elle même. Sinon, le caractère * peut être utilisé pour demander des informations sur le serveur lui même ;
- *PUT* : Permet de transmettre le contenu d'une ressource depuis le client vers le serveur (l'inverse de GET) ;
- *DELETE* permet simplement de supprimer une ressource et *TRACE* permet de recevoir la requête transmis initiale générée par le client ;

7. Codes de réponses

Les codes des réponses serveurs sont similaires à ceux de FTP. Par contre, seul le premier chiffre est utilisé pour répartir les réponses sur 5 catégories et les deux derniers chiffres sont utilisés pour avoir 100 codes pour chaque catégorie:

- 1XX : Message d'information général ;
- 2XX : Requête exécuté avec succès;
- 3XX : La requête nécessite d'autres actions pour être accomplie ;
- 4XX : Requête du client est invalide ;
- 5XX : Le serveur n'a pas été capable d'exécuter la requête ;

Exemple : Exemples de code de réponses

- 100 : Le client peut continuer à transmettre ses requêtes ;
- 200 : La requête a été exécutée avec succès ;
- 206 : La réponse du serveur contient le contenu partiel de la ressource demandée;
- 307 : La ressource demandée est temporairement localisée dans un URL différent ;
- 404 : URL invalide ou la ressource a été supprimée ;
- 505 : Erreur indiquant que le serveur ne supporte pas la version HTTP que le client souhaite utiliser ;

8. Caching et proxies

- Un proxy n'est qu'une machine intermédiaire qui permet d'effectuer la connexion avec le serveur à la place des clients ;
- Essentiellement, le proxy transmet la requête client à la place du client et renvoie les résultats reçus du serveur vers le client. Au cours de ce processus, le proxy mémorise le résultat (Caching) *afin de répondre aux requêtes ultérieurs directement sans transmettre la requête vers le serveur si d'autres demandes pour la même ressource sont reçues par la suite*;
- Ainsi, un proxy permet d'accélérer les réponses HTTP et économiser la bande passante ;
- Un proxy sert aussi à cacher l'adresse du client car quand le proxy effectue le travail précédant, l'adresse IP et Port TCP dans les paquets transmis au serveur ne seront pas ceux du client, mais ceux du proxy;

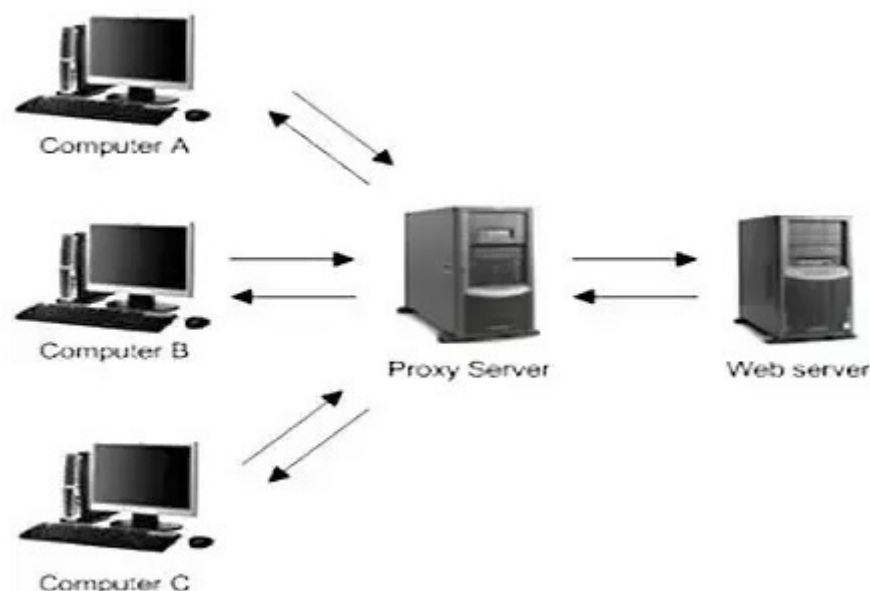


Figure XI.3. Fonctionnement proxy HTTP

9. Types de headers (entêtes)

Le seul entête obligatoire dans une requête c'est le l'entête *Host* qui a pour valeur le nom du serveur HTTP (ou l'adresse IP de celui-ci) désigné ;

Entêtes générales <general-headers> :

- *Connection* : Le client utilise cet entête pour indiquer au serveur que la connexion doit être fermée après l'envoi de la réponse ou bien elle doit être maintenue ; Exemple : "*Connection : close*" signifie que la connexion doit être fermée après l'envoi de la réponse par le serveur ; "*Connection : keep-alive*" signifie que la connexion doit être maintenue ouverte après l'envoi de la réponse par le serveur ;
- *Date* : Indique la date de la génération du message (requête ou réponse) Exemple : , "*Date : Wed, 16 Sep 2018 16:43:50 GMT*"
- *Cache-Control* : Indique si les réponses / requêtes doivent être enregistrées afin de répondre aux requêtes ultérieurs localement; Exemple : "*Cache-Control : no-store*" (les potentiels proxies sur la route ne doivent pas enregistrer le contenu du message). Ce-ci est utile surtout pour les données confidentielles;
- *Trailer* : Indique l'existence d'entêtes après les données transmises car typiquement les entêtes sont transmis avant le contenu. Ainsi, toute machine qui traite le message HTTP sera averti pour séparer les entêtes des données ;
- *Transfer-Encoding* : Méthode d'encodage de tout le message HTTP ;
- *Via* : Utilisé par les machines intermédiaires pour indiquer leur interventions dans la communication ;

Entêtes requêtes <request-headers>:

- *Accept* : Types de média acceptés par le client (text, images, etc);
- *Accept-Language* : Liste les tags des langages acceptés par le client (fr , en, etc);
- *Accept-Encoding* : Utilisé souvent pour indiquer au serveur s'il est possible de transmettre les données en mode compressé. Si présent, celui-ci contiendra les algorithmes de compression supportés (gzip, deflate, etc) ;

- *If-Modified-Since* : Instruit le serveur de transmettre la ressource seulement si la date de modification est supérieure à la date indiquée par cet entête. Cet entête est utilisé par GoogleBot pour éviter de télécharger et indexer des pages non modifiées ;
- *Range* : Permet au client de demander une partie de la ressource spécifiée par un intervalle d'octets (par exemple *Range : bytes=1000-3000* : l'ensemble des octets de la ressource de 1000 jusqu'à 3000).
- *If-Range* : Permet d'appliquer une condition à l'entête *Range*. Si la condition de cet entête est valide, la partie de la ressource indiquée par *Range* sera renvoyée par le serveur. Sinon (la condition *If-Range* n'est pas valide), le serveur renvoie la ressource complète. Exemple : *If-Range: Wed, 21 Oct 2015 07:28:00 GMT* (si la ressource n'a pas été modifiée ultérieurement à cette date, la partie spécifiée par le header *Range* doit être transmise par le Serveur) ; *If-Range* peut être utilisée avec un hash pour vérifier si la ressource a été modifiée ;

Entêtes du contenu <entity-headers> :

- *Content-Encoding* : Indique la méthode utilisée pour encoder la ressource contenue dans le message HTTP (compression généralement) ;
- *Content-Language* : Indique la langue de la ressource ;
- *Content-Length* : Indique la taille en octets de l'entité ;
- *Content-MD5* : Message digest permettant la vérification de l'intégrité de la ressource (hashing) ;
- *Content-Range* : Permet de transmettre une partie de l'entité et de la positionner par rapport à l'entité complète. Par exemple, si la requête GET demande une partie d'un fichier de l'octet 0 jusqu'à l'octet 3000 (avec le header *Range*) d'un fichier de taille de 6000 octets, la réponse du serveur comportera un entête *Content-Range : bytes 0-3000/6000* ;
- *Content-Type* : Indique le type de la ressource (texte, image ou autre) ;
- *Expires* : Spécifie la date après laquelle le contenu doit être considéré comme périmé. Ainsi, si une ressource est enregistrée dans le cache et l'entête *Expires* est inférieur à la date courante, le programme client / proxy (ou navigateur) demandera la ressource à nouveau du serveur (utilisé pour le contenu dynamique) ;
- *Last-Modified* : Date de la dernière modification. La valeur de cet entête peut être utilisée pour éviter de télécharger un fichier qui n'a pas été modifié depuis le dernier téléchargement ;

Remarque : L'entête Cookie et Set-cookie

- Le serveur HTTP peut demander au navigateur client d'ajouter un entête *Cookie* à chacune de ces requêtes http suivantes en transmettant un message http avec l'entête *Set-cookie :[valeur-cookie-client]*. Par la suite, chaque fois que le client communique avec ce serveur, le message http contiendra un entête *Cookie :[valeur-cookie-client]*.
- La valeur du cookie est générée au début d'une manière aléatoire. Par la suite, une fois la valeur cookie est associée à un client, celle-ci permet de suivre sa navigation dans le site et de l'authentifier sans avoir besoin de recevoir son login/mot de passe dans chaque requête.

10. HTTPS

- HTTPS fonctionne exactement comme HTTP sauf que au début, le client et le serveur appliquent une procédure d'échange de Certificats contenant la clé de chiffrement asymétrique ;
- Après la distribution de clés, la communication entre le client et serveur (dans les deux sens) sera cryptée. Par contre, les échanges entre ces deux reposent sur les mêmes entêtes/verbes HTTP ;
- HTTPS utilise le port TCP / 443 ;

Chapitre XII Protocoles Applicatif d'echange Mail (SMTP, POP et IMAP)

XII

1. Protocoles d'échange de mail



- Si le web est l'application qui domine Internet actuellement, l'échange de mails est classé en deuxième position ;
- Le système d'échange de courriers électroniques est inspiré du modèle de courrier traditionnel (*sauf que la transmission de courrier électronique est instantanée*). Deux types de protocoles composent ce système : Protocoles de transmission de mail (SMTP), et les Protocoles d'accès aux mails (POP et IMAP) ;

L'envoi d'un mail passe par deux phases :

1. La première phase consiste à transmettre le mail au serveur mail du destinataire (cette étape est assurée par le protocole *SMTP*) ;
2. La deuxième phase consiste à que l'utilisateur récupère le mail depuis le serveur (cette étape est assurée par le protocole *POP* ou *IMAP*) ;

2. Format de mail électronique

- Comme d'autres protocoles (TCP, IP, FTP, HTTP, etc), les protocoles d'échange de mail fonctionnent avec un format mail. En plus du message qu'un utilisateur souhaite transmettre, *le mail aura un ensemble des entêtes qui décrivent comment le mail sera traité ainsi que son contenu et à qui il est destiné* ;
- Deux formats de mail ont été conçus pour cet objectif :

- Standard *RFC 822* (conçu seulement pour le contenu texte);
- Format *Multipurpose Internet Mail Extensions (MIME)* (conçu pour transférer d'autres types de contenus comme les fichiers images etc);

Remarque : Standard *RFC 822*

- Même si le standard *RFC 822* a été conçu pour des messages textuels seulement, celui-ci permet la définition des entêtes personnalisés par l'utilisateur. La seule contrainte est que le récepteur du message puisse interpréter l'entête personnalisé;
- Ainsi, lors du développement du format MIME, le format *RFC 822* initial a été étendu avec des entêtes spéciaux pour encoder chaque partie du message séparément ;

3. Multipurpose Internet Mail Extensions (MIME)

Le format MIME (Multipurpose Internet Mail Extensions) est basé sur le standard *RFC 822* et ajoute des entêtes pour encoder le contenu multimedia sous forme ASCII avec des entêtes supplémentaires indiquant comment reconstruire le format multimédia à partir du contenu ASCII.

Deux types de structures de contenu MIME existent :

- *Structure Simple (Media Discret)* : le contenu du media est d'un seul type (image ou texte, etc);
- *Structure Complexe* : le message contient plusieurs types de media. Ainsi, le message est composé de plusieurs *body parts* (dit aussi des entités MIME) tel que chaque partie est une *Structure Simple* ;

Avec la structure MIME, le message entier ainsi que les parties (pièces jointes, images, etc) qui le compose sont décrites par des entêtes MIME ;

Les entêtes MIME basiques sont les suivants :

- *MIME-Version*: Cet entête signifie que le message est structuré avec le format MIME et avec quelle version de celui-ci. Cet entête est présent seulement pour le message entier (les parties qui composent le message ne contiennent pas cet entête) ;
- *Content-Type*: Indique le type du contenu du message ou d'une partie du message (exemple : *Content-Type : text/html* pour le contenu texte html ou bien *Content-Type : image/jpeg* pour le contenu image de type jpeg) ;
- *Content-Transfer-Encoding*: Pour un message d'une structure simple indique l'encodage du message. Pour un message composé, indique l'encodage de chaque partie du message ("*Content-Transfer-Encoding : 7bit*" pour encoder un contenu ASCII);
- *Content-ID* : Identifiant d'une partie du message (similairement au Message-ID, celui ci identifie d'une manière unique chaque partie composant un mail électronique) ;
- *Content-Description* : Entête optionnel permettant de décrire le contenu d'un message ou d'une partie d'un message ;

D'autres entêtes MIME ont été définies qui ne sont pas seulement utilisés pour les courriers électroniques (HTTP utilise ces entêtes aussi) :

- *Content-Disposition* : Indique comment le contenu est affiché chez le récepteur ("*Content-Disposition : inline*" pour indiquer que le contenu d'une entité MIME doit être affiché dans le message, "*Content-Disposition: attachment*" indique que le contenu de l'entité MIME est une pièce jointe du mail) ;
- *Content-Length* : Indique la taille du contenu de l'entité en octets ;



Complément

L'entête *Content-Type* Indique au récepteur la nature de l'entité MIME pour que celle-ci soit décodée d'une manière appropriée. La syntaxe générale de cet entête :

Content-Type: <type du contenu>/<Sous Type du contenu> [; parameter1=Valeur1 ; parameter2=Valeur2,; parameterN,=ValeurN]

<type de contenu> = peut prendre l'une des valeurs suivantes : text, image, audio, video ou application

<Sous Type du contenu> = le sous type spécifique de ce contenu (par exemple Content-Type: Image/PNG désigne le sous type *PNG* du type *Image*)

Par exemple :

- Content-Type: text/html ; charset=ascii
- Content-type: image/jpeg; name="ryanpicture.jpg"



Fondamental

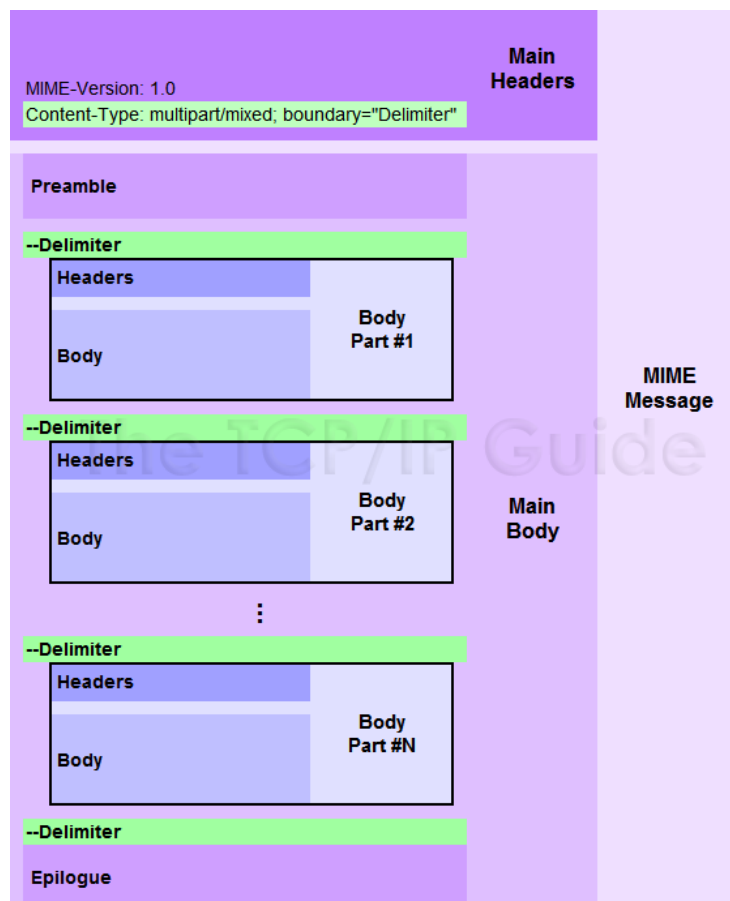
Comme les entêtes HTTP, la ligne contenant un entête MIME ou RFC822 *se termine par un CRLF*

4. Contenu MIME composé

Pour indiquer que le message contient une structure MIME complexe, l'entête Content-type prend la valeur *multipart* ou *message* (dans le cas où le message contient un seul type de media, la valeur de cet entête prend simplement texte, image, audio, vidéo ou application) ;

multipart : Indique que le message est une entité MIME contenant plusieurs entités MIME de type simple ;

message : Indique que le message incorpore un autre message (un autre mail entier / partiel par exemple);

Figure XII.1. Structure mail MIME^{3*}

Syntaxe : Exemple multipart mail

Un contenu de type *multipart* doit contenir la spécification d'un séparateur qui délimite la fin d'une entité MIME imbriquée et le début d'une autre. Un message contient aussi un *preamble* au début et épilogue à la fin du message qui ne seront pas traités par le serveur.

Le séparateur (*boundry delimiter*) est choisi d'une manière aléatoire pour éviter qu'il soit confondu avec le contenu du message. Pour marquer la fin d'une entité, le séparateur est précédé par "--";

Dans l'exemple suivant, `boundary="exempldelimtext123"` (ligne 6) indique que les parties du message sont séparées par `--exempldelimtext123` (lignes 8, 15, 21)

From: Joe Sender <joe@someplace.org>

To: Jane Receiver <jane@somewhereelse.com>

Date: Sun, 1 Jun 2003 13:28:19 -0800

Subject: Photo and discussion

MIME-Version: 1.0

Content-Type: multipart/mixed; boundary="exempldelimtext123"

This is a multipart message in MIME format

--exempldelimtext123

Content-Type: text/plain

Jane, here is the photo you wanted me for the new client.

Here are some notes on how it was processed.

(Blah blah blah...)

Talk to you soon,

Joe.

—exampledelimtext123

Content-Type: image/jpeg; name="clientphoto.jpg"

Content-Transfer-Encoding: base64

SDc9Pjv/2wBDAQoLCw4NDhwQEBw7KCIo0zs70zs70zs

...

zv/wAARCADIARoDASIAAhEBAXEB/8QAHAQAAQUBA

—exampledelimtext123

(Epilogue)

5. SMTP (Simple Mail Transfer Protocol)

- Pour que une machine soit capable de recevoir un mail par SMTP, *celle-ci doit être fonctionnelle 24/7* pour recevoir le mail au moment de son envoi. Ce genre de machine est appelée *Serveur Mail* ;
- SMTP utilise typiquement le port TCP 25 ;
- SMTP utilise les enregistrements *DNS MX* (voir le cours DNS) pour récupérer le nom / adresse IP du serveur mail destinataire ;



Exemple : Utilisation DNS pour repérer le serveur mail du destinataire

Supposant qu'une personne possédant l'adresse mail *P1@gmail.com* souhaite transmettre un mail à *P2@hotmail.com*. (*P1 est P2 possèdent des boîtes mails dans deux différents serveurs mails : gmail.com et hotmail.com*)

1. Ce mail passera au début par le serveur mail du domaine *gmail.com* ;
2. Par la suite, le serveur *gmail.com* relayera le mail au serveur mail du destinataire. Pour faire ceci, le serveur mail de *gmail.com* envoie une *requête DNS pour enregistrements MX dans le domaine hotmail.com afin de repérer le serveur mail dans ce domaine* ;
3. Cette recherche retourne le nom de la machine suivant : *hotmail-com.olc.protection.outlook.com* ; Après la récupération du nom du serveur mail, le serveur mail *gmail.com* initie une session SMTP pour transférer le mail au serveur mail *hotmail-com.olc.protection.outlook.com* (connecte au port TCP 25) ;

Cette étape est omise si le nom de domaine du serveur mail destinataire est le même que celui de la source, par exemple *P1@gmail.com* envoie un mail à *P2@gmail.com* ;

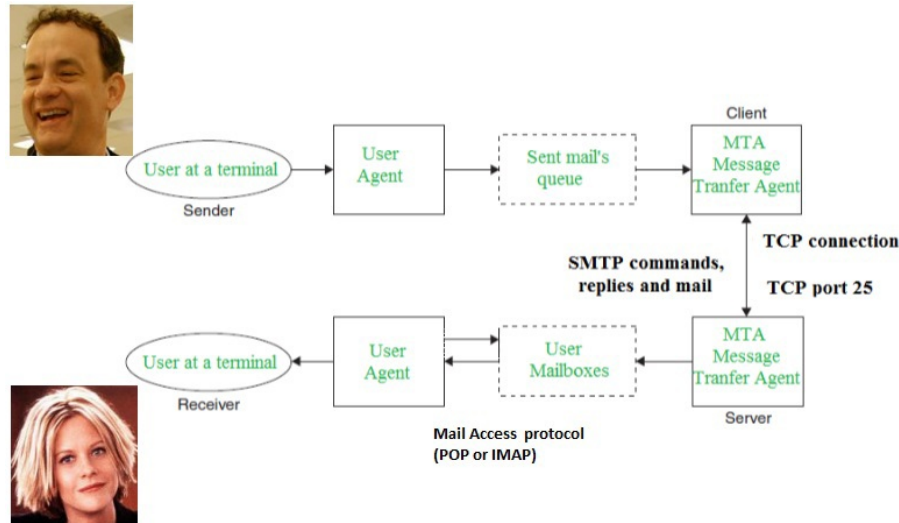


Figure XII.2. Protocoles mails

- Un serveur transmetteur initie une session SMTP par une connexion au port 25 TCP du serveur récepteur
- Le serveur récepteur envoie automatiquement un Code 220 avec un message indiquant qu'il est prêt à recevoir des commandes SMTP;
- Le serveur transmetteur par la suite commence par envoyer une des deux commandes suivantes : *HELO* ou *EHLO* avec son nom du domaine; Par exemple, si *med@Master1.edu* souhaite transmettre un mail à *amine@Liscence3.edu* alors le serveur mail *mail.Master1.edu* (le nom du serveur mail *Master1.edu*) établie une connexion *mail.Liscence3.edu* (le nom du serveur mail *Liscence3.edu*) au port TCP/25 et ensuite une commande "*HELO mail.Master1.edu*" est envoyée de *mail.Master1.edu* vers *mail.Liscence3.edu* ;
- Le récepteur répondra avec le code 250 et son nom de domaine ;
- Si le client transmet *EHLO* au lieu de *HELO*, ceci indique que le transmetteur supporte les extensions de SMTP. En réponse, le serveur récepteur envoie les extensions supportées (par exemple : *PIPELINING* : Commande Pipe Lining pour permettre au serveur transmetteur d'envoyer plusieurs commandes sans attendre les réponses correspondantes aux commandes précédentes ; Message Size Declaration "*SIZE*" pour déclarer la taille maximal du message que le récepteur peut accepter, *DSN* : Delivery Status Notification permet de recevoir des notifications à propos de la livraison / échec du message)
- Pour fermer la session SMTP après la transmission du mail, le transmetteur envoie une commande *QUIT*. Le récepteur répond par un code 221 et un message "*good bye*" et clôture la connexion TCP;

6. Transmission du mail avec SMTP

- Après l'établissement de la session SMTP, la transmission du mail avec SMTP passe par trois étapes :
 1. Envoie de l'adresse mail du transmetteur ;
 2. Envoie des adresses mail des récepteurs (ceux qui possèdent des boîtes mails dans le serveur récepteur) ;
 3. Envoie du contenu du mail ;
- L'adresse mail du transmetteur et la liste des récepteurs représente une enveloppe du contenu du mail ;

- Même si le mail lui même contient les informations nécessaires (*Standard RFC 822 les entêtes To ; From ; Cc ; Bcc ; etc*) pour livrer le mail au destinataire, la transmission des adresses des récepteurs séparément est plus efficace du coté du serveur récepteur;
- Le transmetteur peut vérifier la validité d'une adresse mail par la commande *VRFY* (Par exemple : transmettre "*VRFY codek9111*" vers yahoo.fr afin de vérifier si codek9111@yahoo.fr est une adresse valide dans yahoo.fr) ;
- Pour commencer la transmission du courrier, le serveur transmetteur commence par envoyer la commande *MAIL* suivie par un entête *From* : qui indique l'expéditeur du mail ;
- Par la suite, la commande *RCPT* est transmise suivie par l'entête *To* : et la liste des destinataires gérés par le serveur récepteur ;
- La commande *DATA* indique le début de la transmission du mail lui même. La fin de la transmission du mail est indiquée par la séquence "*CRLF.CRLF*";
- La commande *RSET* sert à arrêter la transmission du mail (suite à une erreur) ;
- *NOOP* permet de vérifier que la connexion avec le serveur est toujours établie ;

De l'autre coté le serveur récepteur répond avec des codes de réponse similaires à ceux générés par FTP (trois chiffres XYZ):

- X prend une valeur de 1 jusqu'à 5 (1 == Réponse positif préliminaire, 2== Commande exécutée avec succès, 3 == Réponse positif intermédiaire, 4== Échec de l'exécution qui peut être temporaire, 5 ==Échec de l'exécution permanent) ;
- Y prend une valeur de 0 jusqu'à 5 (0 == réponse relative à la syntaxe, 1== réponse à une demande d'information, 2== réponse relative à la connexion entre serveur transmetteur et récepteur, la signification de 3 et 4 n'est pas définie, 5== réponse relative au service Mail)
- Z prend une valeur de 0 à 9 et donne 10 codes par catégorie ;

```
HELO Libor
250 dns.terminal.cz Hello Libor, pleased to meet you
MAIL FROM: dostalek@t1.pvt.cz
250 <dostalek@t1.pvt.cz>... Sender ok
RCPT TO: dostalek@t2.pvt.cz
250 <dostalek@t2.pvt.cz>... Recipient ok
DATA
354 Enter mail, end with "." on a line by itself

MIME-Version: 1.0
Content-Type: text/plain
Disposition-Notification-To: dostalek@t1.pvt.cz
Subject: xx
message text
.
250 RAA39113 Message accepted for delivery
```

Figure XII.3. Protocole SMTP^{7*}

Remarque : Content-encoding : base64

Les parties binaires d'un mail sont encodées avec base64 pour éliminer les caractères spéciaux (*CRLF*). Ceci empêche que le contenu transmis par l'utilisateur contient des commandes SMTP valides lors du transport mail.

7. Fonctionnalités de sécurité de SMTP

Plusieurs fonctionnalités ont été implémentées par les serveurs mail pour éliminer les failles de SMTP (cependant chaque serveur implémente ses mécanismes différemment) :

- Limite de la taille / nombre des messages acceptées ;
- Accepter les connexions seulement des appareils autorisés ;
- Vérification de la validité des mails des récepteurs avant d'accepter la réception du mail ;
- Restriction de commandes aux utilisateurs authentifiés ;
- Logging des accès au serveur ;
- Authentification avec *POP before SMTP* ;
- Plusieurs commandes ont été désactivées (par exemple *VERFY* a été désactivée car cette commande a été abusée pour le spam)

8. Post Office Protocol (POP)

- Après la livraison du courrier au serveur mail approprié, l'utilisateur concerné par le courrier pourra le voir lors de son prochain accès à ça boîte mail. L'accès au mail se fait par un protocole indépendant à SMTP. Les deux protocoles d'accès aux mails les plus utilisés sont *POP (Post Office Protocol)* et *IMAP (Internet Message Access Protocol)*.
- La première version de POP (*POP3 est la version courante de ce protocole*) permet simplement à un utilisateur de :
 1. Transmettre son nom et mot de passe pour s'authentifier ;
 2. Ensuite, l'utilisateur peut accéder à ça boîte mail et télécharger ses mails (le contenu de la boîte complète) ;
- Similairement au serveur SMTP, un serveur POP fonctionne 24/7 pour recevoir les demandes de connexion utilisateurs et les servir. Ce type de serveur écoute *généralement* le port *TCP 110* ;
- La dépendance entre POP et SMTP est claire. POP permettra d'accéder aux mails reçus par le serveur SMTP. Ainsi, le serveur SMTP et le serveur POP peuvent être hébergés dans la même machine. Ou bien, le serveur POP possède un accès par réseaux au dépôt des mails SMTP ;

9. Commandes et réponses POP

- Les commandes POP sont de 3 à 4 lettres ASCII comme FTP et SMTP. Par contre les codes réponses sont de deux types seulement : *+OK (commande exécutée avec succès)* et *-ERR (échec d'exécution de la commande)*.
- Chaque commande / réponse échangée entre le serveur et client est terminée par *CRLF* ;
- La session POP commence par une connexion TCP entre le client et le serveur sur le port dédié *110 TCP*.

9.1. Authentification

1. Immédiatement après la connexion, le serveur transmet un message du genre : *" +OK Howdy !"* pour indiquer au client qu'il peut commencer l'échange avec le serveur ;
2. Le client s'authentifie ensuite d'une manière identique à FTP (envoi de la commande *USER* avec le nom de l'utilisateur et ensuite *PASS* avec le mot de passe) ;
3. Si l'utilisateur s'authentifie correctement, le serveur répondra avec un code *+OK* et un message ;


```
+OK Howdy! we are at your service!
USER codek9111@boitemail.com
+OK Waiting for password
PASS CodingMonk?L7*5
+OK codek9111@boitemail.com has 9999 messages
```

9.2. Commande d'échange avec le serveur

Les commandes les plus importantes pour gérer et accéder à la boîte mail sont les suivantes :

1. *STAT* : renvoie le nombre de mails dans la boîte et la taille totale occupée dans celle-ci ;
2. *LIST* : renvoie une liste contenant le numéro de chaque mail accompagné avec ça taille ;
3. *RETR* : récupère le mail désigné par le numéro en paramètre sous format RFC 822 (*voir le cours SMTP*).
Par exemple: après l'envoi "*RETR 1090CRLF*" le serveur répond avec le contenu du mail 1090 si celui-ci existe ;
4. *DELE* : marque le mail désigné *pour être supprimé du serveur* à la fin de la session (le message désigné par cette commande ne sera pas immédiatement supprimé, voir la dernière commande *QUIT*);
5. *NOOP* : teste si la connexion avec le serveur est toujours vivante ;
6. *RSET* : réinitialisation de la session (les suppressions des messages par *DELE* seront annulées) ;
7. *TOP* : renvoie les entêtes du message désigné et les N premières lignes du message ; "*TOP 1 10CRLF*" renvoie l'entête du message en plus des 10 premiers lignes du contenu du message (pas tous les serveurs mails supportent cette commande);
8. *UIDL* : retourne la liste des identificateurs uniques des messages dans la boîte ;

La dernière commande à transmettre au serveur est la commande *QUIT*. Celle-ci indique la fin de la session *ainsi que confirme la suppression des mails marqués par le commande DELE*. Avant l'envoi de *QUIT*, l'utilisateur peut annuler les suppressions par *RSET*;

Si une déconnexion entre le client et le serveur se produit sans que le client envoie "*QUIT*", les messages marqués par *DELE* ne seront supprimés ;

Le modèle d'opération de POP est *offline* car l'utilisateur récupère ses messages sur ça machine locale et *ensuite typiquement supprime les messages déjà lus du serveur*.

Cependant, POP ne permet pas de marquer les messages lus ou non lus par exemple, non plus de regrouper la boîte mail sur plusieurs catégories/ répertoires (spams, news, social medias, work etc). Ce genre de fonctionnalités est assuré par un protocole d'accès aux mails plus sophistiqué appelée IMAP (*Internet Message Access Protocol*);

Remarque

Comme dans le format MIME, le serveur signale la fin d'une réponse par un "." au début de la ligne ;

```
1 STAT
2 +OK 2 574
3 LIST
4 +OK
5 1 414
.
```

```

6 2 160
7 .
8 RETR 1
9 +OK
10 (Message 1 is sent)
11 .
12 DELE 1
13 +OK message 1 deleted
14 RETR 2
15 +OK
16 (Message 2 is sent)
17 .
18 DELE 2
19 +OK message 2 deleted
20 QUIT
21 +OK Bye

```

10. Internet Message Access Protocol (IMAP)

- Similairement à POP, un serveur IMAP permet au client d'accéder à ça boîte de mail. Afin d'accomplir ceci, le serveur IMAP doit accéder aux mails livrés par SMTP (*comme expliqué dans la partie POP3*) ;
- Un serveur IMAP écoute le port TCP/143 ;
- Après la connexion du client au port *TCP/143* , le serveur *IMAP* généralement envoie un message avec le code **OK* (comme le +OK de POP) ;
- Dans certaines situations, le serveur peut reconnaître l'identité du client dès le début de la session. Dans une telle situation le serveur répond avec le code **PREAUTH* et le nom de l'utilisateur pré-authentifié. Cette fonctionnalité est appelée Pré-authentification et dans un tel cas, l'utilisateur n'a pas besoins de s'authentifier par un nom utilisateur et mot de passe ;
- En cas où le serveur n'accepte pas une connexion ou clôture la session, la réponse **BYE* est transmise vers le client ;
- Similairement à POP, en cas d'échec d'une commande le serveur répond avec **NO*. Par contre, les réponses indiquant une erreur sont distinguées par le code **BAD* ;
- Comme POP, les commandes IMAP sont délimitées par CRLF. Par contre, chaque commande transmise par le client doit avoir un code unique (généralement il s'agit d'un code ascendant). Ainsi, la réponse du serveur comprendra l'identifiant de la commande correspondante. De plus, ceci permet l'envoi de plusieurs commandes sans attendre les réponses de commandes précédentes (*par contre, ce-ci ne s'applique pas à toutes les commandes*) ;

11. Les commandes IMAP

Remarque

- Contrairement à POP, IMAP est conçu pour permettre de regrouper / gérer l'espace de l'utilisateur sur plusieurs boîtes mails (*répertoire*). Ainsi, après l'authentification, au lieu que l'utilisateur n'accède directement à ses mails (comme dans POP) , celui-ci sélectionne la boîte mail à la quelle il souhaite accéder ;
- La spécification détaillée de IMAP dit que l'utilisateur a accès à plusieurs boîtes mails, mais en réalité, l'utilisateur possède une seule boîte organisée sur plusieurs répertoires (*folders*).

Exemple :

L'utilisateur propriétaire de *utilisateur@boitemail.com* accède à ça boîte après authentification. Et ensuite, il a le choix entre plusieurs boîtes : "SPAM", "INBOX", "URGENT", etc. Dans l'étape de la sélection, l'utilisateur fait le choix entre les différents répertoires (et non pas une autre boîte du genre *utilisateur.MABOITE2@boitemail.com*) ;

Les commandes IMAP sont réparties sur trois groupes :

- *Commandes "Any state"* : ce type de commande peut être exécuté à n'importe quel moment.
CAPABILITY : pour avoir les fonctionnalités supportées par le serveur ; *NOOP*: pour vérifier si la connexion avec le serveur est toujours valable ; *LOGOUT* : pour quitter la session;
- *Commandes d'authentification* : permettant de s'authentifier ;
- *Commandes après authentification* : généralement, ce genre de commande permet de sélectionner, lister et créer les répertoires (INBOX, SPAM, DRAFT) de la boîte mail ;
- *Commandes après sélection* : permettent d'accéder / gérer les mails situés dans une boîte ;

par ces commandes, l'utilisateur transite entre les états suivants :

1. *Non-Authentifié* : l'utilisateur doit s'authentifier pour passer à l'état "(2) Authentifié" et accéder à son espace;
2. *Authentifié* : Dans cet état, l'utilisateur doit sélectionner la boîte /répertoire au quel il souhaite accéder ;
3. *Boîte/répertoire sélectionnée* : Une fois que la boîte est sélectionnée, l'utilisateur peut accéder et gérer les mails contenus dans la boîte selectionnée ;
4. *Déconnecté* : Une fois que l'utilisateur transmet la commande LOGOUT, celui-ci se déconnecte de la session ;

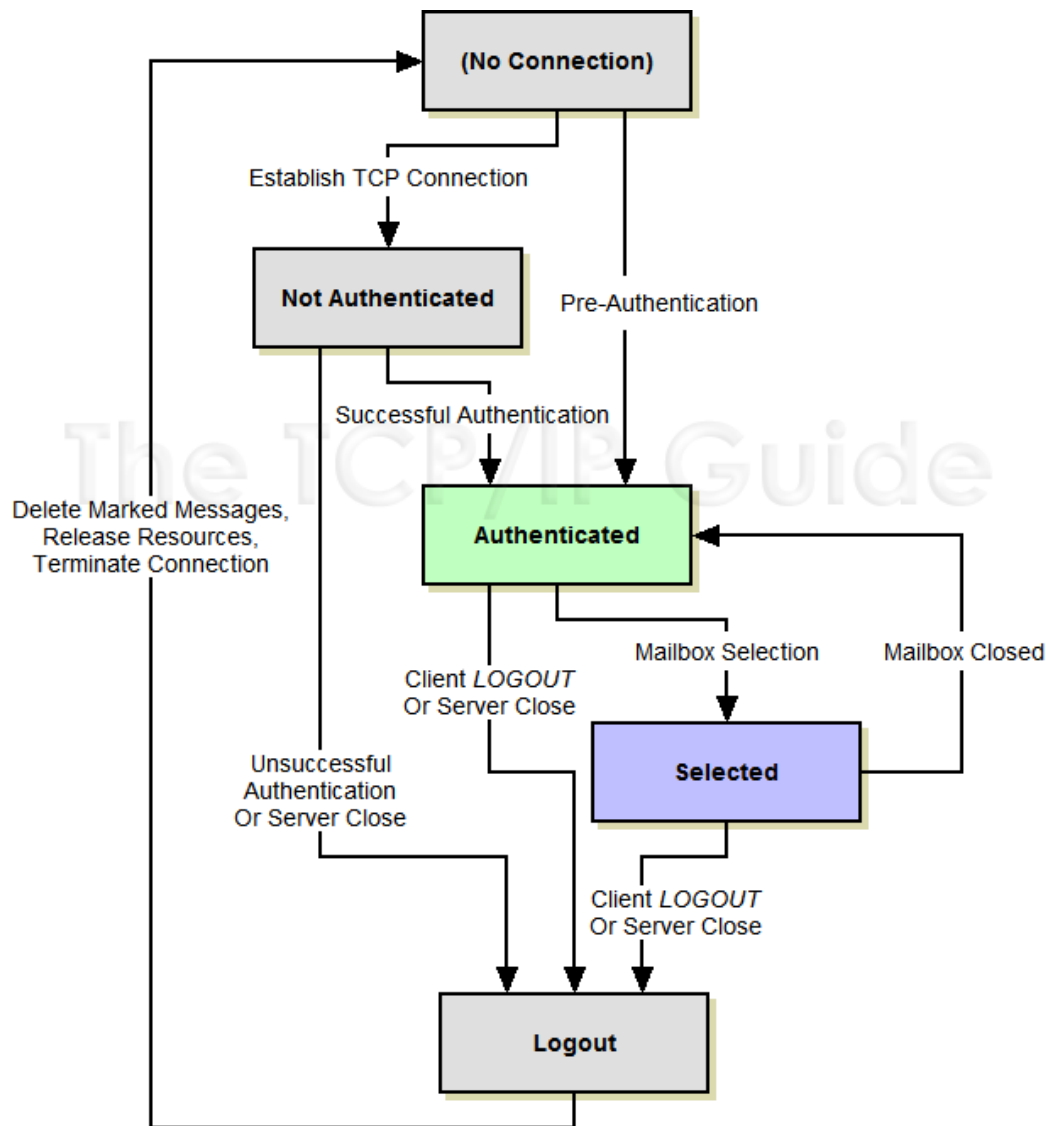


Figure XII.4. État Client IMAP^{3*}

11.1. Commandes d'authentification

- Traditionnellement, l'utilisateur commence par s'authentifier par la commande *LOGIN*. Contrairement à la démarche d'authentification POP, le mot de passe et le nom d'utilisateur sont transmis dans une seule ligne en argument de la commande *LOGIN* ;
- En cas de réussite d'authentification, le serveur répond par **OK* et un message textuel. Sinon, le serveur renvoie le code **NO* pour dire que le processus a échoué ;

Syntaxe

*OK Server ready to receive commands

1 LOGIN nomutilisateur@boitemail.com motdepasseutilisateur

1 OK nomutilisateur@boitemail.com logged in successfully

2 LOGOUT

* BYE tethys.ringofsaturn.com IMAP4rev1 server terminating connection

2 OK LOGOUT completed

- Dans l'exemple précédant, l'utilisateur fournit le nom d'utilisateur et mot de passe sur la même ligne avec la commande LOGIN (*nomutilisateur@boitemail.com motdepasseutilisateur*) ;
- La première commande "LOGIN nomutilisateur motdepasseutilisateur" possède le tag "1" (préfixe). La réponse correspondante commence par le même tag (1). La même chose pour la commande "2 LOGOUT" (*tag =2*) ;

11.2. Commande après authentification :

Après l'authentification, l'utilisateur peut gérer ces boîtes mail (mais ne peut pas accéder directement aux mails avant de spécifier à quelle boîte mail il souhaite accéder). Les commandes les plus importantes de cette étape sont :

- *SELECT* : Permet de sélectionner une boîte mail. Cette commande a pour paramètre le nom de la boîte que l'utilisateur souhaite accéder.
- *EXAMINE* : Fonctionne exactement comme *SELECT* sauf que l'accès à la boîte est en mode de lecture seul (aucun changement à la boîte ne peut être fait) ;
- *CREATE* : permet de créer un nouveau répertoire mail ;
- *DELETE* : Supprime la boîte mail avec le nom fournie en paramètre ;
- *RENAME* : Permet de renommer une boîte mail ("*RENAME NOMBOITE NOUVELNOMBOITE*" renomme la boîte *NOMBOITE* à *NOUVELNOMBOITE*) ;
- *LIST* : Revoie une liste des noms des boîtes mails qui correspond partiellement ou complètement à la chaîne de caractères en paramètre ;
- *STATUS* : Renvoie des informations (comme le nombre de mails non lus dans la boîte) sur la boîte mail avec le nom passé en paramètre ;
- *APPEND* : Ajoute un message à une boîte mail ;

Si la commande *SELECT* ou *EXAMINE* s'exécute correctement (la boîte sélectionnée par le client existe), le serveur répondra avec les informations suivantes concernant la boîte sélectionnée :

1. *EXISTS*: Indique le nombre total des mails dans la boîte ;
2. *RECENT* : Le nombre de mails récemment reçu ;
3. *FLAGS* : Flags des mails supportés par le serveur (généralement : "*\Seen*", "*\Answered*", "*\Flagged*", "*\Deleted*", "*\Draft*" et "*\Recent*") ;
4. *UNSEEN* (*information optionnelle*): le numéro du premier (le plus ancien) mail qui n'a pas été lu ;
5. *PERMANENTFLAGS* (*information optionnelle*): Liste des flags que l'utilisateur peut changer ;
6. *UIDNEXT* : Identifiant unique anticipé du prochain message reçu dans cette boîte ;
7. *UIDVALIDITY* : Identifiant unique du répertoire dans cette session courante ;



Complément

Les flags qui marquent l'état d'un mail ne permettent pas seulement à un client de savoir quel message a été lu ("*\Seen*") et quel message vient d'être reçu ("*\Recent*"). Ce système permet aussi l'accès aux mails par différents utilisateurs sans que l'un interfère avec les autres.

Par exemple, si plusieurs utilisateurs peuvent accéder aux mails / répertoires de la même boîte

- *UIDNEXT* permet à un client de vérifier si un nouveau message a été reçu depuis sa dernière connexion . Si la valeur de *UIDNEXT* augmente par rapport à l'accès précédent, ce-ci signifie qu'un nouvel mail a été reçu ;

- *UIDVALIDITY* permet au programme client de vérifier si une boîte / répertoire a été modifiée par un autre utilisateur (dans ce cas) ;
Si un utilisateur supprime la boîte avec le nom '*INBOX*' et crée une nouvelle boîte avec le même nom ('*INBOX*'), les autres utilisateurs qui accéderont par la suite à '*INBOX*' remarqueront le changement du *UIDVALIDITY* depuis le dernier accès.

11.3. Commandes après sélection de la boîte

Une fois que la boîte est sélectionnée, l'utilisateur peut effectuer des opérations sur les messages de la boîte :

- *SEARCH* : permet de chercher dans la boîte sélectionnée selon le critère en paramètre ;
- *STORE* : Change les flags du message en paramètres ;
- *FETCH* : permet de récupérer un ou plusieurs messages de la boîte mail. Cette commande peut être utilisée pour récupérer une partie du message (Date, Enveloppe, Texte, etc) ;
- *COPY* : copie un ou plusieurs messages à la boîte mail spécifiée en paramètre ;
- *EXPUNGE* : supprime tous les messages qui sont marqués avec le flag "*\Deleted*";
- *CLOSE* : Ferme la boîte courante pour que l'utilisateur peut choisir une autre. Comme dans POP, *CLOSE* opère de la même manière que *QUIT* : quand l'utilisateur envoie la commande *CLOSE* tous les messages marqués par un flag *\Delete* seront supprimés ;



Syntaxe : *FETCH*

Les exemples suivants montrent la syntaxe de la commande *FETCH* :

```
1 TAG001 FETCH 1 full # le serveur transmettra le contenu complet du message 1
2 TAG002 FETCH 3:6 full # le serveur transmettra le contenu complet des messages de
3 à 6
3 TAG003 FETCH 5 FLAGS # le serveur transmettra les flags du message 5
4 TAG004 Fetch 10 (BODY.PEEK[HEADER]) # Renvoie de l'entête du message seulement
5 TAG005 FETCH 1 BODY.PEEK[TEXT] # Renvoie du contenu du message seulement
6 TAG006 FETCH 1 RFC822.SIZE # Renvoie de la taille du message seulement
7 TAG007 FETCH 4 BODY[] # le serveur transmettra le contenu complet du message 4
(fonctionne comme le premier exemple)
```



Syntaxe : *STORE*

TAG001 STORE 20 +FLAGS (\Deleted \Seen) # ajoute le flag *\Deleted* et *\Seen* au message 20

TAG002 STORE 1 -FLAGS (\Deleted \Seen) # supprime le flag *\Deleted* et *\Seen* de la liste des flags du message 1

TAG003 STORE 30 FLAGS \Answered # toute la liste des flags précédents du message 30 seront remplacés par le flag *\Answered*



Syntaxe : Commande *SEARCH*

4 SEARCH FROM notifications@github.com

* SEARCH 1789 1791

4 OK SEARCH Completed

5 SEARCH TO support@cybrary.it

* SEARCH 1900

5 OK SEARCH Completed

- La séquence de commandes prétendant *avec tag ==4* permet de rechercher les messages reçus de la part *notifications@github.com* (en résultat on a obtenu le message numéro 1789 et 1791). La séquence avec *tag==5*, permet de rechercher les mails adressés à *support@cybrary.it*.
- Pour rechercher les mails correspondant à un Flag *\SEEN* : *SEARCH SEEN*
- La recherche peut se faire par contraintes de date :

```
1 TAG001 SEARCH SENTBEFORE 12-Mar-2016 # reçu avant 12-Mar-2016
2 TAG002 SEARCH SENTON 12-Mar-2016 # reçu le jour 12-Mar-2016
3 TAG003 SEARCH SENTSINCE 12-Mar-2016 # reçu depuis 12-Mar-2016
4 TAG006 SEARCH BEFORE 12-Mar-2016 # reçu avant 12-Mar-2016
5 TAG007 SEARCH YOUNGER 3600 # reçu il y a moins d'une heure (3600sec)
6 TAG008 SEARCH OLDER 3600 # reçu il y a plus d'une heure (3600sec)
```

La syntaxe des conditions complexes est une syntaxe *postfixée*

```
1 SEARCH OR (FROM notification@github.com) (FROM issues-reply@bitbucket.org) #
  message de la part de notification@github.com ou issues-reply@bitbucket.org
2 -----
3 SEARCH NOT (OR (FROM notification@github.com) (FROM issues-reply@bitbucket.org))
  # message ne provenant pas ni de notification@github.com ni de issues-reply@bitbucket.
  org
4 -----
```



Complément : Fetch

Le serveur marquera automatiquement un message avec le flag *\Seen* si son contenu a été récupéré par l'utilisateur (avec *FETCH*) et que celui-ci a accédé à la boîte mail avec *SELECT* au lieu de *EXAMINE*. Par contre, les Flags ne changeront pas si l'accès à la boîte été fait par *EXAMINE*.

Chapitre XIII Software Defined Network

XIII

1. Problématiques dans le réseau TCP/IP

- Avec l'évolution des applications et la perspective de futures services, le modèle TCP/IP est devenu incapable de répondre aux besoins de celles-ci.
- Dans un réseaux TCP/IP, le changement de la politique réseau nécessite la reconfiguration de chaque équipement (Switch, Routeur) manuellement. Dans un réseau grand échelle ceci est :
 - *Difficile à accomplir;*
 - *Coûteux en temps et en argent;*
 - *Peut introduire des erreurs de configuration;*
- Cette inflexibilité du réseau TCP/IP a poussé les chercheurs à trouver un nouvel paradigme répondant à ce problème. Ce paradigme, est appelé *Software Defined Network (SDN)*.

2. Principes du SDN

- L'idée de cette technologie est de permettre la programmation du réseau à partir d'une instance centrale qui gouverne le réseau entier. Cette instance est appelée le *Contrôleur*.
- Par ailleurs, l'intelligence des Switchs / Routeurs est exportée de ces derniers au contrôleur. Par intelligence, on désigne les algorithmes (RIP, OSPF, etc) permettant de créer la table de routage au niveau du routeur et la table Source Address Table (SAT) du switch.
- Ce paradigme introduit une nouvelle table utilisée pour dé *Flow Table (Table des Flux)*.
- *L'intelligence des équipements est migré au Contrôleur.*

3. Table de Flux

- Chaque entrée de la Table de Flux est répartie en trois sections:
 - *Matching Rules (aussi appelées Header Fields):* joue le rôle des adresses IP pour les routeurs/ MAC pour les switchs. Par contre, cette section comporte plusieurs valeurs de champs (Numéro de port switch, Numéro de port de la couche transport, Protocole de transport, etc) contrairement à seulement l'adresse IP/MAC. Cette section permet de vérifier si un paquet correspond à la description d'un flux.
 - *Counters:* comporte des statistiques (nombre de paquets, bits, temps écoulé) pour le flux correspondant (celui dans l'entete correspond au Matching Rules).

- *Action*: désigne une ou plusieurs actions à effectuer sur à chaque paquet qui correspond aux Matching Rules. L'action peut être de supprimer le paquet, de changer l'adresse IP destination / source, transmettre le paquet sur un port donné du switch, etc.
- *Idle Timeout*: Temps après le quel l'entrée du flux sera supprimée de la table de flux *si aucun paquet n'est reçu*.
- *Hard Timeout*: Temps après le quel l'entrée du flux sera supprimée;
- Chaque fois qu'un paquet est reçu par le Switch, les valeurs des entêtes Liaison / IP / Transport sont comparées avec chaque Matching Rules afin d'identifier à quel flux ce paquet appartient. *Une fois qu'une entrée correspondante est trouvée dans la Table de Flux, les actions placées dans cette entrée sont appliquées.*
- *Si aucune entrée ne correspond au paquet, le paquet est transmis au Contrôleur. Celui-ci traite le paquet et renvoie une décision au Switch. Cette décision peut être une nouvelle entrée de la table de flux à installer par le Switch SDN.*

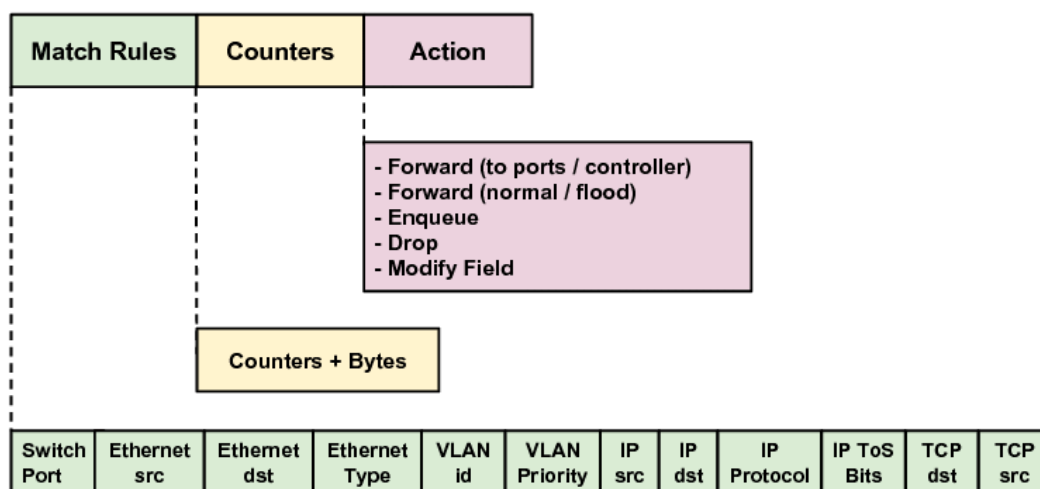


Figure XIII.1. Flux SDN^{10*}

Les actions SDN typiques sont:

- *FORWARD*: Retransmettre le paquet sur un port donné du Switch. Plusieurs types de retransmissions existent:
 - *ALL*: Retransmettre sur tous les ports du Switch;
 - *CONTROLLER*: Encapsuler et retransmettre le paquet au Contrôleur;
 - *LOCAL*: Faire passer le paquet à la couche IP (fonctionnement TCP/IP normal);
- *DROP*: Supprimer le paquet;
- *ENQUEUE*: Mettre le paquet dans une file d'attente d'une interface donnée ;
- *MODIFY FEILD*: Permet de changer la valeur d'un champ le l'entete du paquet (destination source par exemple);
- *Si la liste des Actions correspondante à un flux ne contient aucune action FORWARD, le paquet est supprimé par défaut.*

Par exemple, la deuxième entrée de la table suivante indique que les paquets reçus avec une adresse destination 10.4.1.6 sont retransmis sur le port 2 du Switch. Les cellules avec une valeur * signifient que n'importe quelle valeur sera acceptée (dite aussi ANY);

Remarque: Une entrée appelée Table-Miss doit exister dans la Table de flux afin de définir le traitement des paquets qui ne correspondent à aucune entrée dans la table. Dans cet exemple, cette entrée correspond à la dernière entrée (*toutes les valeurs == *, Action ==Controller*)

Un paquet peut satisfaire les règles de plusieurs entrées. Dans ce cas, les actions de l'entrée avec les valeurs de règles les plus spécifiques (minimum de valeurs ANY) sont appliquées;

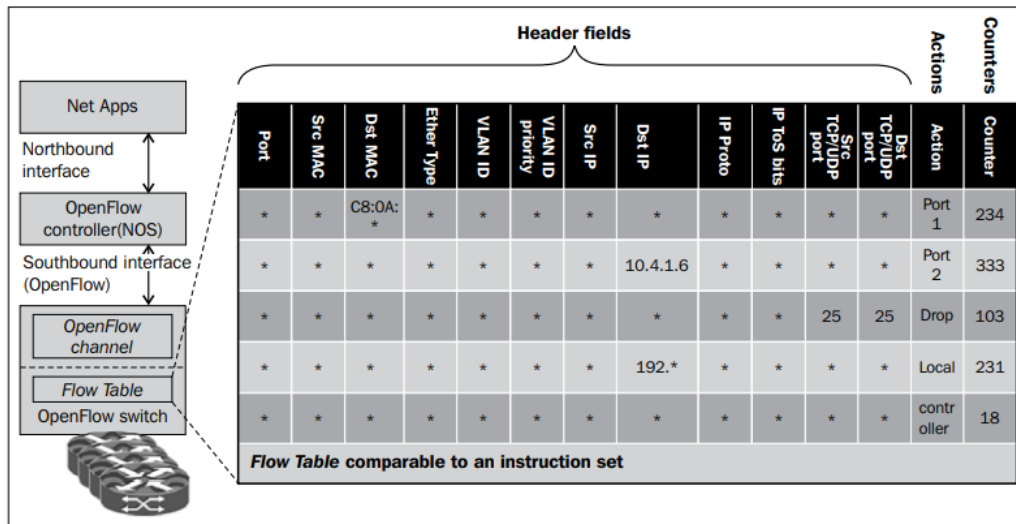


Figure XIII.2. Table des flux SDN^{11*}

4. Contrôleur SDN

- Plusieurs Contrôleurs SDN existent actuellement (POX, NOS, Ryu, Open Floodlight). Un Contrôleur est un logiciel qui contrôle les équipements réseau comme un Système d'exploitation;
- Selon le Contrôleur, celui-ci est programmable avec un langage de programmation moderne (Java, en Python ou C++). La majorité des Contrôleurs actuels et ceux les plus utilisés sont programmables en Python;
- En programmant le Contrôleur, l'utilisateur peut redéfinir la politique du réseau sans reconfigurer les équipements Switchs ;
- Les applications peuvent interagir avec le Contrôleur pour accéder au réseau à travers l'API proposée par le contrôleur. Cette API est appelée aussi Northbound Interface (Interface vers le Nord);

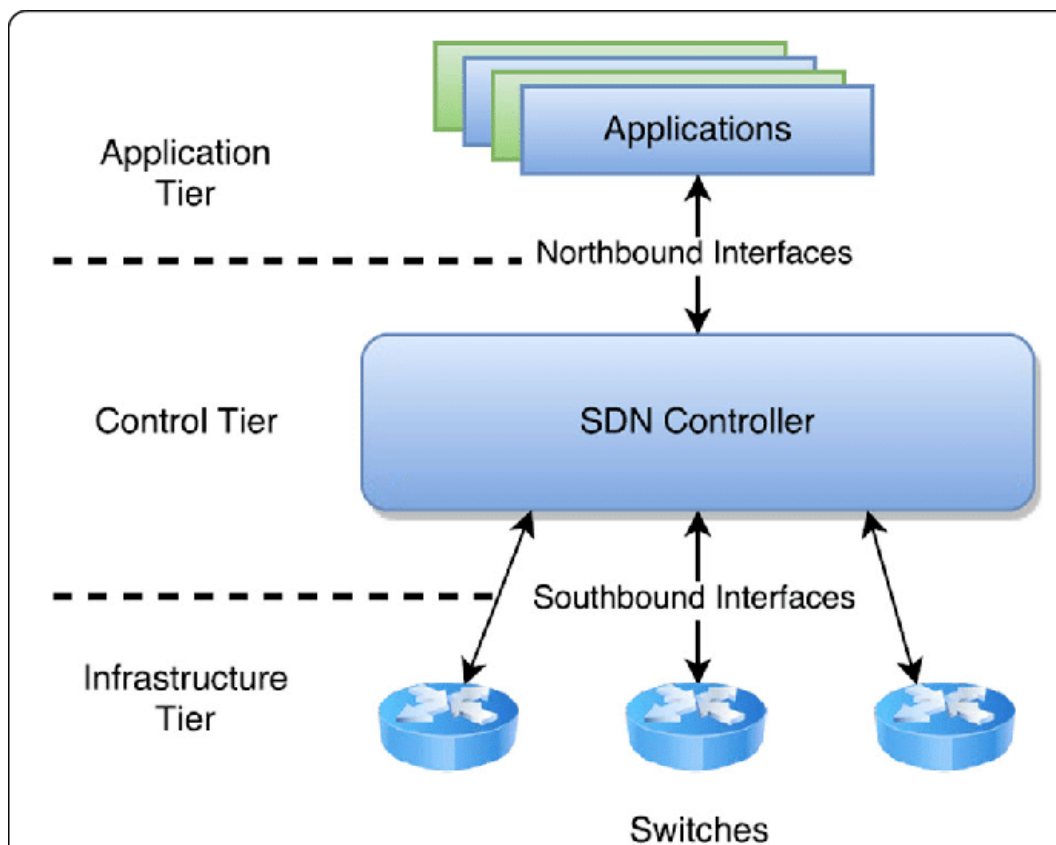


Figure XIII.3. Topologie SDN^{11*}

5. Protocole OpenFlow

- La communication entre les Switchs SDN et le Contrôleur est faite grâce au protocole OpenFlow qui définit les types des messages échangés entre ces derniers.
- Les Switchs SDN doivent être configurés avec un certificat d'authentification et l'adresse IP du Contrôleur. Au lancement du Switch SDN, celui-ci connecte au Contrôleur (par défaut le port TCP 6633 est utilisé par ce protocole) avec le protocole TLS (Transport Layer Security) ;
- Les paquets reçus du Contrôleur auront une action correspondante Local dans la Table de Flux pour que les paquets reçus du Contrôleur soient traités par le protocole OpenFlow;

Remarque : TLS

TLS n'est qu'un protocole de chiffrement qui fonctionne avec TCP et qui commence par l'échange des Certificats numériques (RSA/AES) des deux parties (Switch et Contrôleur) avant que les données ne soient envoyées;

6. Messages OpenFlow

Messages Contrôleur

Après l'établissement de la connexion, le Contrôleur est capable d'interroger un Switch ou de modifier sa configuration / Table de Flux avec les messages suivants:

- *Features*: Le contrôleur peut transmettre un message *Feature Request* pour obtenir les capacités /fonctionnalités supportées du Switch. Ce dernier répond par un *Feature Reply*;
- *Configuration*: Permet de changer ou d'obtenir la configuration d'un Switch;
- *Modify-State*: Ce type de message permet au Contrôleur de manipuler la Table de Flux. Plusieurs sous-types existent pour ce type de message:
 - *ADD*: Permet d'ajouter une entrée à la Table de Flux. Si le flag *Check_Overlap* est allumé, le Switch vérifie si une entrée correspondante existe et si c'est le cas, l'entrée n'est pas ajoutée.
 - *Modify*: Permet de modifier une entrée de la Table de Flux. Si aucune entrée ne correspond au message *Modify*, une entrée est créée dans la Table de Flux.
 - *Delete*: Permet de supprimer l'entrée de flux correspondante.
- *Read State*: Permet de demander les statistiques du Switch;
- *Packet-out / Send-Packet*: Demande au Switch de transmettre le paquet sur un port donné;

Exemple : Création d'entrées dans la table des flux

1. H1 transmet un paquet pour lequel le Switch S1 ne possède aucune entrée (1);
2. S1 transmet un Packet-In au Contrôleur SDN (2);
3. Le Contrôleur SDN transmet un message *Modify-State* pour installer le nouvel flux dans S1 (3);

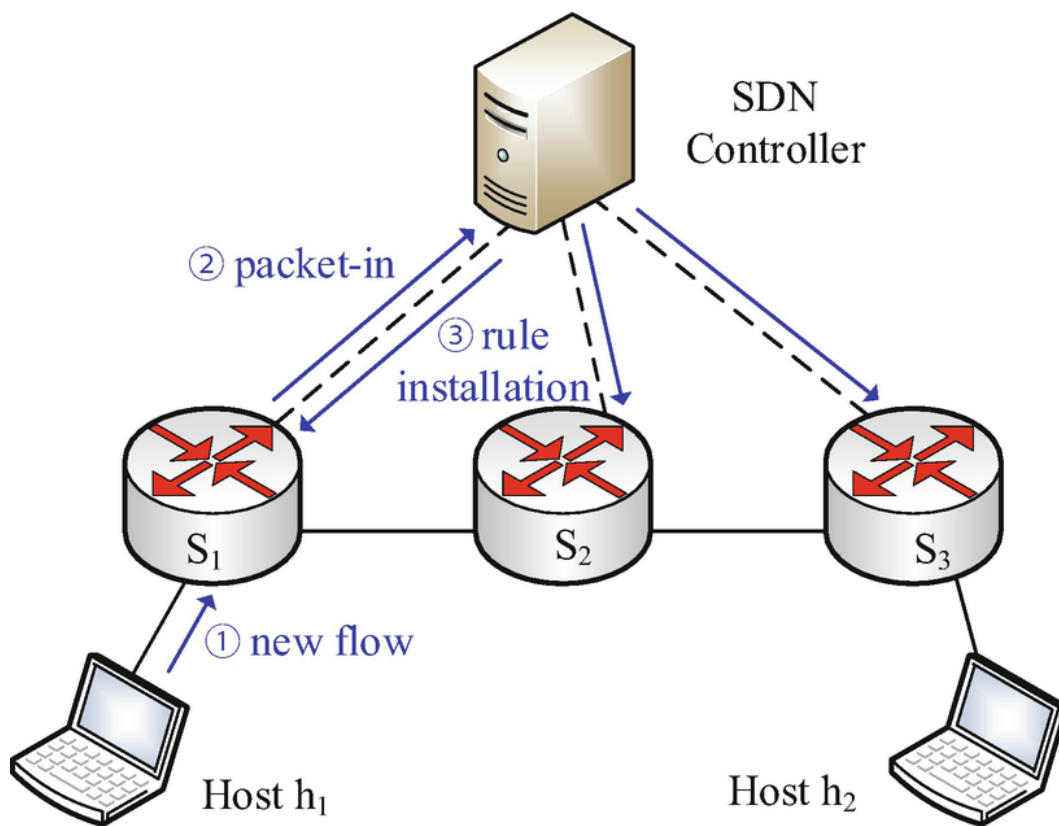


Figure XIII.4. Exemple Packet-in / Modify-State^{11*}

Messages Symétriques

Les messages symétriques peuvent être initiés par le Contrôleur ou le Switch:

- *HELLO*: Ce type de message est transmis lors de l'initiation de la connexion;

- *ECHO*: Ce type de message est utilisé comme les messages Echo request / reply ICMP pour vérifier si la destination est en ligne. Le Switch teste la connexion avec le Contrôleur par ce mécanisme.

En cas où la connexion est rompue entre le Switch / Contrôleur (le Contrôleur ne répond pas au Echo Request du Switch), le Switch commence à utiliser une Table de Flux alternative appelée *Emergency Table* et supprime les entrées installées par le Contrôleur avant la coupure (ce fonctionnement est appelée *Mode d'Urgence*). Le Switch continue à tenter de connecter au Contrôleur et en cas de réussite, le Switch bascule au mode de fonctionnement normal.

Messages Asynchrones

Ce type de message est initié lorsqu'un événement réseau est détecté. Les messages suivants tombent dans cette catégorie:

- *Packet-In*: Lorsqu'un paquet est reçu par un Switch qui ne correspond à aucune entrée de la Table de Flux ou bien l'action correspondante est égale à *CONTROLLER*, un *Packet-In* est généré et transmis vers le Contrôleur. Avant de transmettre le *Packet-In* au Contrôleur, le paquet *initial* (celui pour le quel aucune entrée de la Table de Flux ne corresponde) est placé dans le buffer et un identifiant est affecté à celui-ci. Le *Packet-In* transmis au Contrôleur contiendra l'identifiant buffer et une copie de l'entête du paquet. Lorsque le Contrôleur renvoie une réponse *Packet-Out* ou *Send-Packet*, celui-ci comprendra l'identifiant buffer.
- *Flow-Removal*: Si une entrée de la Table de Flux est supprimée à cause du Idle / Hard Timeout, un message *Flow-Removal* est transmis au Contrôleur;

Avec ces messages basiques d'OpenFlow, le Contrôleur peut gérer la QoS, la consommation d'énergie ainsi que la sécurité. Par contre, le désavantage de l'architecture SDN est le fait que toute la topologie est gérée par une seule instance (le Contrôleur). Plusieurs Contrôleur peuvent coexister dans le même réseau afin de faire face à ce problème.

XIV

Delai introduit par le chiffrement

- ### *Problem de Head of the Line Blocking*

- 94

- Surmonter ce problème conduit à HTTP/2, une initiative poussée par Google et basée sur SPDY. HTTP/2 a introduit deux fonctionnalités principales : le multiplexage et le push serveur. Le multiplexage permet l'envoi/réception synchrone de plusieurs flux de données HTTP logiques sur une seule connexion TCP, au lieu d'ajouter des connexions TCP parallèles. Le push du serveur permet aux serveurs d'anticiper les demandes de ressources et de les envoyer avant la demande du client, qui conserve toujours le pouvoir de refuser le push du serveur. Ces fonctionnalités ajoutent beaucoup d'efficacité aux interactions client serveur dans la majorité des scénarios.

2. Fonctionnalités du protocole Quic

Google a implémenté et déployé QUIC, un nouveau protocole de transport, sur Chrome ainsi que dans leurs services côté serveur (recherche Google, Gmail, Youtube, etc.). Celui-ci est basé sur UDP. En réutilisant un protocole de transport établi à l'échelle mondiale comme UDP, au lieu d'en concevoir QUIC comme un nouveau protocole de transport, il a été possible d'éviter un déploiement compliqué, voire impossible.

Les fonctionnalités intéressantes et nécessaires de TCP ont été implémentées sur le protocole QUIC lui-même, qui se situe entre UDP et HTTP, tandis qu'au niveau applicatif, HTTP a été adapté et appelé HTTP/3. En d'autres termes, les avantages de TCP (fiabilité, robustesse) sont conservés et étendus en les implémentant dans QUIC. Dans le même temps, en adoptant le protocole de transport UDP qui est plus simple comme protocole de transport, ce protocole élimine les principales limitations de TCP (les négociations lentes lors du Three Way Handshake).

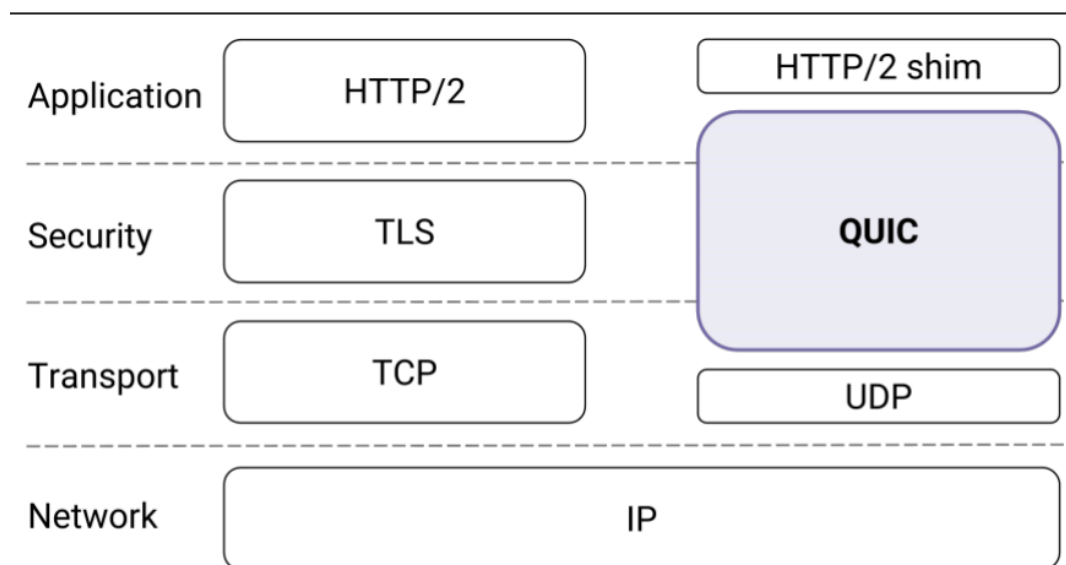


Figure XIV.1. Implémentation de Quic^{12*}

2.1. Multiplexage des flux

QUIC sépare les flux logiques au sein d'une seule connexion physique au lieu d'avoir une connexion par flux. Une connexion QUIC est liée à une paire port UDP/adresse IP et celle-ci est négociée entre deux points de terminaison, comme pour les connexions TCP. Une fois établie, la connexion est identifiée par un « *connection ID* ». Ce protocole offre aussi un contrôle de flux sur la connexion et les flux.

Lors du contrôle de flux, deux applications envoient des datagrammes *WINDOW_UPDATE* à l'homologue pour augmenter la taille de la fenêtre de contrôle de flux et des datagrammes *BLOCKED* pour indiquer qu'il a des données à écrire mais que le flux est bloqué par le contrôle de flux.

Le contrôle de flux de connexion est similaire au contrôle d'un flux individuel sauf que le contrôle de flux d'une connexion représente l'agrégats des flux rassemblés sous la même connexion. Ainsi, lorsque des datagrammes de données sans livrés pour un flux, l'offset et la quantité de données recevable sont mis à jour pour le flux et pour la connexion au même temps.

Une fois qu'une connexion QUIC est établie, chaque côté peut créer des flux et envoyer des données à l'autre extrémité. Les données des flux sont fournies dans l'ordre et de manière fiable, mais différents flux peuvent être livrés d'une manière désordonnée. En d'autres termes, chaque flux envoie des données et maintient l'ordre des données, mais les données peuvent atteindre la destination dans un ordre différent de celui que l'application les a envoyées.

En supposant que les flux A et B soient transférés du serveur au client, le flux A étant démarré en premier, puis le flux B, si le flux A perd un paquet et que le flux B n'est pas affecté, le flux B continue le transfert, tandis que le paquet perdu du flux A est retransmis. Les paquets perdus n'affectent que le flux auquel ils appartiennent.

Avec le concept de flux, une autre connexion logique au même hôte peut également être créée en même temps, sans avoir à attendre la fin de la connexion existante.

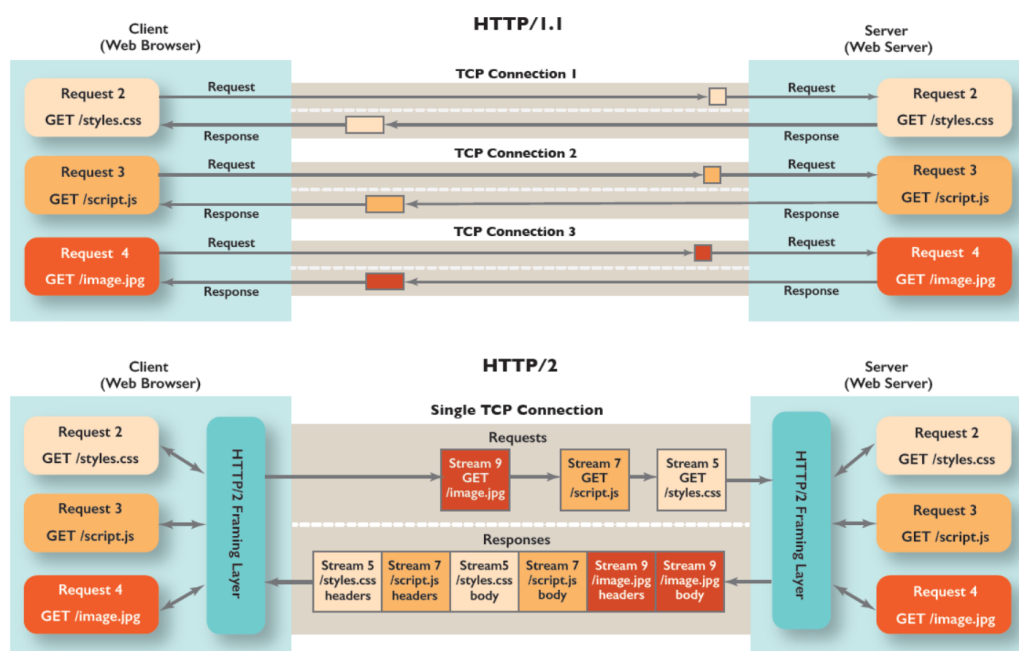


Figure XIV.2. HTTP1.1 vs HTTP2.0^{13*}

2.2. Always secure

Il n'existe pas de version en texte clair de QUIC. Une fois que les paquets de connexion initiaux sont envoyés « en clair » pour négocier les chiffrement, toutes les transmissions QUIC sont chiffrées. Les connexions QUIC sont établies avec cryptographie et sécurité selon TLS 1.3 (RFC 8446).

2.3. Reduced latency

- L'un des problèmes de l'Internet actuel est la latence/ délai (RTT, Round-Trip Time) des messages qui circulent avant l'échange de données réel. Par exemple, la latence peut atteindre 300 à 400 ms si l'on considère la distance moyenne entre l'Europe et les États-Unis. Avec le chiffrement et le three way handshake initial, le TTFB (Time-to-First-Byte, c'est-à-dire le temps nécessaire pour recevoir les premières données depuis l'envoi du GET) augmente significativement.
- Au contraire à TCP, les connexions QUIC sont des conversations uniques entre deux points de terminaison QUIC. L'établissement de la connexion combine la négociation de version avec les échanges des clés de chiffrement pour réduire le RTT.
- QUIC propose des « prises de contact rapides » 1-RTT et 0-RTT, réduisant ainsi le temps nécessaire pour négocier et établir une nouvelle connexion. Au niveau de l'utilisateur, cela promet une meilleure expérience, car le célèbre TCP three way handshake est compressée et le TTFB réduit.
- L'idée derrière 0-RTT est « une reprise de connexion à temps aller-retour nul ». Si le client et le serveur établissent une connexion TLS, une connexion ultérieure peut réutiliser les informations mises en cache lors de la première connexion. Les clients calculent les clés privées au préalable, sans contacter le serveur, mais sur la base de la connexion TLS précédente, en ignorant les transactions nécessaires pour en créer une nouvelle à zéro.
- 0-RTT a déjà été introduit dans TLS 1.3 pour les connexions basées sur TCP. L'idée ne fait référence qu'à la prise de contact TLS, car une connexion TCP est toujours nécessaire avant que les données TLS puissent être échangées, et le client doit de toute façon attendre la fin de la prise de contact TCP avant d'« accélérer » l'échange TLS avec le serveur.
- Avec QUIC, le principe est poussé plus loin : les clients peuvent envoyer des données d'application dès le premier aller-retour de connexion, sans attendre aucune prise de contact préalable. Ainsi, les requêtes HTTP sont envoyées dès que possible, les serveurs peuvent répondre et renvoyer les données plus rapidement.

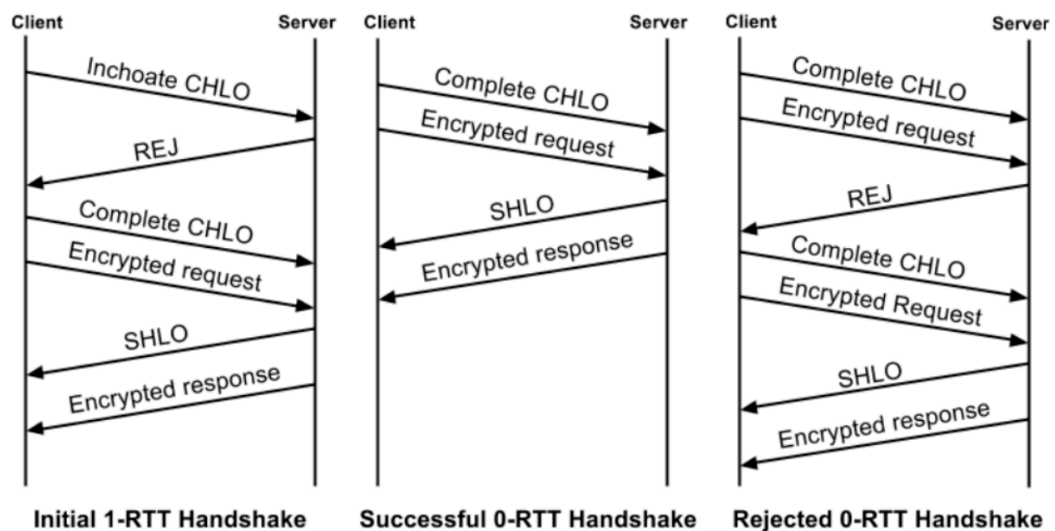


Figure XIV.1. Implémentation de la sécurité dans Quic^{14*}

2.4. Reliability

- Étant donné qu'UDP n'est pas un protocole de transport fiable, QUIC ajoute une couche par-dessus, implémentant les capacités TCP classiques. Bien que le niveau HTTP reste fidèle aux mêmes paradigmes et concepts qu'auparavant (en-têtes et corps, requête et réponse, verbes, cookies, mise en cache), des changements fondamentaux ont été apportés pour que HTTP/3 fonctionne avec QUIC comme mécanisme de transport.
- HTTP/3 configure les flux QUIC et envoie des ensembles de Datagrammes. Les types de Datagrammes les plus importants sont HEADERS (pour envoyer des en-têtes HTTP compressés), DATA (pour envoyer des contenus de données binaires), GOAWAY (pour arrêter les connexions).
- Avec HTTP/3, les flux sont fournis par le protocole QUIC. De plus, les flux sont indépendants les uns des autres en plus de l'utilisation du protocole de compression des entêtes QPACK.

3. Avantages de QUIC et utilisation du connection ID

QUIC devrait apporter de meilleures performances, principalement basées sur la capacité de récupérer plusieurs ressources simultanément. Grâce à 0-RTT, comme indiqué, le client peut commencer à demander des données beaucoup plus rapidement que par une négociation TLS complète. Le serveur Web commence à charger la page plus tôt et le navigateur reçoit les données plus rapidement. Dans l'ensemble, QUIC semble clairement surpasser TCP.

L'utilisation croissante des appareils mobiles a mis en évidence des problèmes sur les mécanismes Internet actuels. Le passage d'un réseau mobile à un autre (par exemple Wi-Fi) est un problème avec TCP, car un appareil doit établir une nouvelle connexion TCP à chaque fois que l'appareil change de réseau. Cela a grandement contribué à repenser les techniques Web et a stimulé les efforts en faveur de QUIC.

Grâce à l'introduction des identifiants de connexion, les connexions peuvent être déplacées entre différents réseaux d'une manière que TCP ne pouvait pas se permettre. Ainsi, par exemple, un téléchargement en cours, qui doit être déplacé d'une connexion au réseau cellulaire vers une connexion WiFi plus rapide lorsque l'utilisateur se déplace vers un emplacement WiFi, devrait pouvoir « survivre » et continuer également sur le réseau cible.

Références



- [01] *The OSI model explained in simple terms*, 2017 <https://tomanagle.medium.com/the-osi-model-explained-in-simple-terms-2abc3c7adadc>
- [02] *OSI Model Layers — “Explained”*, 2017 <https://medium.com/theleanprogrammer/osi-model-layers-explained-ee1d43058c1f>
- [03] *The TCP/IP Guide: A Comprehensive, Illustrated Internet Protocols Reference*, 2017 <http://www.tcpipguide.com/free/index.htm>
- [04] *Les réseaux*, 2017 <https://rmdiscala.developpez.com/cours/LesChapitres.html/Cours1/Chap1.7.htm>
- [05] *Packet Guide to Core Network Protocols* Bruce Hartpence, O'REILLY, 2011, ISBN: 978-1-449-30653-3
- [06] *IPv6 Extension Headers Review and Considerations*, 2017 https://www.cisco.com/en/US/technologies/tk648/tk872/technologies_white_paper0900aecd8054d37d.html
- [07] *Understanding TCP/IP A clear and comprehensive guide to TCP/IP protocols* Libor Dostálek, Alena Kabelová, Packt, 2006, ISBN: 1-904811-71-X
- [09] *What is FTP*, 2017 <https://www.techtarget.com/searchnetworking/definition/File-Transfer-Protocol-FTP>
- [10] *A Survey on the Security of Stateful SDN Data Planes* T. Dargahi et al, IEEE Communications Surveys & Tutorials, 2017, vol. 19, no. 3, pp. 1701-1725
- [11] *Software Defined Networks A Comprehensive Approach 1st edition* Paul Goransson, Chuck Black, Morgan Kaufmann, 2014, ISBN : 978-0124166752

[13] *Efficient High
Performance Protocols
For Long Distance Big
Data File Transfer*

AlMobayed Fadi, 2018.

*Optimisation de l'ingénierie
de HTTP1 à HTTP3, 2024*

<https://www.eet-china.com/mp/a247097.html>

*Reducing Overhead Of
Self-Stabilizing Byzantine
Agreement Protocols For
Blockchain Using Http/3
Protocol: A Perspective
View*

Nur Arifin Akbar et al, SINERGI, 2021 Vol 25, No 3